

智慧製造導論

Introduction to
Smart
Manufacturing

編著：李世淵
(機器人叫獸／ロボット教授)



TRADITION
MEETS
TECHNOLOGY
FOR A BRIGHTER
ISLAND

第五篇

製造資料科學與工業人工智慧

篇章導讀：第五篇 製造資料科學與工業人工智慧

第四篇讓工廠有了資訊流、數位孿生與數位主線，也因此累積了大量的資料。第五篇要回答的是：這些資料如何變成更好的品質、更少的停機與更準時的交貨？本篇由六章組成，從資料科學的基本功出發，經過資料工程與工業分析，再進入視覺檢測、品質、維護與排程四個最常見的工業 AI 應用。它是全書篇幅最重、程式最多的一篇，也是「智慧」二字最直接落地的地方。

本篇要回答的問題

1. 如何以統計與機器學習從製造資料中得到可信的結論？效果量、檢定力、混淆因子與資料洩漏為什麼重要？CNN、LSTM 與 Transformer 在製造中各適合什麼問題？一個工業 ML 專案如何從問題定義走到部署？
2. 製造資料有哪些類型？如何情境化、檢查品質、壓縮與儲存？如何從振動與多變量製程資料中偵測異常、找出根因？資料治理、語意層與 MLOps 如何讓分析長期有效？
3. 視覺檢測的成敗為什麼首先取決於光學？分類、偵測、分割與異常檢測如何選擇？資料、標註與增強如何影響模型？門檻如何依漏檢與誤判的成本決定？
4. 品質管理如何從檢驗、管制、預防走向預測與閉環？SPC、MSA、DOE 與 Run-to-Run 控制如何結合 AI，形成智慧品質？
5. 維護策略如何從壞了才修走向預測維護？剩餘壽命預測的不確定性如何影響決策？物理模型與資料模型如何結合？遠程運維與閉環最佳化如何創造價值？
6. 排程與派工如何在交期、換線、在製品與能源之間取捨？派工規則、求解器、後設啟發式與學習型方法各適合什麼情境？晶圓廠的重入式流程如何派工？

六章的分工

第 22 章〈製造資料科學基礎〉是全篇的方法論基礎。本章從敘述統計、機率分布、假設檢定、效果量與檢定力談起；以辛普森悖論說明混淆因子的陷阱；比較監督式與非監督式學習，示範過度配適與資料洩漏；以 NumPy 從零實作一維 CNN 與注意力機制，說明 CNN、LSTM 與 Transformer 的

原理；最後以成本導向的門檻與資料漂移監控，建立工業 ML 管線的完整概念。

第 23 章〈製造資料工程與工業分析〉處理資料從產生到可用的過程。本章說明製造資料的類型、情境化與品質檢查；介紹歷史資料庫、旋轉門壓縮、串流與資料湖倉；以包絡分析找出軸承缺陷頻率，以主成分分析的 T^2 與 SPE 偵測多變量異常並追查根因；討論資料治理、資料契約與語意層；最後以漂移偵測、冠軍挑戰者與影子部署說明 MLOps。

第 24 章〈機器視覺與 AI 品質檢測〉是第一個應用章。本章從照明、相機與像素解析度的設計談起，以平場校正說明光學的重要；介紹 CNN、Vision Transformer 與 YOLO，實作 IoU、NMS 與 AP；以只用良品訓練的異常檢測產生異常熱圖；以資料增強示範穩健性；最後以雙門檻與人工複檢，依漏檢與誤判的成本設計檢測流程。

第 25 章〈智慧品質：SPC、DOE 與閉環控制〉把品質管理帶進智慧製造。本章回顧 TQM、精實、六標準差與 QMS；以 $\bar{X}$ -R 與 EWMA 管制圖、製程能力與量具 GR&R 建立可信的量測與管制；以因子實驗與反應曲面找出最佳條件，以 Run-to-Run 控制補償 CMP 的漂移；提出本書整理的 DFSQ 六階段閉環架構；最後以良率模型與晶圓圖分析半導體與電子組裝的品質。

第 26 章〈預測維護、遠程運維與製程最佳化〉處理設備的可靠度與閉環決策。本章以韋伯分布分析壽命資料、求出最佳更換週期；以模擬比較定期更換、狀態門檻與剩餘壽命預測三種策略；比較物理、資料與混合模型的外推並量化不確定性；說明遠程運維的架構、資安與服務模式；最後以 MPC 與強化學習示範製程與維護的閉環決策。

第 27 章〈智慧排程、派工與生產控制〉是本篇的收尾。本章介紹流程型、零工式、彈性零工式與重入式流程；以單機範例說明交期與換線的取舍；以經典的 FT06 題目比較派工規則、限制規劃求解器與模擬退火；以交叉熵法學習派工規則，討論強化學習與孿生排程；最後以重入式產線的模擬說明晶圓廠派工、Little 定律與 X 因子。



圖 V-1 第五篇第六章的學習主線

本篇在全書中的位置

如果說第四篇讓工廠有了「數位分身」與「記憶」，第五篇就是讓工廠學會「思考」。第 22、23 章是方法與資料的基礎，第 24 到 27 章則對應工廠最核心的四個問題：品質看得見嗎（視覺檢測）、品質穩得住嗎（智慧品質）、設備撐得住嗎（預測維護）、生產排得好嗎（排程派工）。對照第 5 章的參考架構，這些應用位於 ISA-95 的第 3 層，向下依賴第 11 章的工業通訊與邊緣運算、第 17 章的 MES 與主資料、第 19 至 21 章的變生與模擬，向上則服務企業的決策。

本篇也為後面各篇鋪路：第 23 章的語意層與知識圖譜、第 26 章的閉環決策與第 27 章的學習型排程，是第六篇企業 AI 大腦與代理式 AI 的基礎；遠程運維與服務模式，將在第 32 章的服務型製造深化；排程的能源目標與第 34 章的綠色製造相連；AI 的治理、可信與資安，則在第 35、36 章完整討論。貫穿本篇的主題是「可信的閉環」：資料要可信、量測要可信、模型要經過驗證並量化不確定性，決策要有權限、限制與紀錄。本篇的範例程式中，有不少結果並不如預期——混合模型沒有改善外推、預測維護仍有少量故障、模擬退火距最佳仍有差距——這些都被保留下來，因為認識方法的限制，與認識它的能力同樣重要。

閱讀建議

第五篇橫跨統計、機器學習、影像處理、品質工程、可靠度、控制與作業研究，背景不同的讀者可以有不同的重點。資訊與 AI 背景的讀者，第 22 章較為熟悉，可以把時間留給第 23 章的資料工程與第 24 到 27 章的領域知識——工業 AI 的難處往往不在模型，而在光學、量測、物理與現場的限

制；機械、工業工程與品質背景的讀者，建議仔細閱讀第 22 章的統計與機器學習基礎，再以第 25 到 27 章串連熟悉的品質、維護與排程方法；電機與控制背景的讀者，可以從第 23 章的頻譜分析與第 26 章的 MPC 切入。課堂教學時建議安排六到八週，每章約一週，第 22、23 章可視需要延長。

實作方面，建議至少完成以下六項：第 22 章的資料洩漏實驗與一維 CNN、第 23 章的包絡分析或 PCA 異常偵測、第 24 章的平場校正與異常熱圖、第 25 章的 EWMA 管制圖與 Run-to-Run 控制、第 26 章的維護策略模擬、第 27 章的 FT06 求解與重入式產線模擬。這六項串起來，正好走過「資料、分析、檢測、品質、維護、排程」一條完整的工業 AI 路線。本篇的程式都只需要 NumPy、SciPy、scikit-learn 等常用套件，第 27 章另需安裝開源的 OR-Tools；相關範例程式將於本書的實驗倉庫中維護。

對高中 AI 機器人社團而言，第 24 章的照明實驗與資料增強最容易上手：用手機與檯燈就能體會「光打對了，AI 才看得見」；第 25 章的紙飛機 DOE 與第 27 章的派工規則比賽，也很適合在社團中分組競賽。

讀完本篇後，建議你挑選一個你熟悉的製造問題，試著回答：資料從哪裡來、量得準嗎、有沒有情境？要回答的是統計問題、預測問題還是決策問題？模型的誤差與不確定性有多大、在什麼範圍內可信？決策的門檻依據什麼成本？誰批准自動執行、上線後如何監控與更新？能回答這些問題，你就已經掌握了製造資料科學與工業 AI 的核心能力。

表 V-1 第五篇目錄

章	節
第 22 章 製造資料科學基礎	22.1 敘述統計、機率分布與假設檢定
	22.2 探索性資料分析與視覺化
	22.3 監督式、非監督式學習與泛化
	22.4 深度學習：CNN、RNN、Transformer 概要
	22.5 從問題定義到可部署模型：工業 ML 管線
第 23 章 製造資料工程與工業分析	23.1 製造資料的類型、情境化與資料品質
	23.2 Historian、串流、事件資料與資料湖倉

章	節
	23.3 時序特徵、頻譜分析、異常偵測與根因分析
	23.4 資料治理、語意層與工業知識圖譜
	23.5 MLOps、ModelOps 與模型漂移監測
第 24 章 機器視覺與 AI 品質檢測	24.1 工業影像、光學、照明與相機配置
	24.2 CNN、Vision Transformer 與 YOLO
	24.3 分類、偵測、分割與異常檢測
	24.4 資料蒐集、標註、增強與合成影像
	24.5 Precision、Recall、mAP 與漏檢、誤判成本
第 25 章 智慧品質：SPC、DOE 與閉環控制	25.1 TQM、精實、六標準差與 QMS
	25.2 SPC、管制圖與製程能力指標
	25.3 實驗設計、反應曲面與 Run-to-Run 控制
	25.4 Smart Quality 與 DFSQ 六階段
	25.5 良率分析、製程內檢測與半導體、電子組裝案例
第 26 章 預測維護、遠程運維與製程最佳化	26.1 可靠度、可用度、可維護度與維護策略演進
	26.2 狀態監測、故障診斷與剩餘壽命預測
	26.3 物理資訊 AI、混合模型與不確定性
	26.4 遠程運維系統架構與服務模式
	26.5 MPC、最佳化與強化學習的閉環決策
第 27 章 智慧排程、派工與生產控制	27.1 Flow Shop、Job Shop、FJSP 與重入式流程
	27.2 目標函數：Makespan、交期、WIP、換線與能源
	27.3 派工規則、MILP 與後設啟發式方法
	27.4 機器學習、強化學習與孿生排程
	27.5 晶圓廠與封測廠智慧派工案例

第五篇 製造資料科學與工業人工智慧

第 22 章

製造資料科學基礎

編著 李世淵（機器人叫獸） | 助教 李天宇、李宇晴

本章重點

- 敘述統計、機率分布、假設檢定、效果量與檢定力
- 探索性資料分析、辛普森悖論與混淆因子
- 監督式與非監督式學習、過度配適與資料洩漏
- 以 NumPy 實作一維 CNN 與注意力機制
- CNN、LSTM、Transformer 的原理與製造應用
- 工業 ML 管線：成本導向門檻與資料漂移監控

叫獸開講：資料說溫度越高越好，可是.....

有一年，一家射出成形廠的品保主管興沖沖地拿著一張散布圖來找我。他把過去三個月的生產資料全部匯出，畫出模具溫度與良率的關係：點雲清楚地往右上方延伸，相關係數高達 0.8 以上。「老師，資料說得很清楚，溫度越高良率越好！我們打算把所有機台的設定溫度都調高十度。」

我請他先別急，把資料依機台分開再畫一次。這一分，畫面完全變了：每一台機台內部，溫度越高，良率反而越低。原來，工廠裡有三代機台：新機台的設定溫度本來就比較高，加上設備本身較精密，良率也比較好；舊機台溫度設得低，良率也差。把三代機台混在一起看，「新機台」這個隱藏因素讓溫度和良率看起來正相關；分開來看，才看到溫度真正的影響。如果照原本的計畫把溫度調高，每一台機台的良率都會下降。

這位主管後來說，這是他學到最寶貴的一課：資料不會說謊，但資料也不會自己說話；怎麼問問題、怎麼切資料、怎麼驗證結論，比畫出一張漂亮的圖重要得多。

這一章是第五篇〈製造資料科學與工業人工智慧〉的第一章，要建立分析製造資料所需的共同基礎：敘述統計、機率分布與假設檢定；探索性資料分析與視覺化；監督式與非監督式學習，以及模型能否推廣到新資料的「泛化」；深度學習中的 CNN、RNN 與 Transformer；最後，從問題定義到可部署模型的工業機器學習管線。往後幾章的品檢、品質、維護與排程，都會建立在這一章的基礎上。

（說明：本節情境由作者協助企業分析生產資料的經驗綜合改寫，人物與細節已調整。）

學習目標

讀完本章後，你應該能夠：

1. 以敘述統計描述製造資料，說明常見的機率分布，並以假設檢定、信賴區間與效果量比較兩組資料，理解檢定力與樣本數的關係。
2. 進行探索性資料分析，辨識辛普森悖論、混淆因子與異常值，並選擇合適的視覺化方式。
3. 區分監督式、非監督式與半監督式學習，說明過度配適與泛化，並能正確地切分資料以避免資料洩漏。

4. 說明 CNN、RNN / LSTM 與 Transformer 的基本原理與適用資料，並以程式實作一維 CNN 與注意力機制。
5. 說明工業機器學習管線的各個階段，依成本決定判定門檻，並以族群穩定指標監控資料漂移。
6. 說明製造資料的特性與限制：資料少、不平衡、有批次結構、會漂移，以及領域知識的重要性。

核心概念

表 22-1 本章核心名詞中英對照

中文名詞	英文	一句話說明
敘述統計	Descriptive Statistics	以平均、中位數、標準差等數字摘要資料
假設檢定	Hypothesis Testing	以機率判斷觀察到的差異是否可能只是偶然
p 值	p-value	若虛無假設為真，觀察到至少這麼極端結果的機率
效果量	Effect Size	差異的實際大小，與樣本數無關
檢定力	Statistical Power	真的有差異時，檢定能夠檢出的機率
探索性資料分析	Exploratory Data Analysis, EDA	以統計與圖形了解資料、發現問題與假設
辛普森悖論	Simpson's Paradox	分組與合併後的趨勢方向相反的現象
監督式學習	Supervised Learning	以有標籤的資料學習輸入到輸出的對應
非監督式學習	Unsupervised Learning	在沒有標籤的資料中找出結構，例如分群
過度配適	Overfitting	模型記住訓練資料的雜訊，對新資料表現變差
資料洩漏	Data Leakage	測試資料的資訊不當地進入訓練，使評估過度樂觀
卷積神經網路	Convolutional Neural Network, CNN	以共享權重的卷積核擷取局部特徵的網路
長短期記憶網路	Long Short-Term Memory, LSTM	以閘門控制記憶、處理序列資料的循環網路
注意力機制	Attention	依相關程度加權彙整序列中其他位置資訊的方法
資料漂移	Data Drift	上線後資料分布與訓練時不同

22.1 敘述統計、機率分布與假設檢定

22.1.1 製造資料的特性

製造資料與網路、金融資料有幾個不同的特性，值得在一開始就認識：

- 資料少而貴：一個新產品可能只有幾十批試產資料；破壞性的檢驗，每一筆資料都要犧牲一件產品。
- 標籤稀少且不平衡：良品很多，不良品很少，而且每一種缺陷又不一樣。
- 有結構：資料依機台、批料、班別、模具分群，同一群的資料彼此相似，不是獨立的樣本。
- 會變：設備老化、保養、換料、換季，資料的分布隨時間改變。
- 有物理：資料背後有物理與製程的機制，領域知識常常比演算法更重要。

這些特性會一再出現在本章的例子中，也決定了製造資料科學的許多做法。

22.1.2 敘述統計

描述一組資料，最基本的是「中心」與「散布」。中心可以用平均數或中位數表示；散布可以用標準差、全距或四分位距（第 75 與第 25 百分位數之差）表示。平均數與標準差容易受極端值影響，中位數與四分位距則較穩健。在製造中，散布往往比中心更重要：平均值偏了，可以調整設定；變異大了，就要找出變異的來源（第 25 章）。

製程能力指標 Cpk 結合了中心與散布，衡量製程與規格之間的餘裕：

$$Cpk = \min(USL - \bar{x}, \bar{x} - LSL) \div 3s$$

其中 USL 與 LSL 為規格上、下限， $\bar{x}$ 為平均， s 為標準差。 Cpk 越大，表示越不容易超出規格；一般常以 1.33 作為基本要求。

22.1.3 常見的機率分布

- 常態分布：許多量測值受到大量小因素的共同影響，近似常態分布，例如尺寸、重量。
- 二項分布與卜瓦松分布：計數資料，例如一批中的不良數、一片晶圓上的缺陷數。
- 指數分布與威布爾分布：時間資料，例如故障間隔與壽命（第 12、26 章）。

- 對數常態分布：只能為正值、右偏的資料，例如粒徑、修復時間。

選錯分布，結論就可能出錯。例如以常態分布估計不良率的極端尾端，常會低估真正的風險；壽命資料用常態分布，可能得到負的壽命。

22.1.4 假設檢定、信賴區間與效果量

兩台 CNC 加工同一種軸，規格為 20 ± 0.015 公釐。兩台各取 30 件量測，機台 B 的平均好像比較大。這個差異是真的，還是抽樣的偶然？假設檢定回答這個問題：先假設兩台沒有差異（虛無假設），再計算在這個假設下，觀察到至少這麼大差異的機率，也就是 p 值。p 值很小，就表示「沒有差異」的假設難以解釋資料。

但 p 值只回答「是不是偶然」，不回答「差多少」與「重不重要」。因此還需要：

- 信賴區間：差異可能落在什麼範圍。
- 效果量：差異相對於變異的大小，例如 Cohen's d，與樣本數無關。
- 檢定力：如果真的有某個大小的差異，這個實驗有多少機會檢出它。

下面的程式比較兩台機台，並以模擬估計檢定力：

```
import numpy as np
from scipy import stats
# 敘述統計與假設檢定：兩台 CNC 加工的軸徑 (mm) 是否有差異？差異大到值得處理嗎？
rng = np.random.default_rng(1)
a = rng.normal(20.000, 0.004, 30)          # 機台 A, 30 件
b = rng.normal(20.003, 0.005, 30)          # 機台 B, 30 件 (平均偏大 3 μm)
LSL, USL = 19.985, 20.015                 # 規格 20 ± 0.015 mm
def describe(x, name):
    q1, q3 = np.percentile(x, [25, 75]); s = x.std(ddof=1)
    cpk = min(USL - x.mean(), x.mean() - LSL) / (3 * s)
    print(f"{name}: 平均 {x.mean():.4f}、中位數 {np.median(x):.4f}、標準差 {s*1000:.2f} μm、  
四分位距 {(q3-q1)*1000:.2f} μm、Cpk {cpk:.2f}")
describe(a, "機台 A"); describe(b, "機台 B")
print(f"常態性 Shapiro-Wilk p 值: A {stats.shapiro(a).pvalue:.2f}, B  
{stats.shapiro(b).pvalue:.2f}")
t = stats.ttest_ind(a, b, equal_var=False)          # Welch t 檢定
d = (b.mean() - a.mean()) / np.sqrt((a.var(ddof=1) + b.var(ddof=1)) / 2)
print(f"Welch t 檢定: t = {t.statistic:.2f}, p = {t.pvalue:.4f}; 效果量 Cohen's d =  
{d:.2f}")
print(f"Mann-Whitney U 檢定 (不假設常態): p = {stats.mannwhitneyu(a, b).pvalue:.4f}")
lo, hi = stats.ttest_ind(b, a, equal_var=False).confidence_interval(0.95)
print(f"平均差 95% 信賴區間: {lo*1000:.2f} ~ {hi*1000:.2f} μm")
for n in [30, 100, 300, 600]:
    hits = sum(stats.ttest_ind(rng.normal(0, 4.5, n), rng.normal(1, 4.5, n)).pvalue < 0.05 for _ in range(2000))
    print(f"差異 1 μm、每組 {n} 件: 檢定力約 {hits/2000:.0%}")
```

執行結果：

表 22-2 兩台機台的比較

項目	機台 A	機台 B
平均 (mm)	19.9997	20.0028
標準差	3.37 μm	4.56 μm
四分位距	3.26 μm	6.37 μm
Cpk	1.46	0.89

Welch t 檢定得到 $t = -3.00$ 、 $p \approx 0.0041$ ；不假設常態的 Mann-Whitney U 檢定 $p \approx 0.0122$ ，兩者都顯示差異不太可能是偶然。平均差的 95% 信賴區間約為 $1.03 \sim 5.18 \mu\text{m}$ ，效果量 Cohen's d 約 0.77，屬於中到大的差異。更重要的是工程意義：機台 B 的 Cpk 只有 0.89，低於 1.33 的一般要求，即使平均差只有約 $3 \mu\text{m}$ ，也值得處理。

檢定力的模擬則提醒我們另一面：若真實差異只有 $1 \mu\text{m}$ ，每組 30 件時檢定力只有約 13%，也就是說，十次實驗中有將近九次會得出「沒有顯著差異」的結論；每組要約 300 件，檢定力才達到約 77%，600 件時約 96%。因此，「沒有顯著差異」不等於「沒有差異」，可能只是樣本太少。另一方面，在資料量很大的工廠中，幾乎任何微小的差異都會「顯著」，這時更要看效果量與工程意義，而不是只看 p 值。

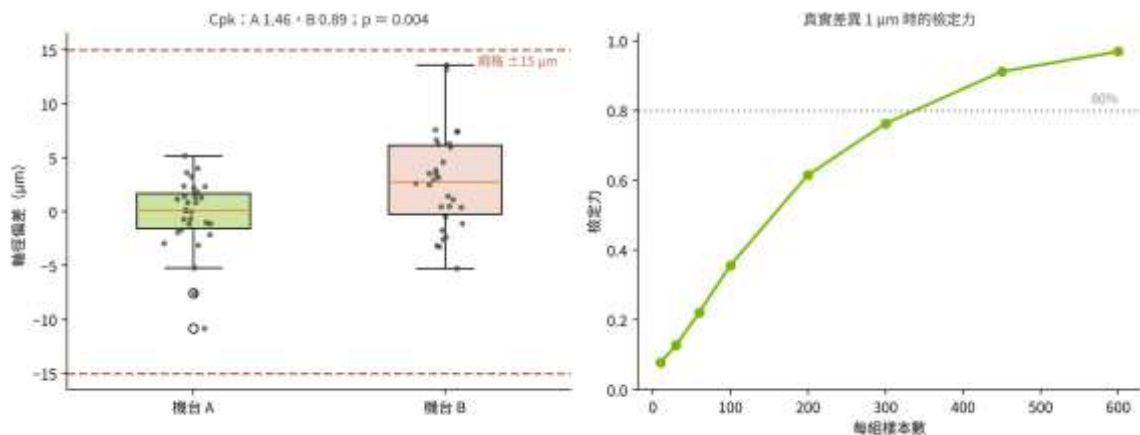


圖 22-1 兩台機台的軸徑分布與檢定力

程式中機台 A 的常態性檢定 p 值約 0.02，表示以 30 件樣本來看，資料與常態分布有些偏離。這

在小樣本中並不罕見，也是程式同時提供不假設常態的 Mann-Whitney 檢定的原因。實務上，應先畫圖看資料的形狀，再選擇方法，這正是下一節探索性資料分析的精神。

22.1.5 多重比較與 p 值的誤用

在製造資料中，常常同時檢查幾十甚至幾百個參數與品質的關係。若每一個都以 0.05 為顯著水準，即使所有參數都與品質無關，平均每 20 個也會有 1 個「顯著」。這稱為多重比較問題。對策包括：調整顯著水準（例如 Bonferroni 校正、控制偽發現率），以及把資料分成探索用與確認用兩部分，在探索資料上找到的假設，要在另一份資料上確認。

22.2 探索性資料分析與視覺化

22.2.1 先看資料，再下結論

統計學家 John Tukey 在 1977 年出版的《探索性資料分析》中，強調在建立模型與檢定之前，先以圖形與簡單的統計了解資料。統計學家 Francis Anscombe 在 1973 年構造了四組著名的資料：它們的平均、變異數、相關係數與迴歸線幾乎完全相同，畫成散布圖卻完全不同，有的是直線、有的是曲線、有的被一個極端值主導。這四組資料常被稱為安斯庫姆四重奏，提醒人們：只看摘要數字，可能完全誤解資料。

製造資料的探索性分析，通常從以下幾個問題開始：

- 資料的完整性：缺值、重複、單位不一致、時間戳記錯誤（第 23 章）。
- 每個變數的分布：直方圖、箱形圖，有沒有異常值、多峰、截斷。
- 時間上的變化：趨勢、週期、階躍，對應保養、換料、換班。
- 變數之間的關係：散布圖、相關矩陣，以及依機台、批料、班別分組的比較。

表 22-3 製造資料常用的視覺化

目的	圖形	製造上的例子
看分布	直方圖、箱形圖、小提琴圖	尺寸分布、各機台的比較
看時間	趨勢圖、管制圖	溫度漂移、良率變化（第 25 章）

目的	圖形	製造上的例子
看關係	散布圖、相關熱圖、配對圖	製程參數與品質
看組成	柏拉圖、堆疊長條圖	缺陷類別排序
看空間	熱圖、晶圓圖	晶圓上的缺陷分布
看高維	主成分分析、t-SNE 投影	操作模式、異常批次

22.2.2 辛普森悖論

叫獸開講中的情境，就是辛普森悖論：分組來看的趨勢，在合併之後反轉。下面的程式重現這個情境：三代機台的設定溫度與基本良率不同，但每一台機台內部，溫度每升高 1 °C，良率下降 0.15 個百分點：

```
import numpy as np
# 探索性資料分析：辛普森悖論—整體看「溫度越高良率越高」，分機台看卻是「溫度越高良率越低」
rng = np.random.default_rng(3)
rows = []
for m, (t0, y0) in {"M1": (180, 90.0), "M2": (195, 94.0), "M3": (210, 98.0)}.items(): # 新機台設定溫度較高、本身良率也較高
    T = rng.normal(t0, 4, 200)
    Y = y0 - 0.15 * (T - t0) + rng.normal(0, 0.8, 200) # 同一機台內：溫度每升 1 °C，良率降 0.15%
    rows += [(m, t, y) for t, y in zip(T, Y)]
M = np.array([r[0] for r in rows]); T = np.array([r[1] for r in rows]); Y = np.array([r[2] for r in rows])
slope = lambda x, y: np.polyfit(x, y, 1)[0]
print(f"全部資料：相關係數 {np.corrcoef(T, Y)[0,1]:+.2f}，斜率 {slope(T, Y):+.3f} %/°C")
for m in ["M1", "M2", "M3"]:
    k = M == m
    print(f" {m}: 平均溫度 {T[k].mean():.0f} °C，平均良率 {Y[k].mean():.1f}%，機台內斜率 {slope(T[k], Y[k]):+.3f} %/°C")
Tc = np.concatenate([T[M == m] - T[M == m].mean() for m in ["M1", "M2", "M3"]])
Yc = np.concatenate([Y[M == m] - Y[M == m].mean() for m in ["M1", "M2", "M3"]])
print(f"控制機台後的斜率 {slope(Tc, Yc):+.3f} %/°C")
```

執行結果：把全部 600 筆資料混在一起，溫度與良率的相關係數高達約 +0.85，迴歸斜率約為每 °C +0.225 個百分點，看起來「溫度越高越好」。依機台分開後，三台的機台內斜率分別約為 -0.144、-0.139 與 -0.166，方向完全相反；把每台機台各自減去平均之後，共同的斜率約為 -0.150，正好接近真正的效果 -0.15。

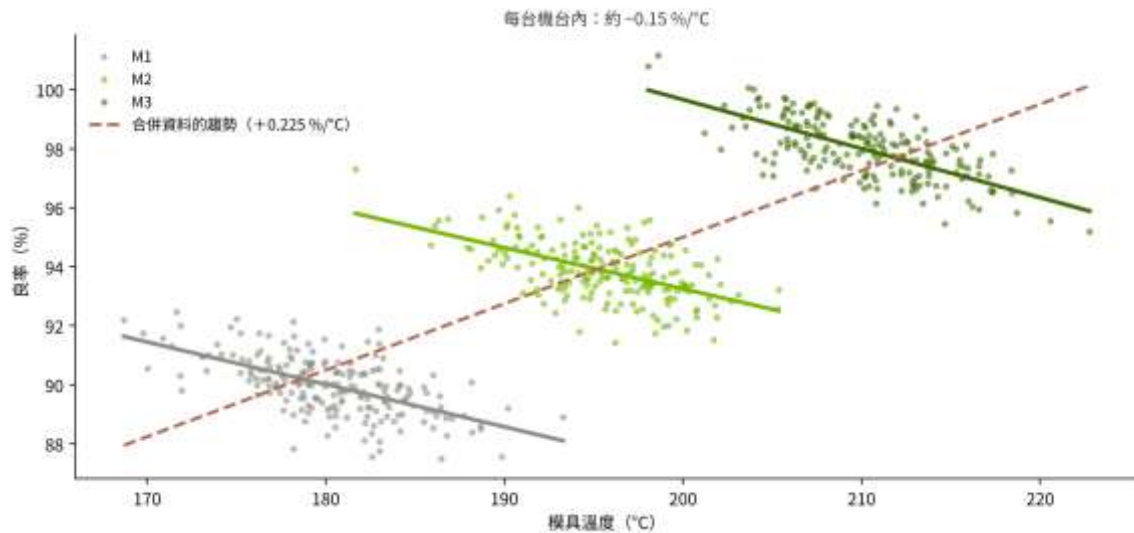


圖 22-2 辛普森悖論：合併與分組的趨勢相反

機台在這裡是一個「混淆因子」：它同時影響溫度（新機台設得高）與良率（新機台本身較好）。只要沒有考慮它，溫度就會「冒領」機台的功勞。在製造資料中，類似的混淆因子到處都是：機台、模具、批料、班別、季節、產品組合。辨識它們，需要的是領域知識，而不是更複雜的演算法。

這也說明了觀察資料與實驗的根本差異：從現場累積的資料，只能看到「相關」；要確認「因果」，最可靠的方法是在控制其他條件下主動改變溫度，也就是第 25 章的實驗設計。當實驗不可行時，至少要在分析中控制已知的混淆因子，並對結論保持謹慎。

22.2.3 視覺化的原則

好的製造資料視覺化，應該讓讀者一眼看出重點，並且不誤導：

- 依問題選擇圖形，並依製造的結構分組或分色（機台、批料、時間）。
- 標示規格界限、管制界限與目標值，讓讀者知道「多少才算好」。
- 座標軸從合理的位置開始，避免誇大或縮小差異；單位要清楚。
- 顯示資料本身，而不只是摘要：在箱形圖上疊加資料點，在平均值上加上信賴區間。
- 時間序列標示重要事件，例如保養、換料、參數調整。

22.3 監督式、非監督式學習與泛化

22.3.1 機器學習的三種型態

表 22-4 機器學習的型態與製造上的例子

型態	資料	任務	製造上的例子	本書相關章節
監督式學習	有標籤 (答案)	分類、迴歸	缺陷判定、虛擬量測、剩餘壽命預測	第 21、24、26 章
非監督式學習	沒有標籤	分群、降維、異常偵測	操作模式分群、異常批次偵測	第 23、26 章
半監督與自監督	少量標籤、大量無標籤	以無標籤資料預訓練，再以少量標籤微調	瑕疵影像的預訓練模型	第 24 章
強化學習	與環境互動的獎勵	學習決策策略	排程、機器人控制	第 27、30 章

監督式學習是最常見的型態：給模型大量「輸入—答案」的配對，讓它學會從輸入預測答案。分類預測的是類別（良品或不良品），迴歸預測的是數值（膜厚、尺寸）。常用的方法包括線性與邏輯斯迴歸、決策樹、隨機森林與梯度提升樹、支持向量機與神經網路。在製造的表格資料上，梯度提升樹與隨機森林常有很好的表現；影像、訊號與文字資料，則以深度學習為主。

非監督式學習在沒有標籤的資料中找結構。分群把相似的樣本分在一起，例如把設備的運轉資料分成幾種操作模式；降維把高維資料投影到少數幾個方向，例如主成分分析；異常偵測找出與大多數資料不同的樣本，在不良品稀少、難以標註時特別有用（第 26 章）。

22.3.2 過度配適與泛化

模型的目的不是在訓練資料上表現好，而是在「沒見過的新資料」上表現好，這種能力稱為泛化。太簡單的模型學不到資料中的規律（欠配適）；太複雜的模型則連雜訊都記了下來（過度配適），在訓練資料上近乎完美，在新資料上卻變差。

22.3.3 資料洩漏：製造資料最常見的陷阱

評估泛化的標準做法，是把資料切成訓練集與測試集，或以交叉驗證輪流當測試集。但製造資料有結構：同一批料、同一個模具、同一天生產的樣本彼此相似。如果隨機切分，同一批料的樣本可能同時出現在訓練集與測試集中，模型只要「認得這批料」就能答對，評估結果就會過度樂觀。這是資料洩漏最常見的形式之一。上線後，模型面對的是「新的批料」，表現就會比評估時差。

下面的程式模擬一家射出成形廠的缺陷預測：40 批料、每批 25 件，每批料有自己的「指紋」（水分、黏度造成的特徵偏移），部分批料本身較容易出缺陷。程式比較隨機切分與依批料分組切分的交叉驗證，並觀察決策樹深度對訓練與測試準確率的影響；最後以 K-means 對無標籤的運轉資料分群：

```
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import KFold, GroupKFold, cross_val_score, train_test_split
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
# 監督式學習、泛化與資料洩漏：預測射出件是否有缺陷；同一批料的樣本彼此相似，隨機切分會高估準確率
rng = np.random.default_rng(7)
n_lot, per = 40, 25
lot = np.repeat(np.arange(n_lot), per)
lot_sig = rng.normal(0, 1, (n_lot, 6)) # 每批料的「指紋」（水分、黏度...
造成的特徵偏移)
lot_bad = rng.random(n_lot) < 0.4 # 部分批料本身比較容易出缺陷
X = lot_sig[lot] + rng.normal(0, 0.3, (n_lot * per, 6)) # 6 個製程特徵
p = 1 / (1 + np.exp(-(1.2 * X[:, 0] - 0.8 * X[:, 1] + 2.0 * lot_bad[lot] - 1.0)))
y = (rng.random(len(p)) < p).astype(int)
rf = RandomForestClassifier(n_estimators=200, random_state=0)
acc_rand = cross_val_score(rf, X, y, cv=KFold(5, shuffle=True, random_state=0)).mean()
acc_group = cross_val_score(rf, X, y, cv=GroupKFold(5), groups=lot).mean()
print(f"缺陷率 {y.mean():.0%}；隨機切分交叉驗證準確率 {acc_rand:.1%}；依批料分組切分
{acc_group:.1%}")
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0)
for depth in [1, 2, 3, 5, 8, 12, None]:
    t = DecisionTreeClassifier(max_depth=depth, random_state=0).fit(Xtr, ytr)
    print(f" 決策樹深度 {str(depth):>4}：訓練 {t.score(Xtr, ytr):.1%}，測試 {t.score(Xte,
yte):.1%}")
modes = np.r_[rng.normal([2, 0], 0.4, (300, 2)), rng.normal([-1, 2], 0.4, (300, 2)),
rng.normal([-1, -2], 0.4, (300, 2))]
for k in [2, 3, 4, 5]:
    km = KMeans(k, n_init=10, random_state=0).fit(modes)
    print(f"  K-means k = {k}：輪廓係數 {silhouette_score(modes, km.labels_):.2f}")
```

執行結果：缺陷率約 43%。隨機森林在隨機切分的交叉驗證中準確率約 74.1%，依批料分組切分時只有約 65.9%。差距約 8 個百分點，就是「認得批料」帶來的虛假成績；上線後面對新批料，比較接近的是 65.9%。

決策樹的結果則清楚地顯示過度配適：深度 1 時訓練與測試準確率分別約 66.4% 與 63.0%；深度 3 時測試準確率最高，約 68.3%；深度繼續增加，訓練準確率一路升到 100%，測試準確率卻下降到約 65% 左右。

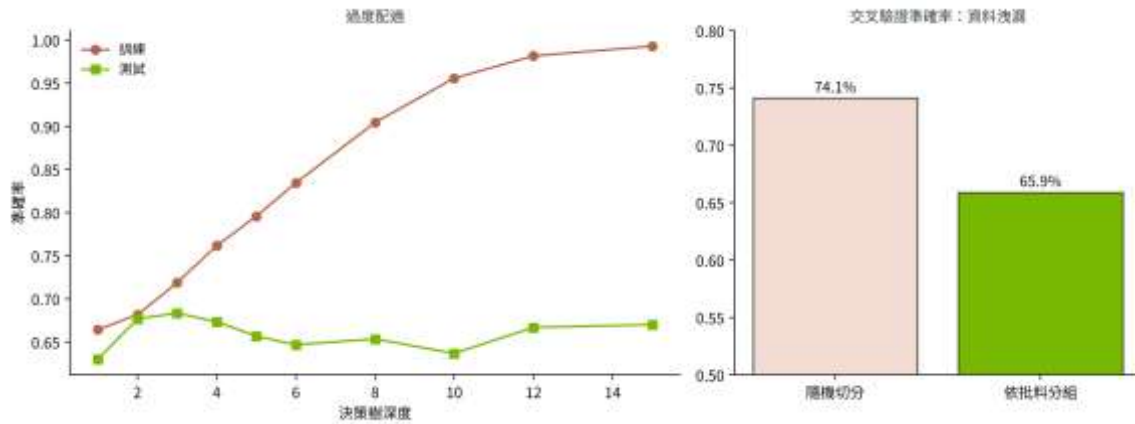


圖 22-3 過度配適與資料洩漏

非監督式學習的部分，K-means 以 $k = 3$ 分群時輪廓係數最高（約 0.80）， $k = 2、4、5$ 時分別約為 0.57、0.64 與 0.48，正確找出了資料中的三種操作模式。輪廓係數衡量每個樣本與自己群內的相似程度相對於與其他群的差異，是選擇群數的常用依據之一。

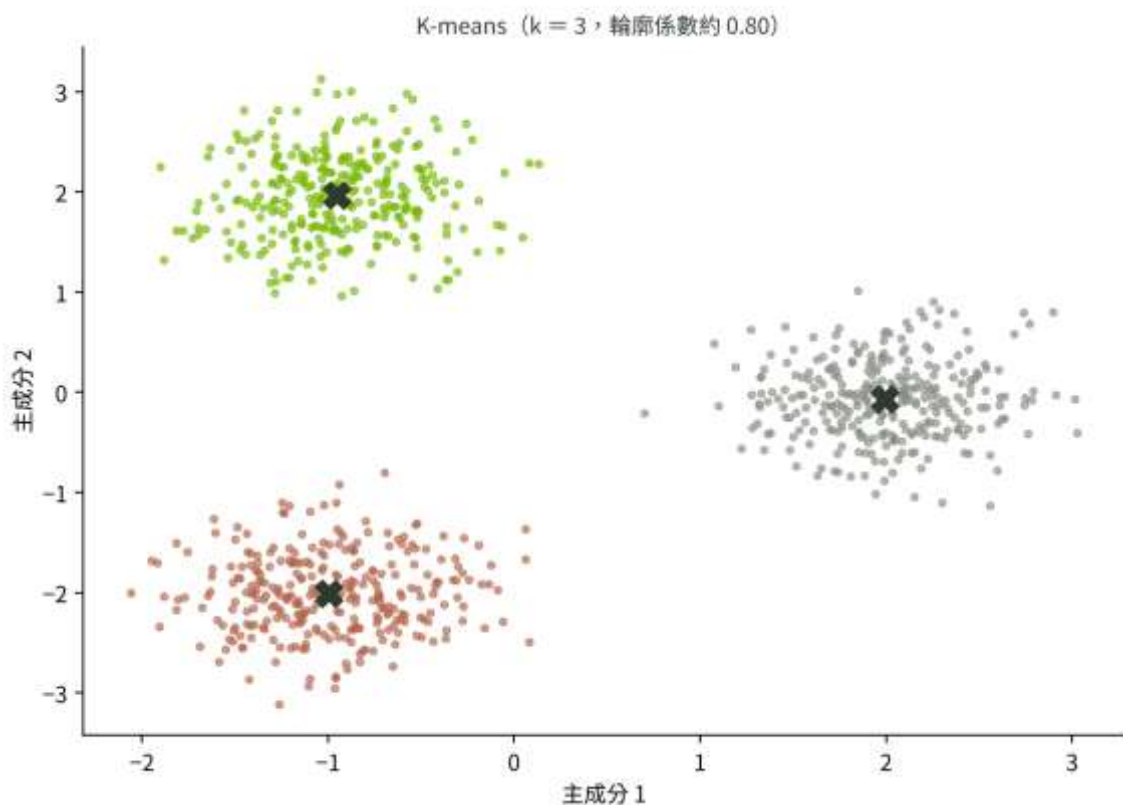


圖 22-4 以 K-means 分出三種操作模式

這個例子給製造資料分析一個重要的提醒：切分資料的方式，要模擬模型上線後會遇到的情況。若模型會面對新的批料、新的模具或未來的時間，就應該依批料、模具或時間切分。以時間序列資料為例，應以過去的資料訓練、未來的資料測試，而不是隨機打亂。其他常見的資料洩漏還包括：用到了在預測時還拿不到的資訊（例如事後的檢驗結果被當成特徵），以及在切分之前就以全部資料做標準化或特徵選擇。

22.3.4 模型的評估指標

除了準確率，製造中常用的評估指標還有：

- 混淆矩陣：真陽性、偽陽性、真陰性、偽陰性的數量。
- 召回率（敏感度）：所有真正的不良品中，被抓出來的比例；漏判的代價高時特別重要。
- 精確率：被判為不良品的樣本中，真正是不良品的比例；誤判會增加複檢成本。
- ROC 曲線與 AUC：模型區分兩類的能力，與判定門檻無關。
- 迴歸指標：平均絕對誤差、均方根誤差，以及相對於規格寬度的誤差比例。

不良率很低時，準確率會嚴重誤導：不良率 1% 時，把所有產品都判為良品，準確率就有 99%，卻一個不良品也沒抓到。22.5 節將以成本的觀點選擇判定門檻。

22.4 深度學習：CNN、RNN、Transformer 概要

22.4.1 從特徵工程到表徵學習

傳統的機器學習，常需要工程師先從原始資料中算出「特徵」：從振動訊號算出均方根值、峰度與特定頻帶的能量，從影像算出邊緣與紋理。深度學習則以多層的神經網路，直接從原始資料中學習有用的表徵。這在影像、聲音、振動與文字等高維、非結構化的資料上特別有優勢，但也需要更多的資料與運算，而且較難解釋。

表 22-5 三種主要的深度學習架構

架構	代表性工作	核心理念	適合的資料	製造上的例子
卷積神經網路	LeCun 等人 1998 年的 LeNet ; 2012 年 AlexNet 在影像辨識競賽的突破	以小的卷積核在資料上滑動，共享權重擷取局部特徵	影像、振動、光譜	瑕疵檢測、軸承診斷
循環神經網路與 LSTM	Hochreiter 與 Schmidhuber 1997 年提出 LSTM	以隱藏狀態記住過去，閘門控制記住與遺忘	時間序列	剩餘壽命預測、製程軌跡
Transformer	Vaswani 等人 2017 年〈Attention Is All You Need〉	以注意力機制讓每個位置直接參考其他所有位置	序列、文字、影像片段、多模態	大型語言模型、時間序列基礎模型

22.4.2 卷積神經網路

卷積神經網路的關鍵是卷積：一個小小的權重視窗（卷積核）在訊號或影像上滑動，每到一個位置就計算一次加權和。同一個卷積核在所有位置共享相同的權重，所以不論故障特徵出現在訊號的哪裡，都能被同樣地偵測到。多個卷積核可以學到不同的局部模式，再經過非線性的啟動函數與池化，最後由輸出層做出判斷。

下面的程式以 NumPy 從零寫一個最小的一維 CNN，辨識軸承振動訊號中是否有週期性的衝擊（故障的典型特徵）。正常訊號是一個正弦波加上雜訊；故障訊號每隔約 40 點多出一個快速衰減的衝擊。網路只有一層：8 個長度 16 的卷積核、ReLU、全域平均池化與一個輸出神經元，總共 145 個參數，以隨機梯度下降訓練。作為對照，也以「均方根值超過門檻就判為故障」的簡單方法比較：

```
import numpy as np
# 深度學習的最小實作：以 NumPy 從零寫一個一維 CNN，辨識軸承振動訊號中的週期衝擊（故障）
rng = np.random.default_rng(0)
L, K, C = 256, 16, 8 # 訊號長度、卷積核長度、卷積核數
def signal(fault):
    t = np.arange(L)
    x = 0.8 * np.sin(2 * np.pi * t / 32 + rng.uniform(0, 6.3)) + rng.normal(0, 0.5, L)
    if fault: # 故障：每隔約 40 點出現一次快速衰減的衝擊
        start = rng.integers(0, 40)
        for s in range(start, L - 12, 40):
            x[s:s + 12] += 2.0 * np.exp(-np.arange(12) / 3) * np.sin(np.arange(12) * 2.0)
    return x
def data(n):
```

```

y = rng.integers(0, 2, n); return np.array([signal(f) for f in y]), y
Xtr, ytr = data(600); Xte, yte = data(400)
W = rng.normal(0, 0.3, (C, K)); b = np.zeros(C); v = rng.normal(0, 0.3, C); c = 0.0
def forward(x):
    win = np.lib.stride_tricks.sliding_window_view(x, K)           # (L-K+1, K)
    z = win @ W.T + b                                               # 卷積
    a = np.maximum(z, 0)                                           # ReLU
    h = a.mean(axis=0)                                             # 全域平均池化
    p = 1 / (1 + np.exp(-(h @ v + c)))                             # 輸出層
    return p, (win, z, h)
def train(epochs=30, lr=0.2):
    global W, b, v, c
    for ep in range(epochs):
        for i in rng.permutation(len(Xtr)):
            p, (win, z, h) = forward(Xtr[i]); g = p - ytr[i]      # 交叉熵對輸出的梯度
            gv, gc = g * h, g
            gz = g * v * (z > 0) / len(z)                         # 反向傳播經過平均池化與
ReLU
            gw = gz.T @ win; gb = gz.sum(axis=0)
            v -= lr * gv; c -= lr * gc; W -= lr * gw; b -= lr * gb
acc = lambda X, y: np.mean([(forward(x)[0] > 0.5) == t for x, t in zip(X, y)])
rms = lambda X: np.sqrt((X**2).mean(axis=1))
thr = np.median(rms(Xtr))                                         # 對照：只用均方根值判斷
print(f"對照：均方根值門檻法 測試準確率 {np.mean((rms(Xte) > thr) == yte):.1%}")
print(f"訓練前 CNN 測試準確率 {acc(Xte, yte):.1%}")
train(); print(f"訓練後 CNN 測試準確率 {acc(Xte, yte):.1%}; 參數總數 {W.size + b.size +
v.size + 1}")

```

執行結果：只看均方根值的門檻法，測試準確率約 76.2%，因為衝擊雖然增加了能量，但與正弦波和雜訊的能量變化重疊。訓練前的 CNN 準確率約 49.2%，相當於亂猜；訓練後達到約 96.8%。

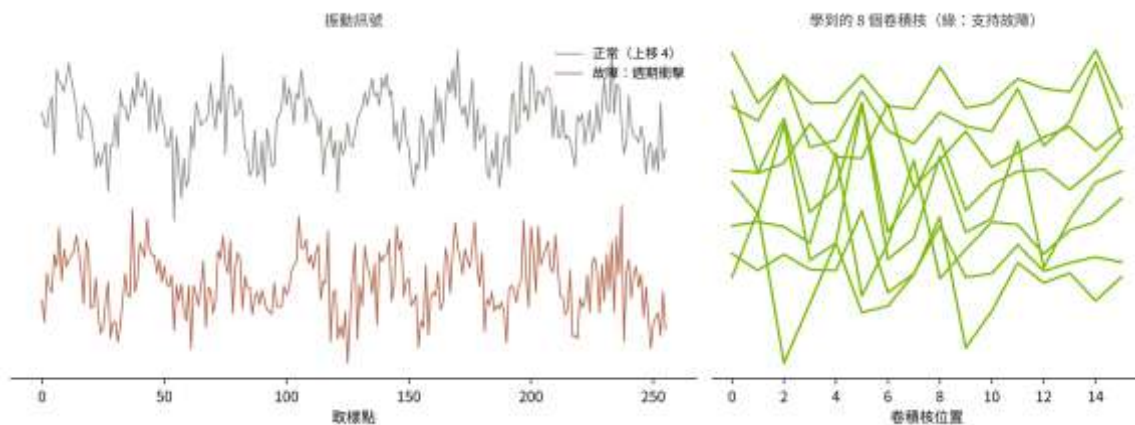


圖 22-5 一維 CNN：正常與故障訊號，以及學到的卷積核

這個小網路之所以能勝過均方根值，是因為卷積核學會了衝擊的「形狀」：快速上升、振盪、衰減。這正是傳統振動診斷中工程師以包絡分析等方法尋找的特徵，只是這裡由資料自動學到。實務上的 CNN 會有多層卷積、數十萬到數百萬個參數，並以 PyTorch 等框架訓練；但其原理與這 145 個參數的小網路相同。這個例子也提醒我們：在有明確物理特徵的問題上，好的特徵工程（例如包絡頻譜）常常與深度學習一樣有效，而且更容易解釋。

22.4.3 循環神經網路與 LSTM

時間序列資料，例如設備的振動趨勢、製程的溫度軌跡，需要模型能「記住過去」。循環神經網路在每個時間步更新一個隱藏狀態，把過去的資訊帶到現在。但簡單的循環網路很難記住很久以前的資訊，因為梯度在反向傳播時會逐漸消失。Hochreiter 與 Schmidhuber 在 1997 年提出的長短期記憶網路 (LSTM)，以閘門控制記憶的寫入與輸出，大幅改善了這個問題；原始版本只有輸入與輸出兩個閘門，Gers、Schmidhuber 與 Cummins 於 1999 年加入遺忘閘門 (2000 年發表於期刊)，成為今日最常用的形式，長期以來是時間序列建模的主力，常用於剩餘壽命預測 (第 26 章)。

22.4.4 Transformer 與注意力

2017 年，Vaswani 等人發表〈Attention Is All You Need〉，提出完全以注意力機制為基礎的 Transformer。注意力的想法很直觀：序列中的每一個位置，都以一個「查詢」向量去和所有位置的「鍵」向量比較相似度，再依相似度加權彙整所有位置的「值」向量。這讓模型可以直接參考任何距離的資訊，而且可以平行運算，成為今日大型語言模型與許多影像、時間序列基礎模型的骨幹 (第 14、28 章)。

下面的程式以 NumPy 實作縮放點積注意力。8 個時間點 (例如 8 個製程步驟) 各有 4 維特徵，其中第 6 步與第 2 步非常相似；為了說明，查詢、鍵與值的投影矩陣都設為單位矩陣：

```
import numpy as np
# Transformer 的核心：縮放點積注意力。每個時間點「詢問」其他時間點，依相似度加權彙整資訊
rng = np.random.default_rng(2)
def attention(Q, Kx, V):
    s = Q @ Kx.T / np.sqrt(Q.shape[1])          # 相似度分數，除以  $\sqrt{d}$  避免數值過大
    w = np.exp(s - s.max(axis=1, keepdims=True)); w /= w.sum(axis=1, keepdims=True)  # softmax
    return w @ V, w
T, d = 8, 4                                     # 8 個時間點 (例如 8 個製程步驟)、4 維特徵
x = rng.normal(0, 1, (T, d)); x[5] = x[1] + 0.05 * rng.normal(0, 1, d)  # 第 6 步與第 2 步非常相似
Wq, Wk, Wv = np.eye(d), np.eye(d), np.eye(d)   # 為了說明，投影矩陣設為單位矩陣
out, w = attention(x @ Wq, x @ Wk, x @ Wv)
np.set_printoptions(precision=2, suppress=True)
print("第 6 步對各步的注意力權重:", w[5])
print("最受注意的步驟 (不含自己): 第", int(np.argsort(w[5])[-2]) + 1, "步")
```

執行結果：第 6 步對 8 個步驟的注意力權重約為 0.01、0.44、0.02、0.01、0.02、0.48、0.01、

0.00，除了自己 (0.48) 之外，最受注意的是與它相似的第 2 步 (0.44)，其餘幾乎被忽略；每一列權重的總和都是 1。在真實的 Transformer 中，投影矩陣是學出來的，模型會學到「哪些步驟之間的關係對預測有用」，例如某一道熱處理的溫度與後面某一道量測結果的關係，而不只是表面上的相似。

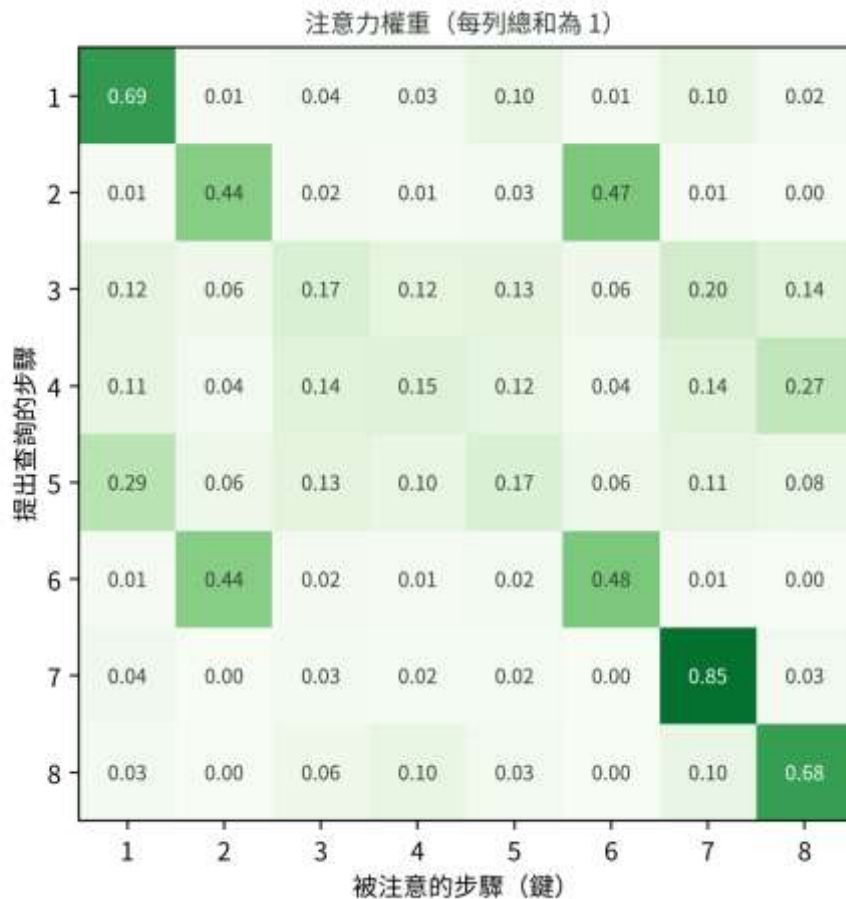


圖 22-6 縮放點積注意力的權重矩陣

22.4.5 深度學習在製造上的限制

深度學習很強大，但在製造中要特別注意：

- 資料需求：深度模型需要大量標註資料，而製造中的不良樣本稀少。遷移學習、預訓練模型與第 21 章的合成資料是常用的對策。
- 可解釋性：模型為什麼判定這件是不良品？品質與法規要求常需要解釋，第 24 章將介紹可解釋的方法。
- 運算與部署：模型要在邊緣裝置上即時執行時，需要壓縮與最佳化 (第 11、23 章)。

- 穩健性：光線、角度、設備的小改變，就可能讓模型失準，需要持續監控與更新。

22.5 從問題定義到可部署模型：工業 ML 管線

22.5.1 為什麼大多數的模型沒有上線？

很多工廠的機器學習專案，在概念驗證時表現亮眼，卻始終沒有真正上線，或上線不久就被關掉。原因多半不在演算法，而在演算法以外的部分：問題定義不清、資料品質不佳、評估方式不符合實際、上線後沒有人維護、現場不信任結果。第 19 章的蒙地卡羅分析也指出，數位孿生專案最大的風險常是「現場沒有持續使用」，機器學習專案也是如此。

一個廣為使用的流程參考是跨產業資料探勘標準流程 (CRISP-DM)，始於 1996 年，1.0 版於 1999 年在布魯塞爾的研討會上發表，常見的逐步指南文件版本則標示為 2000 年，把資料探勘專案分為六個階段：業務理解、資料理解、資料準備、建模、評估與部署，並強調各階段之間會反覆來回。工業機器學習管線可以在這個框架上，再加入上線後的監控與維護：



圖 22-7 工業機器學習管線

表 22-6 工業機器學習管線的各階段

階段	關鍵問題	常見陷阱	本書相關章節
問題定義	要支援什麼決策？誰使用？錯了的代價是什麼？	只說「用 AI 提升良率」，沒有明確的決策與指標	第 6 章
資料蒐集與理解	資料從哪來？標籤怎麼定義？有哪些混淆因子？	標籤不一致、時間不同步、忽略批次結構	第 17、20、23 章

階段	關鍵問題	常見陷阱	本書相關章節
資料準備與特徵	需要哪些特徵？預測時拿得到嗎？	用到事後資訊（資料洩漏）	本章 22.3 節
建模與驗證	切分方式是否模擬上線情境？指標是否反映成本？	隨機切分、只看準確率	本章 22.3 節
決策設計	判定門檻怎麼定？不確定時怎麼辦？	預設門檻 0.5、沒有人工覆核機制	本章 22.5 節、第 36 章
部署	在哪裡執行？延遲與可靠度要求？如何與 MES 整合？	只在筆電上跑得動	第 11、23 章
監控與維護	資料是否漂移？效果是否下降？誰負責重訓？	上線後無人看管	第 20 章

22.5.2 依成本決定判定門檻

分類模型輸出的通常是一個機率，例如「這一件是不良品的機率為 0.12」。要不要判為不良品，取決於門檻。很多人直接使用 0.5，但在製造中，漏判與誤判的代價往往相差很多：一件不良品流到客戶手上，可能造成退貨、索賠與信譽損失；一件良品被誤判，只是多花一次複檢的成本。

若模型輸出的機率是校準過的（也就是說，預測 10% 的那些樣本中，真的約有 10% 是不良品），以期望成本最小化的觀點，當「判為不良品的期望成本」小於「判為良品的期望成本」時就應該判為不良品，可以推得最佳門檻約為：

$$\text{最佳門檻} = \text{誤判成本} \div (\text{誤判成本} + \text{漏判成本})$$

22.5.3 上線後的資料漂移

模型上線後，世界會改變：原料換了供應商、設備保養、季節變化、產品組合改變。當輸入資料的分布與訓練時不同，模型的表現就可能下降，這稱為資料漂移。常用的監控方法之一是族群穩定指標（Population Stability Index, PSI）：把訓練資料的某個特徵分成若干區間，比較上線資料在各區間的比例與訓練時的差異。實務上常以 PSI 小於 0.1 表示穩定、0.1 ~ 0.25 表示需要注意、大於 0.25 表示有明顯變化，但這只是經驗法則，應依應用調整。

下面的程式把兩件事放在一起：一個缺陷預測模型，以驗證資料找出使每件平均成本最低的門檻，並與 0.5 比較；接著模擬上線後四個月，第 1 個特徵因為換原料而逐月偏移，監控 PSI 與實際成本：

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score, precision_score, recall_score
# 工業 ML 管線的兩個關鍵決策：(1) 依成本決定判定門檻；(2) 上線後監控資料漂移
rng = np.random.default_rng(5)
def make(n, shift=0.0):
    X = rng.normal(0, 1, (n, 4)); X[:, 0] += shift          # shift：上線後原料換供應商，第
    1 個特徵整體偏移
    logit = 2.0 * X[:, 0] + 1.0 * X[:, 1] - 4.5             # 缺陷率約 6%
    return X, (rng.random(n) < 1 / (1 + np.exp(-logit))).astype(int)
Xtr, ytr = make(20000); Xva, yva = make(10000)
m = LogisticRegression().fit(Xtr, ytr); pva = m.predict_proba(Xva)[:, 1]
C_FN, C_FP = 5000, 200                                     # 漏判一件流到客戶 5,000 元；
誤判送複檢 200 元
def cost(p, y, t): pred = p >= t; return (C_FN * np.sum(~pred & (y == 1)) + C_FP *
np.sum(pred & (y == 0))) / len(y)
def psi(a, b, bins=10):                                     # 族群穩定指標 (PSI)
    edges = np.quantile(a, np.linspace(0, 1, bins + 1)); edges[0], edges[-1] = -np.inf,
np.inf
    pa = np.histogram(a, edges)[0] / len(a) + 1e-6; pb = np.histogram(b, edges)[0] /
len(b) + 1e-6
    return np.sum((pb - pa) * np.log(pb / pa))
print(f"驗證集缺陷率 {yva.mean():.1%}; AUC {roc_auc_score(yva, pva):.3f}; 全部判良品的準確率
{1 - yva.mean():.1%}")
ts = np.linspace(0.01, 0.9, 90); cs = [cost(pva, yva, t) for t in ts]; tb =
ts[int(np.argmin(cs))]
for t in [0.5, tb]:
    pr = pva >= t
    print(f"門檻 {t:.2f}: 召回率 {recall_score(yva, pr):.1%}, 精確率 {precision_score(yva,
pr):.1%}, 每件平均成本 {cost(pva, yva, t):.0f} 元")
print(f"理論最佳門檻 C_FP/(C_FP+C_FN) = {C_FP/(C_FP+C_FN):.3f}")
for k, s in enumerate([0.0, 0.2, 0.5, 1.0]):
    Xn, yn = make(5000, s); pn = m.predict_proba(Xn)[:, 1]
    print(f"上線第 {k+1} 月 (偏移 {s}): 特徵 PSI {psi(Xtr[:,0], Xn[:,0]):.3f}, 實際缺陷率
{yn.mean():.1%}, "
        f"以原門檻判定的成本 {cost(pn, yn, tb):.0f} 元/件")
```

執行結果：驗證集的缺陷率約 6.1%，模型的 AUC 約 0.910。若把所有產品都判為良品，準確率就有 93.9%，這再次說明準確率在不平衡資料上的誤導性。

表 22-7 判定門檻與成本

門檻	召回率	精確率	每件平均成本
0.50	24.5%	69.0%	231 元
0.03 (成本最佳)	91.9%	17.6%	77 元

以 0.5 為門檻，精確率雖然有 69.0%，召回率卻只有 24.5%，四件不良品漏掉三件，每件平均成本約 231 元。以成本最低的門檻約 0.03 判定，召回率提升到約 91.9%，精確率降到約 17.6%（每抓到一件不良品，要複檢約五件），但每件平均成本降到約 77 元，只有原來的三分之一。理論最佳門檻 $200 \div (200 + 5,000) \approx 0.038$ ，與搜尋結果相近。

上線後的監控結果：第 1 個月沒有偏移，PSI 約 0.001，成本約 84 元 / 件，與驗證時相近；第 2 個月偏移 0.2，PSI 約 0.053，仍在穩定範圍；第 3 個月偏移 0.5，PSI 約 0.235，接近需要注意的上緣，實際缺陷率升到約 10.8%，成本升到約 95 元；第 4 個月偏移 1.0，PSI 約 0.927，明顯漂移，缺陷率約 19.0%，成本約 116 元 / 件。

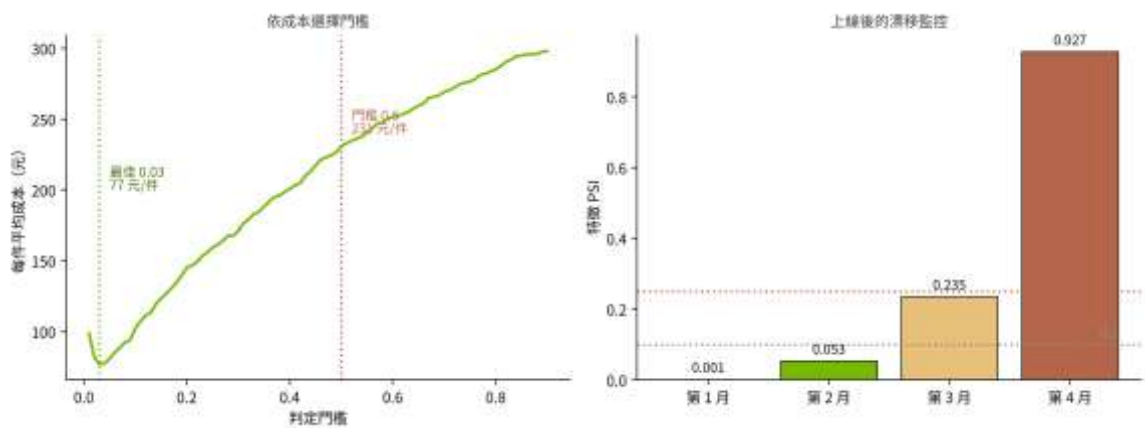


圖 22-8 依成本選擇門檻，與上線後的資料漂移監控

這裡值得細想的是：在這個模擬中，特徵與缺陷之間的關係並沒有改變，模型本身仍然「正確」；變的是輸入的分布，使得缺陷變多、成本上升。PSI 的警示提醒工程師：製程的狀態改變了，應該回頭了解原因（是原料嗎？），並檢查模型的校準、門檻與重訓的需要。若變的是特徵與缺陷之間的關係（稱為概念漂移），例如換了模具之後同樣的參數產生不同的結果，模型本身就失效了，只能靠持續追蹤預測準確度與定期的實際量測來發現，這與第 21 章虛擬量測的抽測修正是同一個道理。

22.5.4 部署與維護

模型要真正產生價值，必須融入現場的工作流程：

- 執行位置：需要毫秒級回應的（例如產線上的視覺檢測），部署在邊緣；需要大量運算、可以接受延遲的（例如每日的良率分析），部署在伺服器或雲端（第 11、23 章）。

- 與系統整合：預測結果要寫回 MES，觸發分流、複檢或警示，而不是只顯示在另一個畫面上（第 17 章）。
- 人工覆核：不確定的預測交由人員判斷，人員的判斷又成為新的標籤，讓模型持續學習。
- 版本與血緣：模型的每一個版本、使用的資料與門檻都要記錄，以便追溯與回復（第 20 章）。
- 負責人：每個上線的模型都要有明確的負責人，定期檢視監控報表。

這些工作常被合稱為機器學習維運（MLOps），第 23 章將從資料工程的角度深入介紹。

案例

案例 22-1 教學情境：資料說溫度越高越好

背景：叫獸開講中的射出成形廠（綜合情境），合併三個月的資料後發現模具溫度與良率高度正相關，計畫把所有機台的溫度調高。

做法：依機台分組重新分析，發現每台機台內部溫度與良率呈負相關；機台世代同時影響溫度設定與良率，是混淆因子。之後以實驗設計在單一機台上確認溫度的效果，並把「機台」納入所有分析。

成果：避免了一次會讓所有機台良率下降的錯誤調整；團隊也建立了分析前先依結構分組檢視的習慣。

啟示：相關不等於因果；混淆因子需要領域知識才能辨識，確認因果最可靠的方法是實驗。

討論：（1）你的工廠資料中，可能有哪些混淆因子？（2）無法做實驗時，還有哪些方法可以降低誤判？

案例 22-2 產業：漂亮的準確率，上線就失準

背景：一家電子組裝廠的資料團隊以三個月的資料訓練焊點缺陷預測模型，隨機切分的測試準確率超過九成，上線後卻表現平平。

做法：檢討後發現，同一批电路板的資料同時出現在訓練與測試集中，模型學到的是批次的特

徵；改以「依批次」與「依時間」切分重新評估，並把只有在檢驗後才知道的特徵移除。

成果：重新評估的準確率較低，但與上線後的表現一致；團隊據此調整特徵與模型，並設定合理的期望。

啟示：評估方式必須模擬上線情境；資料洩漏讓模型看起來比實際好，是製造 AI 專案失敗的常見原因。

討論：（1）還有哪些形式的資料洩漏？（2）如何向管理者說明「準確率降低了，但評估更可信」？

案例 22-3 產業：以 CNN 診斷軸承故障

背景：一家風機與泵浦的維修服務商，過去依振動的均方根值與經驗判斷軸承狀況，常常太晚發現故障。

做法：蒐集正常與已知故障軸承的振動資料，以包絡頻譜等傳統特徵與一維 CNN 兩種方法建立診斷模型，並在不同轉速與負載的資料上以分組方式評估。

成果：兩種方法都明顯優於只看均方根值；CNN 在轉速變動大的設備上表現較好，傳統特徵則較容易向客戶解釋，團隊最後兩者並用。

啟示：深度學習能自動學到故障特徵，但好的特徵工程同樣有效且易解釋；選擇方法要考慮資料量、可解釋性與部署條件。

討論：（1）訓練資料中的故障樣本從哪裡來？（2）如何確保模型在新設備上仍然有效？

案例 22-4 產業：依成本調整判定門檻

背景：一家汽車零件廠的視覺檢測模型以 0.5 為門檻，誤判少、精確率高，但仍有不良品流到客戶端，被客戶要求改善。

做法：與品保及業務共同估計漏判與誤判的成本，發現漏判的代價約為誤判的二十倍以上；據此把門檻大幅調低，並增加一個人工複檢站處理被判為可疑的零件。

成果：流到客戶端的不良品明顯減少，雖然複檢量增加，整體品質成本下降。

啟示：門檻是商業決策，不是技術預設值；與現場和管理者一起定義成本，是模型上線的重要一步。

討論：（1）漏判成本難以估計時怎麼辦？（2）複檢站的產能如何影響門檻的選擇？

案例 22-5 台灣情境：資料科學人才的養成

背景：台灣製造業近年積極推動數位轉型，許多企業發現最缺的不是軟體，而是同時懂製程與資料的人才。

做法：常見的做法包括讓製程工程師學習統計與機器學習、讓資料科學家駐點產線了解製程，以及由大學與企業合作開設以真實資料為題的課程與專題。

成果：具備領域知識的資料分析人才，較能提出正確的問題、辨識混淆因子與資料洩漏，讓模型真正上線。

啟示：製造資料科學的關鍵是「領域知識 + 統計思維 + 工程實作」的結合；本章的基礎是三者的共同語言。

討論：（1）你認為製程工程師與資料科學家各自最需要補強什麼？（2）學校的專題如何更貼近真實的製造資料？

案例 22-6 教學情境：沒有顯著差異，所以沒有差異？

背景：一個學生專題團隊比較兩種刀具塗層的壽命，各測試 5 支刀具，t 檢定的 p 值為 0.3，於是結論為「兩種塗層沒有差異」。

做法：老師請團隊計算效果量與檢定力，發現兩種塗層的平均壽命相差約兩成，但每組只有 5 支、變異又大，檢定力不到兩成；團隊改以信賴區間呈現結果，並估計需要的樣本數。

成果：結論改為「目前的資料不足以判斷，平均差異可能在 -10% 到 +50% 之間」，並提出擴大測試的計畫。

啟示：不顯著不等於沒有差異；信賴區間、效果量與檢定力，比單一的 p 值更能呈現證據的強弱。

討論：(1) 如何在有限的預算下設計更有效率的實驗？(2) 什麼情況下「沒有差異」的結論是可靠的？

實作活動

活動 22-1 敘述統計與檢定

蒐集一組你能量測的資料（例如 3D 列印件的尺寸，兩種參數各 20 件），計算敘述統計與 Cpk ，以 t 檢定與 Mann-Whitney 檢定比較，並報告信賴區間與效果量；再以 22.1.4 節的程式估計檢出 0.05 mm 差異所需的樣本數。

活動 22-2 探索性資料分析

以公開的製造資料集或第 17 ~ 21 章的模擬資料，進行一次完整的探索性分析：檢查缺值與異常值、畫出各變數的分布與時間趨勢、依機台或批次分組比較，並寫下三個值得驗證的假設。

活動 22-3 資料切分實驗

以 22.3.3 節的程式為基礎，改變批料指紋的強度與每批的樣本數，觀察隨機切分與分組切分的差距如何變化，並說明原因。

活動 22-4 一維 CNN

以 22.4.2 節的程式為基礎，改變衝擊的強度與雜訊大小，畫出 CNN 與均方根值門檻法的準確率曲線；再把學到的 8 個卷積核畫出來，看看它們是否像衝擊的形狀。

活動 22-5 門檻與成本

以 22.5.3 節的程式為基礎，改變漏判與誤判的成本比例，畫出最佳門檻與成本的變化；並加入「人工複檢站每日產能有限」的限制，重新選擇門檻。

活動 22-6 漂移監控儀表板

以 22.5.3 節的程式為基礎，為四個特徵都計算每月的 PSI，設計一個簡單的監控報表，並寫出 PSI 超過門檻時的處理流程。相關範例程式將於本書的實驗倉庫中維護。

延伸閱讀

- John W. Tukey，Exploratory Data Analysis (Addison-Wesley，1977)：探索性資料分析的經典。
- Douglas C. Montgomery，Introduction to Statistical Quality Control：製造統計與品質管制的標準教材（第 25 章亦將引用）。
- Trevor Hastie、Robert Tibshirani 與 Jerome Friedman，The Elements of Statistical Learning：統計學習的經典教材。
- Ian Goodfellow、Yoshua Bengio 與 Aaron Courville，Deep Learning (MIT Press，2016)：深度學習的系統性教材。
- Ashish Vaswani 等人，〈Attention Is All You Need〉，NeurIPS 2017：Transformer 的原始論文。
- Pete Chapman 等人，CRISP-DM 1.0 Step-by-step Data Mining Guide (常見版本為 2000 年)：資料探勘專案的流程參考。

表 22-8 本章思考地圖

本章概念	關鍵提問	後續延伸章節
敘述統計與檢定	差異是真的嗎？有多大？重要嗎？樣本夠嗎？	第 25 章
探索性資料分析	資料有什麼結構？有哪些混淆因子？	第 23 章
監督與非監督學習	有沒有標籤？要預測還是要探索？	第 24、26 章
泛化與資料洩漏	切分方式模擬了上線情境嗎？	第 23、24 章
深度學習	資料量夠嗎？需要解釋嗎？能部署嗎？	第 24、28、30 章
ML 管線與門檻	錯了的代價是什麼？門檻怎麼定？	第 25、36 章

本章概念	關鍵提問	後續延伸章節
漂移與維護	上線後誰負責監控？何時重訓？	第 20、23 章

章末小結

- 製造資料少而貴、標籤稀少不平衡、有批次結構、會隨時間改變，而且背後有物理。敘述統計描述中心與散布，Cpk 衡量製程與規格的餘裕；選擇合適的機率分布很重要。範例中機台 B 的平均偏大約 $3\text{ }\mu\text{m}$ （95% 信賴區間約 $1.03 \sim 5.18\text{ }\mu\text{m}$ ），Cpk 只有 0.89。p 值之外還要看信賴區間、效果量與檢定力：差異 $1\text{ }\mu\text{m}$ 時，每組 30 件的檢定力只有約 13%。
- 探索性資料分析強調先看資料再下結論。辛普森悖論的範例中，合併資料的斜率為正（約 $+0.225\text{ \%}/^{\circ}\text{C}$ ），控制機台後卻是負的（約 $-0.150\text{ \%}/^{\circ}\text{C}$ ）；混淆因子需要領域知識辨識，確認因果最可靠的方法是實驗。
- 機器學習分為監督式、非監督式、半監督與強化學習。過度配適讓訓練準確率趨近 100% 而測試準確率下降；隨機切分讓批次資訊洩漏，範例中把準確率從約 65.9% 高估為約 74.1%。切分方式必須模擬上線情境；不良率低時準確率會誤導。
- CNN 以共享權重的卷積核擷取局部特徵，LSTM 以閘門記住長期資訊，Transformer 以注意力直接參考所有位置。範例中 145 個參數的一維 CNN 把軸承故障的辨識準確率從均方根值法的約 76.2% 提升到約 96.8%。深度學習在製造上要注意資料需求、可解釋性、部署與穩健性。
- 工業 ML 管線從問題定義到監控維護；CRISP-DM 1.0 於 1999 年發表。依成本選擇門檻，範例中把每件成本從約 231 元降到約 77 元；上線後以 PSI 等指標監控資料漂移，並區分資料漂移與概念漂移。模型要融入 MES 與工作流程，並有人工覆核、版本血緣與明確的負責人。

習題

一、選擇題

1. 較不受極端值影響的散布指標是？（A）標準差 （B）全距 （C）四分位距 （D）變異數

2. 故障間隔與壽命資料常用的分布是？(A) 常態分布 (B) 威布爾分布 (C) 均勻分布 (D) 二項分布
3. 真的有差異時，檢定能夠檢出的機率稱為？(A) p 值 (B) 檢定力 (C) 顯著水準 (D) 效果量
4. 分組與合併後的趨勢方向相反的現象稱為？(A) 長鞭效應 (B) 辛普森悖論 (C) 過度配適 (D) 資料漂移
5. 在沒有標籤的資料中找出群組的方法屬於？(A) 監督式學習 (B) 非監督式學習 (C) 強化學習 (D) 遷移學習
6. 同一批料的樣本同時出現在訓練與測試集中，造成評估過度樂觀，稱為？(A) 資料洩漏 (B) 欠配適 (C) 梯度消失 (D) 資料協調
7. 以共享權重的小視窗在資料上滑動擷取局部特徵的網路是？(A) CNN (B) LSTM (C) 決策樹 (D) K-means
8. 提出 Transformer 的論文是？(A) 〈Attention Is All You Need〉 (B) 〈Long Short-Term Memory〉 (C) 〈Gradient-Based Learning Applied to Document Recognition〉 (D) 〈Deep Residual Learning〉
9. CRISP-DM 的第一個階段是？(A) 建模 (B) 業務理解 (C) 部署 (D) 資料準備
10. 用來監控特徵分布變化的族群穩定指標縮寫為？(A) PSI (B) PID (C) PLC (D) PCA

二、簡答題

11. 說明製造資料的五個特性，以及它們對分析方法的影響。
12. 為什麼 p 值之外，還需要報告信賴區間與效果量？
13. 什麼是混淆因子？以辛普森悖論的例子說明。
14. 說明過度配適的現象，以及如何偵測與避免。
15. 為什麼製造資料不宜隨機切分訓練與測試集？應該怎麼切分？
16. 比較 CNN、LSTM 與 Transformer 的核心想法與適用資料。

17. 為什麼不良率低時不應只看準確率？應看哪些指標？
18. 說明資料漂移與概念漂移的差異，以及各自的偵測方法。

三、計算題

19. 某尺寸的平均 20.003 mm、標準差 0.004 mm，規格 20 ± 0.015 mm，求 Cpk。
20. 同時檢定 100 個與品質無關的參數，顯著水準 0.05，預期會有幾個「顯著」？以 Bonferroni 校正，每個檢定的顯著水準應為多少？
21. 某模型在 10,000 件中：真陽性 46、偽陰性 4、偽陽性 184、真陰性 9,766。求召回率、精確率與準確率。
22. 不良率 1% 時，把全部產品判為良品，準確率與召回率各為多少？
23. 漏判成本 5,000 元、誤判成本 200 元，校準良好的模型的最佳判定門檻約為多少？
24. 某特徵的 PSI 為 0.18，依常用的經驗法則應如何解讀？
25. 一維 CNN 有 8 個長度 16 的卷積核（各有偏差），經全域池化後接一個輸出神經元（含偏差），參數總數為多少？
26. 縮放點積注意力中，若查詢與鍵的維度為 64，相似度分數要除以多少？

四、申論與設計題

27. 為一家工廠「以 AI 降低焊接不良率」的構想，依工業 ML 管線寫出問題定義、資料、評估方式、門檻設計與上線監控計畫。
28. 評論以下說法：「資料量夠大，就不需要領域知識。」
29. 比較在製造品質預測中使用梯度提升樹與深度學習的優缺點，並說明你會如何選擇。

五、是非題

30. 檢定結果不顯著，就表示兩組沒有差異。
31. 在資料量很大時，很小的差異也可能統計顯著，但不一定有工程意義。
32. 訓練準確率越高，模型在新資料上的表現一定越好。

33. 時間序列資料應以過去訓練、未來測試，而不是隨機打亂。

34. 分類模型的判定門檻一律應設為 0.5。

35. PSI 穩定，就保證模型的預測仍然準確。

六、進階挑戰題

36. 以 PyTorch 實作 22.4.2 節的一維 CNN，增加為兩層卷積，並以不同轉速的資料測試其泛化能力；以梯度權重等方法視覺化模型關注的訊號位置。

37. 以第 21 章的虛擬量測資料，比較嶺迴歸、隨機森林與梯度提升樹，並以時間切分評估；分析各模型在機台保養後的表現。

38. 實作一個簡單的概念漂移偵測器：在上線資料中定期取得少量真實標籤，以滑動視窗監控預測誤差，並與 PSI 比較兩者對不同漂移類型的敏感度。

七、補充題

39. 列出 CRISP-DM 的六個階段。

40. 列出表 22-6 中工業機器學習管線的七個階段。

提示與參考解答

1. C。

2. B。

3. B。

4. B。

5. B。

6. A。

7. A。

8. A。

9. B。

10. A。

11. 參考：資料少而貴（重視實驗設計與小樣本方法）、標籤稀少不平衡（不能只看準確率，需異常偵測或合成資料）、有批次結構（分組切分、考慮混淆因子）、會漂移（監控與重訓）、有物理（結合領域知識與物理模型）。

12. 參考：p 值只說明差異是否可能是偶然，不說明差異多大；信賴區間給出可能的範圍，效果量衡量實際大小，兩者才能判斷工程意義。

13. 參考：同時影響原因與結果的變數。範例中機台世代同時影響溫度設定與良率，使溫度看似提升良率，控制機台後才看到溫度的負面影響。

14. 參考：模型在訓練資料上表現很好、在新資料上變差。以驗證集或交叉驗證偵測；以限制模型複雜度、正規化、更多資料與適當的特徵避免。

15. 參考：同一批料、模具或時間的樣本彼此相似，隨機切分會讓測試集中出現模型「認得」的資訊。應依上線情境依批料、模具或時間切分。

16. 參考：CNN 以共享權重擷取局部特徵，適合影像與訊號；LSTM 以閘門記住長期資訊，適合時間序列；Transformer 以注意力直接參考所有位置、可平行運算，適合長序列與多模態。

17. 參考：不良率低時，全判良品也有很高的準確率卻抓不到不良品。應看召回率、精確率、混淆矩陣、AUC，並以成本評估。

18. 參考：資料漂移是輸入分布改變，可用 PSI 等統計量監控；概念漂移是輸入與輸出的關係改變，需以真實標籤監控預測誤差。

19. $\min(20.015 - 20.003, 20.003 - 19.985) \div (3 \times 0.004) = 0.012 \div 0.012 = 1.00$ 。

20. $100 \times 0.05 = 5$ 個； $0.05 \div 100 = 0.0005$ 。

21. 召回率 $46 \div 50 = 92\%$ ；精確率 $46 \div 230 = 20\%$ ；準確率 $(46 + 9,766) \div 10,000 = 98.12\%$ 。

22. 準確率 99%，召回率 0%。

23. $200 \div (200 + 5,000) \approx 0.038$ 。

24. 介於 0.1 與 0.25 之間，表示有中度變化，需要注意並了解原因。

25. $8 \times 16 + 8 + 8 + 1 = 145$ 。

26. $\sqrt{64} = 8$ 。

27. 評分重點：決策與成本明確；資料與標籤定義合理並考慮混淆因子；切分方式模擬上線；門檻依成本設計並有人工覆核；監控、重訓與負責人清楚。

28. 評分重點：能以辛普森悖論與資料洩漏說明資料量無法取代領域知識；指出領域知識在問題定義、特徵、混淆因子與結果解讀上的角色。

29. 評分重點：梯度提升樹在表格資料上準確、資料需求少、較易解釋；深度學習擅長影像、訊號等非結構化資料但需大量資料。選擇依資料型態、資料量、可解釋性與部署條件。

30. X。可能只是樣本太少、檢定力不足。

31. ✓。

32. X。可能過度配適，應以新資料評估。

33. ✓。

34. X。應依漏判與誤判的成本決定。

35. X。可能發生概念漂移，需以真實標籤監控。

36. 評分重點：網路結構與訓練正確；以分組方式評估不同轉速；視覺化結果能對應到衝擊的位置。

37. 評分重點：時間切分正確；比較指標合理；能說明保養前後模型表現的差異與更新策略。

38. 評分重點：偵測器設計合理，能以模擬顯示 PSI 對資料漂移敏感、預測誤差監控對概念漂移敏感。

39. 業務理解、資料理解、資料準備、建模、評估、部署。

40. 問題定義、資料蒐集與理解、資料準備與特徵、建模與驗證、決策設計、部署、監控與維護。

第 23 章

製造資料工程與工業分析

編著 李世淵 (機器人叫獸) | 助教 李天宇、李宇晴

本章重點

- 製造資料的類型、情境化與資料品質檢查
- Historian、旋轉門壓縮、串流與資料湖倉
- 時序特徵、包絡分析與軸承缺陷頻率
- PCA 的 T^2 與 SPE 異常偵測、貢獻圖與根因分析
- 資料治理、資料契約、語意層與工業知識圖譜
- MLOps：漂移偵測、冠軍挑戰者與影子部署

叫獸開講：八成的時間在整理資料

有一次，我請一位剛到工廠報到的資料科學碩士生，記錄他第一個專案的時間分配。專案的目標很明確：預測一台注塑機的不良品。三個月後，他給我看他的紀錄，自己都嚇了一跳：真正在「建模」的時間不到一成，八成以上的時間花在找資料、搬資料、對資料、整理資料。

他遇到的問題五花八門。溫度感測器有幾天「卡住」，數值一直不變，資料庫卻照樣存著；某次韌體更新後，壓力的單位從 bar 變成了 MPa，沒有人通知；機台的時鐘比 MES 快了三分鐘，資料對不起來；最麻煩的是，歷史資料庫裡只有一串串的點位名稱，例如「IMM07_AI_023」，沒有人記得那是哪一個感測器、裝在哪裡；不良品的紀錄在品保的 Excel 裡，只記到「哪一天、哪一班」，對不到是哪一模、哪一件。

他說：「我以為我的工作是在做 AI，結果我的工作是在做資料考古。」

這不是個案。許多工廠的 AI 專案，最後卡住的地方都不是演算法，而是資料：資料不完整、不可信、對不上、看不懂。資料工程，就是讓資料「可用」的工作：把資料可靠地蒐集起來、加上情境、檢查品質、妥善儲存、清楚定義、持續治理，讓分析與 AI 可以建立在可信的基礎上。

這一章，我們從製造資料的類型與品質談起，介紹歷史資料庫、串流與資料湖倉等儲存與處理架構；以時序特徵、頻譜分析、異常偵測與根因分析示範工業分析的常用方法；討論資料治理、語意層與工業知識圖譜；最後介紹讓模型持續可靠運作的 MLOps 與模型漂移監測。

（說明：本節情境由作者指導學生與企業合作的經驗綜合改寫，人物與細節已調整。）

學習目標

讀完本章後，你應該能夠：

1. 說明製造資料的類型與情境化的意義，並以程式檢查資料品質、依時間對齊工單資料。
2. 說明歷史資料庫、串流處理、事件資料與資料湖倉的角色，並以程式實作旋轉門壓縮演算法。
3. 從振動訊號中計算時域特徵與頻譜，並以包絡分析找出軸承缺陷的特徵頻率。
4. 以主成分分析建立多變量的正常模型，以 T^2 與 SPE 偵測異常，並以貢獻圖輔助根因分析。

5. 說明資料治理、資料契約、語意層與工業知識圖譜的角色，並以程式實作資料契約檢查。
6. 說明 MLOps 與 ModelOps 的流程，並以影子部署、冠軍挑戰者與漂移檢定示範模型的持續維運。

核心概念

表 23-1 本章核心名詞中英對照

中文名詞	英文	一句話說明
時序資料	Time-Series Data	依時間記錄的連續量測，例如溫度、壓力、振動
事件資料	Event Data	在特定時間發生的離散事件，例如警報、開停機、工單開始
情境化	Contextualization	為資料加上設備、工單、產品、批號等背景資訊
資料品質	Data Quality	資料的完整性、正確性、一致性、即時性等特性
歷史資料庫	Historian	專門儲存與查詢大量工業時序資料的系統
旋轉門壓縮	Swinging Door Trending	只保留必要轉折點、誤差在容差內的時序壓縮法
串流處理	Stream Processing	資料到達時即時處理，而非先儲存再批次處理
資料湖倉	Data Lakehouse	結合資料湖的彈性與資料倉儲的管理能力的架構
包絡分析	Envelope Analysis	解調高頻共振訊號、找出衝擊重複頻率的振動分析法
主成分分析	Principal Component Analysis, PCA	以少數方向描述多變數之間相關結構的降維方法
SPE	Squared Prediction Error	樣本與主成分模型的殘差平方和，偵測相關結構的破壞
資料契約	Data Contract	資料生產者與使用者對欄位、單位、範圍、頻率的約定
語意層	Semantic Layer	把原始資料對應到一致的業務與工程意義的中介層
MLOps	Machine Learning Operations	讓機器學習模型可靠開發、部署、監控與更新的工程實務
影子部署	Shadow Deployment	新模型與線上模型並行預測，但不影響實際決策

23.1 製造資料的類型、情境化與資料品質

23.1.1 製造資料的類型

表 23-2 製造資料的主要類型

類型	特徵	例子	典型來源	本書相關章節
時序資料	大量、連續、固定或不定頻率	溫度、壓力、電流、振動	PLC、SCADA、感測器	第 10、11 章
事件資料	離散、帶時間戳記與屬性	警報、開停機、換模、工單開始與結束	SCADA、MES	第 17 章
交易與主資料	結構化、相對穩定	工單、BOM、途程、料號	ERP、MES、PLM	第 12、17 章
品質資料	抽樣、延遲、常與產品序號連結	尺寸量測、化驗、檢驗結果	QMS、LIMS、CMM	第 25 章
影像與聲音	高維、非結構化	外觀影像、X 光、聲紋	相機、麥克風	第 24 章
文件與文字	非結構化、語意豐富	維修紀錄、作業指導書、異常報告	文件系統、CMMS	第 28 章

這些資料的頻率、格式與擁有者都不同。一個典型的分析問題，例如「為什麼這批產品的不良率偏高？」，往往需要同時使用好幾種：時序資料告訴我們製程參數，事件資料告訴我們何時換模、何時警報，交易資料告訴我們是哪一張工單、哪一批原料，品質資料告訴我們結果。把它們連在一起，就是情境化。

23.1.2 情境化：讓資料知道自己是誰

歷史資料庫中的一筆資料，原始的樣子可能只是「IMM07_AI_023，2026-09-01 08:15:20，61.3」。人看不懂，模型也無從理解。情境化就是為資料加上背景：這是 7 號注塑機的模具溫度，單位 °C；當時正在生產工單 WO-101，產品是軸承座 A，使用原料批號 R-3321，由早班操作。

情境化有三個層次：

- 資產情境：資料來自哪一台設備、哪一個位置、什麼感測器、什麼單位。這可以由第 20 章的資產管理殼或第 11 章的統一命名空間提供。
- 生產情境：資料發生時在做什麼工單、產品、批號與製程步驟。這需要依時間對齊 MES 的事件。
- 品質情境：這段時間生產的產品，品質結果如何。這需要第 17 章的製造履歷把產品序號與時間連結起來。

23.1.3 資料品質

資料品質的常見面向包括：完整性（有沒有缺值）、正確性（數值是否反映真實）、一致性（不同系統、不同時間的定義是否一致）、即時性（資料多久才到）與唯一性（有沒有重複）。在製造資料中，最常見的問題有：

- 缺值：通訊中斷、設備停機、閘道器重啟。
- 卡值：感測器故障或通訊凍結，數值長時間完全不變，資料庫卻照樣記錄。
- 超出範圍：通訊錯誤碼（例如 999、-9999）被當成量測值寫入。
- 重複與亂序：閘道器重送、晚到的資料，造成重複時間戳與時間倒退。
- 單位與定義改變：韌體更新、設備更換後，單位或點位意義改變。
- 時鐘不同步：不同系統的時間差幾秒到幾分鐘，導致對齊錯誤（第 11 章的時間同步）。

下面的程式模擬一段 100 分鐘的模具溫度資料，刻意加入卡值、超出範圍、缺值、重複與晚到的資料，以程式自動檢查與清理，再以 pandas 的 merge_asof 依時間把每一筆資料對齊到當時的工單與產品：

```
import numpy as np, pandas as pd
# 資料品質與情境化：檢查感測資料的常見問題，再依時間對齊 MES 工單，讓每筆資料知道「在做哪一張工單、哪個產品」
rng = np.random.default_rng(0)
t = pd.date_range("2026-09-01 08:00", periods=600, freq="10s")
temp = 60 + 5 * np.sin(np.arange(600) / 60) + rng.normal(0, 0.3, 600)
temp[100:130] = temp[99]          # 卡值：感測器凍結 30 筆
temp[250] = 999.0                 # 超出量測範圍（通訊錯誤碼）
temp[400:405] = np.nan           # 缺值
df = pd.DataFrame({"ts": t, "temp": temp})
df = pd.concat([df, df.iloc[[300, 301]]]).reset_index(drop=True)      # 重複資料
df.loc[len(df)] = [pd.Timestamp("2026-09-01 08:05:00"), 61.0]      # 晚到、時間順序
```

錯亂的資料

```
def quality(df, lo=0, hi=200, stuck_n=10):
    r = {"筆數": len(df), "缺值": int(df.temp.isna().sum()),
        "重複時間戳": int(df.ts.duplicated().sum()),
        "時間倒退": int((df.ts.diff().dt.total_seconds() < 0).sum()),
        "超出範圍": int(((df.temp < lo) | (df.temp > hi)).sum())}
    runs = (df.temp.diff() != 0).cumsum(); r["卡值段數"] =
int((df.groupby(runs).temp.transform("size") >= stuck_n).groupby(runs).first().sum())
    return r
wo = pd.DataFrame({"start": pd.to_datetime(["2026-09-01 08:00", "2026-09-01 08:40",
"2026-09-01 09:10"]),
    "work_order": ["WO-101", "WO-102", "WO-103"], "product": ["軸承座 A", "
軸承座 A", "法蘭 B"]})
print("清理前:", quality(df))
clean = df.drop_duplicates("ts").sort_values("ts")
clean = clean[(clean.temp >= 0) & (clean.temp <= 200)]
runs = (clean.temp.diff() != 0).cumsum(); stuck =
clean.groupby(runs).temp.transform("size") >= 10
clean = clean[~stuck].dropna()
print("清理後:", quality(clean))
ctx = pd.merge_asof(clean, wo, left_on="ts", right_on="start")      # 依時間對齊工單（情
境化）
print(ctx.groupby(["work_order", "product"]).temp.agg(["count", "mean"]).round(2))
```

執行結果：清理前共 603 筆，檢查出缺值 5 筆、重複時間戳 3 筆、時間倒退 2 處、超出範圍 1 筆、卡值 1 段。清理後剩下 563 筆，各項檢查都歸零。情境化之後，每一筆資料都帶有工單與產品：WO-101 (軸承座 A) 209 筆、平均 61.69 °C · WO-102 (軸承座 A) 174 筆、平均 57.50 °C · WO-103 (法蘭 B) 180 筆、平均 62.68 °C 。

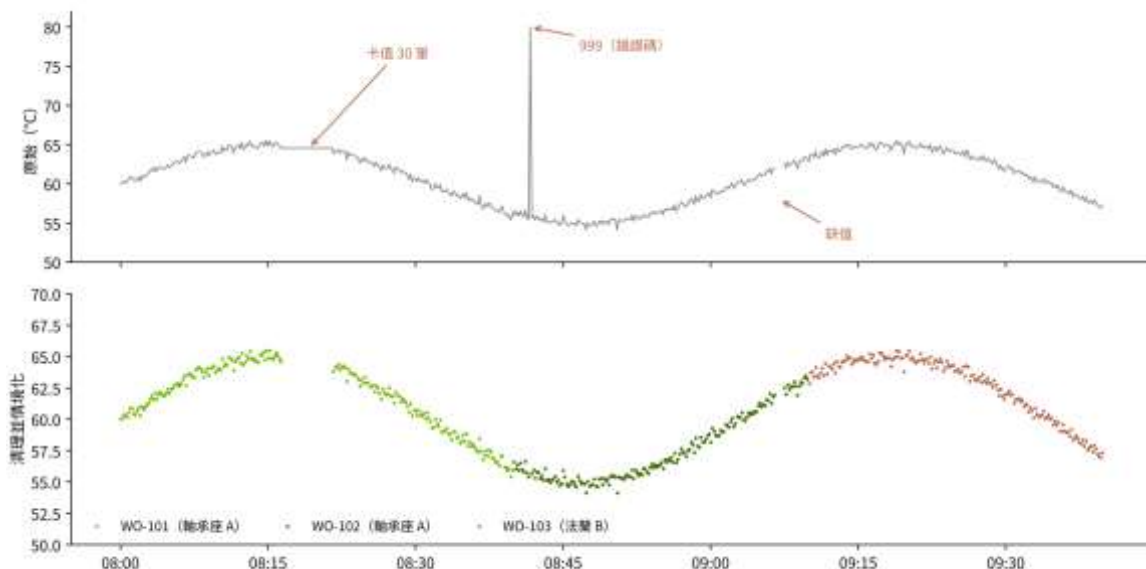


圖 23-1 資料品質問題與情境化

這個例子有兩點值得注意。第一，卡值最危險：它在數值上看起來完全正常，只有檢查「是否長時間不變」才會發現；若沒有清除，模型會誤以為那段時間製程非常穩定。第二，清理的規則本身也

需要領域知識與紀錄：「連續 10 筆完全相同」在溫度上很可疑，在開關訊號上卻很正常；被清除的資料也應保留標記，而不是默默刪除，讓之後的分析知道哪裡曾經有問題。

23.2 Historian、串流、事件資料與資料湖倉

23.2.1 製造資料的儲存架構

製造資料從現場到分析，通常經過幾個層次：



圖 23-2 製造資料平台的分層架構

- 邊緣：PLC、感測器與邊緣閘道，負責蒐集、初步過濾與緩衝，網路中斷時暫存資料（第 11 章）。
- 歷史資料庫（Historian）：工廠內專門儲存時序資料的系統，擅長高頻寫入、壓縮與依時間查詢，常與 SCADA 和 DCS 搭配（第 18 章）。
- 串流平台：讓資料在到達時就被處理與分送，例如以 MQTT 或 Kafka 等訊息平台傳遞，以串流處理引擎即時計算統計與警報。
- 資料湖與資料倉儲：資料湖以低成本儲存大量原始資料，格式不拘；資料倉儲儲存整理過、結構化的資料，供報表與分析。
- 資料湖倉：近年興起的架構，在資料湖的開放檔案格式上加入交易、版本與結構管理，讓同一份資料同時支援分析報表與機器學習。Databricks 的研究人員在 CIDR 2021 研討會上發表的論文，系統性地提出了這個概念。

這些層次不是非此即彼。一個常見的組合是：歷史資料庫負責工廠內的即時與短中期資料，串流平台把資料送到企業的資料湖倉，進行跨廠區、長期的分析與 AI 訓練。

23.2.2 歷史資料庫與壓縮

一座工廠可能有數萬個點位，每秒記錄一次，一年就是數千億筆資料。歷史資料庫的一個關鍵技術是壓縮：多數時候製程是穩定的，不需要每一點都存。

常用的方法有兩種。死區法：數值變化超過設定的死區才記錄。旋轉門法 (Swinging Door Trending) 則更聰明：從最後一個保留的點出發，想像有上下兩扇「門」，門軸在該點上下各偏離容差的位置；每來一個新點，兩扇門就各自往能涵蓋這個點的方向轉；當兩扇門轉到交錯，表示無法再用一條直線在容差內涵蓋所有點，就保留前一點，從那裡重新開始。這個方法由 Foxboro 公司的 Edgar H. Bristol 提出，美國專利 US4669097 (1985 年申請、1987 年核准) 即以「旋轉門法」為名，之後被許多商用歷史資料庫採用。

下面的程式實作旋轉門壓縮，對一小時、每秒一點的溫度資料 (包含一段緩慢的上升與一次階躍)，比較不同容差下的壓縮比與重建誤差，並與「每 60 秒取一點」的固定取樣比較：

```
import numpy as np
# Historian 的旋轉門 (Swinging Door) 壓縮：只保留必要的轉折點，誤差不超過設定的容差
rng = np.random.default_rng(1)
t = np.arange(3600.0) # 1 小時、每秒 1 點
x = 80 + 3 * np.tanh((t - 1200) / 120) + 0.8 * np.sin(t / 300) + rng.normal(0, 0.05, t.size)
x[2400:] += 1.5 # 一次階躍變化
def swinging_door(t, x, dev):
    keep = [0]; a = 0; up, lo = np.inf, -np.inf # 從已保留的點 a 出發，維護上下兩扇門的斜率
    for i in range(1, len(t)):
        dt = t[i] - t[a]
        up = min(up, (x[i] + dev - x[a]) / dt); lo = max(lo, (x[i] - dev - x[a]) / dt)
        if lo > up: # 兩扇門打開到交錯：前一點必須保留
            a = i - 1; keep.append(a)
            dt = t[i] - t[a]; up = (x[i] + dev - x[a]) / dt; lo = (x[i] - dev - x[a]) / dt
    keep.append(len(t) - 1); return np.array(keep)
for dev in [0.05, 0.1, 0.2, 0.5]:
    k = swinging_door(t, x, dev); xr = np.interp(t, t[k], x[k])
    print(f"容差 {dev:.2f}：保留 {len(k):4d} 點，壓縮比 {len(t)/len(k):5.1f} 倍，最大重建誤差 {np.abs(xr-x).max():.3f}")
k = np.arange(0, 3600, 60); xr = np.interp(t, t[k], x[k])
print(f"固定每 60 秒取樣：保留 {len(k)} 點，最大重建誤差 {np.abs(xr-x).max():.3f}")
```

執行結果：

表 23-3 旋轉門壓縮的結果 (3,600 點)

方法	保留點數	壓縮比	最大重建誤差
旋轉門，容差 0.05	1,355	2.7 倍	0.091
旋轉門，容差 0.10	219	16.4 倍	0.184
旋轉門，容差 0.20	15	240.0 倍	0.298
旋轉門，容差 0.50	8	450.0 倍	0.805
固定每 60 秒取樣	60	60 倍	1.573

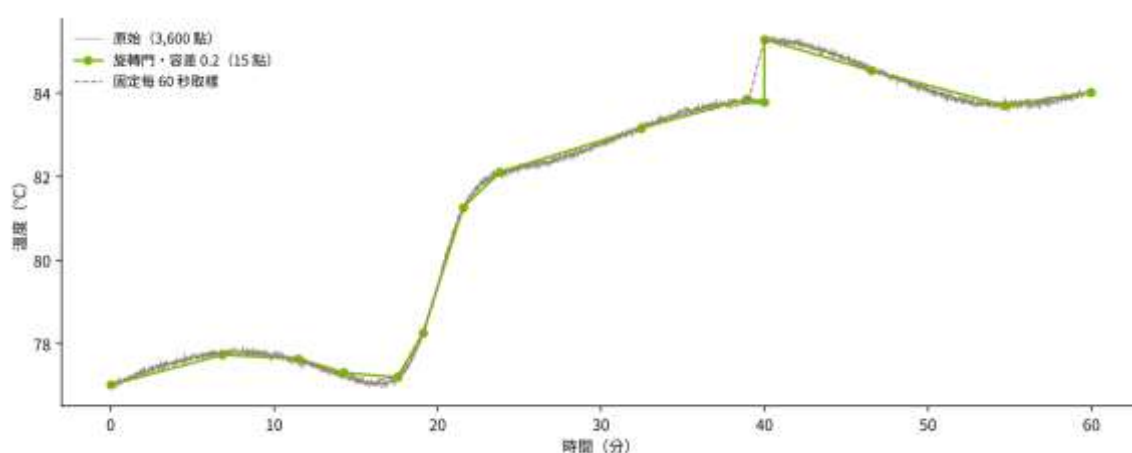


圖 23-3 旋轉門壓縮：保留的轉折點與重建曲線

容差為 0.05 時，因為雜訊的標準差也是 0.05，大部分的點都被當成「轉折」保留，壓縮比只有 2.7 倍；容差 0.2 時，只保留 15 點就重建出整條曲線，壓縮比高達 240 倍。相比之下，固定每 60 秒取一點雖然有 60 倍的壓縮比，卻在階躍處產生約 1.57 的誤差：它不知道哪裡重要。在這個簡化的實作中，最大重建誤差約為容差的 1.5~2 倍，與容差同一數量級。

壓縮是一個取舍：容差太大，會抹掉對分析有用的細節，例如短暫的尖峰或振盪；容差太小，壓縮效果有限。實務上，容差應依感測器的精度與分析需求逐點設定。更重要的是，若要做振動或高頻的分析，原始的高頻資料應另外以適當的方式保存，不能依賴為趨勢監控而壓縮過的歷史資料。

23.2.3 串流與事件

批次處理是先把資料存起來，再定時（例如每天晚上）計算；串流處理則在資料到達時就計算。製造中需要串流的情況很多：即時的統計製程管制、設備異常警報、虛擬量測的逐片預測（第 21

章)、即時的產量與稼動率看板。

串流處理有幾個與批次不同的概念：

- 時間視窗：以固定長度（例如每 1 分鐘）或滑動的視窗計算統計量。
- 事件時間與處理時間：資料「發生」的時間與「被處理」的時間不同；晚到的資料要依事件時間放回正確的視窗。
- 狀態與恰好一次：串流計算常要保留狀態（例如累計產量），並確保系統重啟後不會重複或遺漏計算。

事件資料，例如開停機、警報與工單狀態，常常比時序資料更有價值，因為它們直接描述了「發生了什麼」。第 17 章的工單事件、第 26 章的警報與維修事件，都是分析停機原因與設備可靠度的基礎。

23.3 時序特徵、頻譜分析、異常偵測與根因分析

23.3.1 時序特徵

時序資料很少直接丟進模型，通常先轉換為特徵。常用的特徵可以分為幾類：

表 23-4 常用的時序特徵

類別	特徵	意義	典型應用
時域統計	平均、標準差、均方根、峰對峰值	訊號的水準與能量	溫度、電流的監控
形狀	峰度、偏度、波峰因子	衝擊與不對稱的程度	軸承與齒輪的早期損傷
趨勢	斜率、變化率、累積量	緩慢的漂移與劣化	刀具磨耗、結垢
頻域	頻譜峰值、頻帶能量	週期性的來源	不平衡、對心不良、軸承缺陷
時頻	短時傅立葉轉換、小波	頻率隨時間的變化	啟停過程、間歇性故障
事件	次數、持續時間、間隔	離散的行為模式	警報頻率、開停機次數

特徵的設計要結合物理：知道故障會造成什麼樣的訊號，才知道該算什麼特徵。這也是第 22 章所說「好的特徵工程常與深度學習一樣有效」的原因。

23.3.2 頻譜與包絡分析

旋轉機械的許多故障，都有自己的「特徵頻率」。例如滾動軸承的外環有缺陷時，每一顆滾珠經過缺陷處，就會產生一次衝擊，衝擊的頻率稱為外環缺陷頻率（Ball Pass Frequency of the Outer race，BPFO），可以由軸承的幾何與轉速計算：

$$\text{BPFO} = (n \div 2) \times \text{fr} \times [1 - (d \div D) \times \cos \varphi]$$

其中 n 為滾珠數， fr 為軸的轉速頻率， d 為滾珠直徑， D 為節圓直徑， φ 為接觸角。

問題是，早期的衝擊很微弱，而且每次衝擊激發的是軸承座的高頻共振，在原始的頻譜中，BPFO 的能量很小，被轉速頻率與雜訊掩蓋。包絡分析的步驟是：先以帶通濾波器留下共振所在的頻帶，再以希伯特轉換取出訊號的「包絡」（振幅的起伏），最後對包絡做頻譜分析，衝擊的重複頻率就會清楚地出現。

下面的程式模擬一個外環有缺陷的軸承：轉速 1,500 rpm（25 Hz），9 顆滾珠，滾珠直徑 7.9 mm，節圓直徑 34.5 mm。每次滾珠經過缺陷，激發 3 kHz 的衰減共振，再加上轉速頻率、二倍頻與較大的雜訊：

```
import numpy as np
from scipy.signal import hilbert, butter, filtfilt
# 頻譜分析：軸承外環缺陷在原始頻譜中被雜訊與高頻共振掩蓋，以包絡分析找出缺陷特徵頻率（BPFO）
fs, T = 20000, 1.0; t = np.arange(0, T, 1 / fs); rng = np.random.default_rng(2)
shaft = 25.0 # 轉速 1500 rpm → 25 Hz
n, d, D = 9, 7.9, 34.5 # 滾珠數、滾珠直徑、節圓直徑
(mm), 接觸角 0
bpfo = n / 2 * shaft * (1 - d / D) # 外環缺陷特徵頻率
x = 0.5 * np.sin(2 * np.pi * shaft * t) + 0.2 * np.sin(2 * np.pi * 2 * shaft * t) +
rng.normal(0, 0.6, t.size)
for k in np.arange(0.01, T, 1 / bpfo): # 每次滾珠經過缺陷，激發 3 kHz 的結構共振
    i = int(k * fs); m = min(400, t.size - i); tt = np.arange(m) / fs
    x[i:i + m] += 1.0 * np.exp(-tt * 800) * np.sin(2 * np.pi * 3000 * tt)
def peak(f, A, lo, hi):
    s = (f > lo) & (f < hi); return f[s][np.argmax(A[s])]
f = np.fft.rfftfreq(t.size, 1 / fs)
print(f"理論 BPFO = {bpfo:.1f} Hz；均方根 {np.sqrt(np.mean(x**2)):.2f}，峰度 {np.mean((x-x.mean())**4)/x.var()**2:.2f}")
A = np.abs(np.fft.rfft(x)) / t.size
print(f"原始頻譜 5~200 Hz 最大峰：{peak(f, A, 5, 200):.1f} Hz（轉速頻率）")
b, a = butter(4, [2000, 4000], btype="band", fs=fs) # 帶通濾波留下共振頻帶
env = np.abs(hilbert(filtfilt(b, a, x))) # 希伯特轉換取包絡
E = np.abs(np.fft.rfft(env - env.mean())) / t.size
print(f"包絡頻譜 5~200 Hz 最大峰：{peak(f, E, 5, 200):.1f} Hz → 與 BPFO 相符")
```

執行結果：理論的 BPFO 約為 86.7 Hz。訊號的均方根約 0.73，峰度約 2.93，與常態雜訊的 3 相

近，從時域統計看不出明顯異常。原始頻譜在 5~200 Hz 的最大峰是 25.0 Hz 的轉速頻率，看不到缺陷；包絡頻譜的最大峰則出現在 87.0 Hz（頻率解析度為 1 Hz），與理論的 BPFO 相符，缺陷被清楚地找出來。

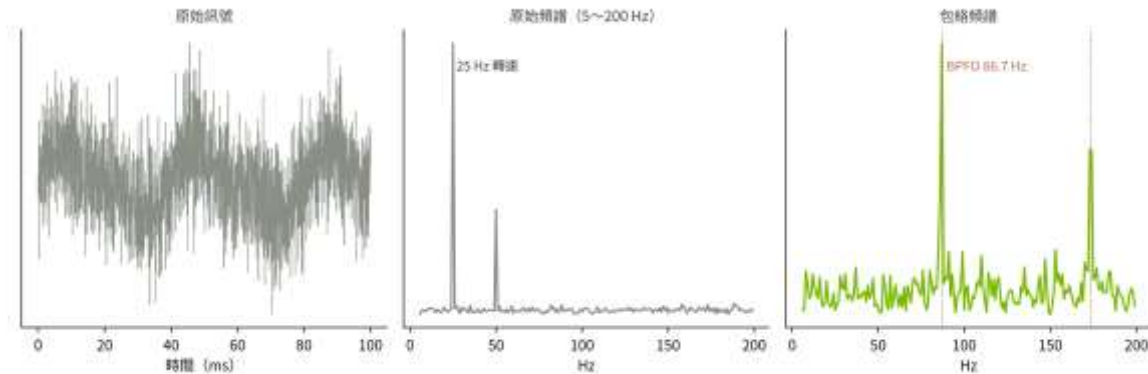


圖 23-4 原始頻譜與包絡頻譜：找出軸承外環缺陷頻率

這個例子說明了物理知識在工業分析中的力量：知道軸承的幾何與轉速，就知道該在哪個頻率找證據；知道衝擊會激發共振，就知道要先濾波再取包絡。同樣的方法可以找出內環、滾珠與保持架的缺陷頻率，是第 26 章預測維護的基本工具。

23.3.3 多變量異常偵測

製程中的許多變數彼此相關：進料流量增加，反應溫度、夾套溫度與攪拌電流都會跟著變化。只看單一變數的管制圖（第 25 章），可能錯過「每個變數都在範圍內，但它們之間的關係不對」的異常。多變量統計製程監控以主成分分析描述正常時變數之間的相關結構，再以兩個統計量監控：

- T^2 ：樣本在主成分空間中距離正常中心多遠，偵測「整體負荷或操作點的異常」。
- SPE（又稱 Q 統計量）：樣本與主成分模型的殘差，偵測「變數之間的相關結構被破壞」，例如感測器漂移或新的故障模式。

偵測到異常後，以貢獻圖找出哪些變數對 T^2 或 SPE 的貢獻最大，作為根因分析的起點。

下面的程式模擬一個有 6 個變數的反應器：正常時，6 個變數由「負荷」與「原料品質」兩個潛在因子決定。以 1,000 筆正常資料建立主成分模型與 99% 的管制界限，再模擬從第 100 點開始，夾套溫度感測器逐漸漂移：

```
import numpy as np
# 多變量異常偵測與根因：以主成分分析建立正常模型，用  $T^2$  與 SPE 偵測異常，再以貢獻圖找出可疑變數
```

```

rng = np.random.default_rng(3)
names = ["進料流量", "反應溫度", "夾套溫度", "壓力", "攪拌電流", "產品濃度"]
def process(n, fault=None, t0=0):
    z = rng.normal(0, 1, (n, 2)) # 兩個潛在因子：負荷與原料品質
    L = np.array([[1, .2], [.8, .5], [.7, .4], [.6, -.3], [.5, .6], [.3, .9]])
    X = z @ L.T + rng.normal(0, 0.15, (n, 6))
    if fault == "jacket": X[t0:, 2] += np.linspace(0, 1.5, n - t0) # 夾套溫度感測器漂
移（破壞變數間的相關）
    return X
Xn = process(1000); mu, sd = Xn.mean(0), Xn.std(0); Z = (Xn - mu) / sd
U, S, Vt = np.linalg.svd(Z, full_matrices=False); k = 2; P = Vt[:k].T; lam = S[:k]**2 /
(len(Z) - 1)
def stats_(X):
    Z = (X - mu) / sd; T = Z @ P
    t2 = np.sum(T**2 / lam, axis=1); E = Z - T @ P.T; spe = np.sum(E**2, axis=1)
    return t2, spe, E
t2n, spen, _ = stats_(Xn); lim_t2, lim_spe = np.percentile(t2n, 99), np.percentile(spen,
99)
print(f"前兩個主成分解釋變異 {np.sum(S[:2]**2)/np.sum(S**2):.1%}; 管制界限 T² {lim_t2:.2f},
SPE {lim_spe:.2f}")
Xf = process(300, "jacket", 100); t2, spe, E = stats_(Xf)
fa = np.mean(spe[:100] > lim_spe); first = 100 + int(np.argmax(spe[100:] > lim_spe))
print(f"故障前誤報率 {fa:.0%}; 故障自第 100 點開始，之後 SPE 首次超限於第 {first} 點；T² 超限比
例 {np.mean(t2[100:] > lim_t2):.0%}, SPE {np.mean(spe[100:] > lim_spe):.0%}")
contrib = np.mean(E[200:]**2, axis=0)
for i in np.argsort(-contrib)[:3]: print(f" SPE 貢獻：{names[i]}
{contrib[i]/contrib.sum():.0%}")
print(f"對照：夾套溫度單變數 ±3σ 管制超限比例 {np.mean(np.abs((Xf[100:,2]-mu[2])/sd[2]) >
3):.0%}")

```

執行結果：前兩個主成分解釋了約 98.0% 的變異，99% 管制界限為 T^2 約 9.23、SPE 約 0.39。故障前的誤報率約 1%，符合 99% 界限的預期。故障開始後，SPE 在第 149 點首次超限，之後 200 點中約 65% 超限； T^2 的超限比例只有約 2%，因為漂移並沒有讓整體負荷異常，而是破壞了夾套溫度與其他變數之間的關係。貢獻圖顯示，夾套溫度對 SPE 的貢獻約占 74%，反應溫度與進料流量分別只有約 7% 與 6%，直接指向夾套溫度。

作為對照，如果只對夾套溫度做單變數的 $\pm 3\sigma$ 管制，故障期間只有約 6% 的點超限：漂移量 1.5 相對於夾套溫度本身因負荷變化而產生的大幅波動並不顯眼，單變數管制圖幾乎看不出問題。

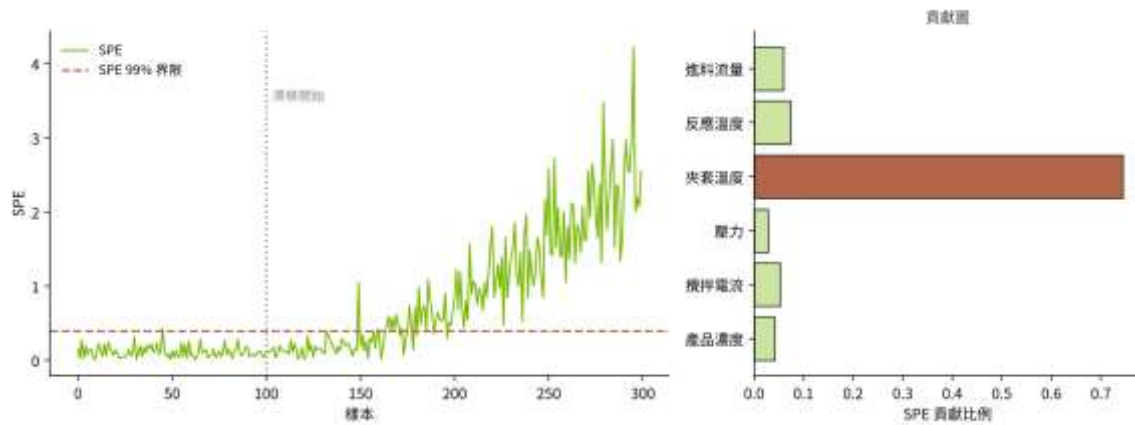


圖 23-5 以 PCA 的 SPE 偵測感測器漂移與貢獻圖

23.3.4 根因分析

偵測到異常只是第一步，真正有價值的是找出原因。常用的方法包括：

- 貢獻圖：如上例，指出哪些變數最異常，但「最異常的變數」不一定是「原因」，可能只是受影響的結果。
- 時間先後：哪個變數最先出現異常？原因通常先於結果。
- 分組比較：異常集中在哪台機台、哪批原料、哪個班別？這與第 20 章數位主線的向後追溯、第 22 章的混淆因子分析相同。
- 因果結構：以製程的流程圖、物理關係或知識圖譜（23.4 節）描述變數之間的因果方向，縮小可能的原因範圍。
- 魚骨圖與五個為什麼：以人機料法環測組織團隊的討論，資料分析提供證據（第 25 章）。

資料分析的角色，是把「大家的猜測」變成「有證據支持的假設」，再以實驗或現場確認。沒有領域專家參與的根因分析，常常停留在相關性而找不到真正的原因。

23.4 資料治理、語意層與工業知識圖譜

23.4.1 資料治理

資料治理是確保資料在整個組織中可信、可用、安全的制度。它回答的問題包括：每一類資料由誰負責？資料的定義是什麼？品質要求是什麼？誰可以存取？保存多久？出了問題誰處理？第 17 章的主資料治理、第 20 章的血緣與模型治理，都是資料治理的一部分。

表 23-5 製造資料治理的要素

要素	內容	本書相關章節
角色與責任	資料擁有者、資料管理員、使用者的權責	第 17、20 章
定義與目錄	點位、資料表、指標的定義、單位與來源	本章 23.4.3 節
品質	品質規則、監控與問題處理流程	本章 23.1 節
資料契約	生產者與使用者之間的格式、單位與服務水準約定	本章 23.4.2 節
血緣與版本	資料從哪裡來、經過哪些處理	第 20 章
存取與資安	權限、敏感資料、跨廠與跨企業分享	第 33、35 章
保存與刪除	保存期限、法規要求、封存	第 17 章

23.4.2 資料契約

叫獸開講中「韌體更新後單位從 bar 變成 MPa」的問題，根源在於資料的生產者（設備端）與使用者（分析團隊）之間沒有約定。資料契約把這種約定寫成可被程式檢查的規格：欄位名稱、型別、單位、合理範圍、更新頻率、負責人，以及變更時的通知程序。資料在進入資料平台之前就自動檢查，不符合契約的資料被攔下並通知負責人，而不是默默地污染後面的分析。

下面的程式以一個簡單的資料契約，檢查四則來自主軸監測的訊息：

```
# 資料契約：生產者（設備端、MES）與使用者（分析、AI）約定資料的欄位、型別、單位、範圍與更新頻率，
# 上線前自動檢查
contract = {"name": "cnc.spindle.v2", "owner": "設備課 王工程師", "freshness_s": 5,
            "fields": {"asset_id": {"type": str, "pattern": "CNC-"},
                       "ts": {"type": float},
                       "spindle_temp": {"type": float, "unit": "degC", "min": -10, "max": 150},
                       "spindle_speed": {"type": float, "unit": "rpm", "min": 0, "max": 24000}}}
def validate(msg, now):
    errs = []
    for k, rule in contract["fields"].items():
        if k not in msg: errs.append(f"缺少欄位 {k}"); continue
        v = msg[k]["value"] if isinstance(msg[k], dict) else msg[k]
        if not isinstance(v, rule["type"]): errs.append(f"{k} 型別錯誤 ({type(v).__name__})"); continue
        if "unit" in rule and msg[k].get("unit") != rule["unit"]: errs.append(f"{k} 單位 {msg[k].get('unit')}，契約為 {rule['unit']}")
        if "min" in rule and not rule["min"] <= v <= rule["max"]: errs.append(f"{k} = {v} 超出 [{rule['min']}, {rule['max']}]")
        if "pattern" in rule and not v.startswith(rule["pattern"]): errs.append(f"{k} 命名不符")
```

```

    if "ts" in msg and now - msg["ts"] > contract["freshness_s"]: errs.append(f"資料延遲
{now - msg['ts']:.0f} 秒")
    return errs or ["通過"]
msgs = {"正常": {"asset_id": "CNC-07", "ts": 100.0, "spindle_temp": {"value": 41.5,
"unit": "degC"}, "spindle_speed": {"value": 8000.0, "unit": "rpm"}},
        "韌體更新後改用華氏": {"asset_id": "CNC-07", "ts": 100.0, "spindle_temp": {"value":
106.7, "unit": "degF"}, "spindle_speed": {"value": 8000.0, "unit": "rpm"}},
        "新機台命名不一": {"asset_id": "cnc_09", "ts": 100.0, "spindle_temp": {"value":
39.0, "unit": "degC"}, "spindle_speed": {"value": 9000.0, "unit": "rpm"}},
        "開道器延遲且缺欄位": {"asset_id": "CNC-05", "ts": 80.0, "spindle_temp": {"value":
45.0, "unit": "degC"}}}
for k, m in msgs.items(): print(f"{k}: ", " ; ".join(validate(m, now=101.0)))

```

執行結果：正常的訊息通過檢查；韌體更新後改用華氏的訊息被攔下，理由是「spindle_temp 單位 degF，契約為 degC」，注意它的數值 106.7 仍在契約的範圍內，只檢查範圍是抓不到的；新機台的設備代碼寫成 cnc_09，命名不符；開道器延遲的訊息缺少 spindle_speed 欄位，且延遲 21 秒，超過 5 秒的新鮮度要求。

資料契約的價值，在於把問題擋在源頭，並讓責任清楚：契約上寫著負責人，攔下的問題就送給他處理，而不是由下游的分析團隊事後「考古」。它也與第 20 章的資產管理殼和語意識別碼相輔相成：資產管理殼定義「這個屬性是什麼」，資料契約定義「這份資料必須符合什麼」。

23.4.3 語意層與工業知識圖譜

叫獸開講中「IMM07_AI_023」這類點位名稱，是工業資料最常見的困境：原始的點位名稱是給控制系統用的，不是給人或分析用的。語意層在原始資料與使用者之間建立一層一致的「意義」：

- 命名與階層：依第 17 章 ISA-95 的設備階層（企業、廠區、區域、產線、工作單元），以一致的規則命名，例如「台南廠 / 射出區 / L2 / IMM07 / 模具溫度」，這也是第 11 章統一命名空間的做法。
- 屬性與單位：每一個點位有明確的物理量、單位、量測範圍與精度。
- 關係：這個感測器量測哪一台設備的哪一個部位？這台設備屬於哪條產線？目前在做哪張工單？這些關係可以用第 20 章的知識圖譜描述。
- 業務指標：稼動率、良率、單位能耗等指標的計算公式與資料來源，統一定義，避免各部門各算各的。

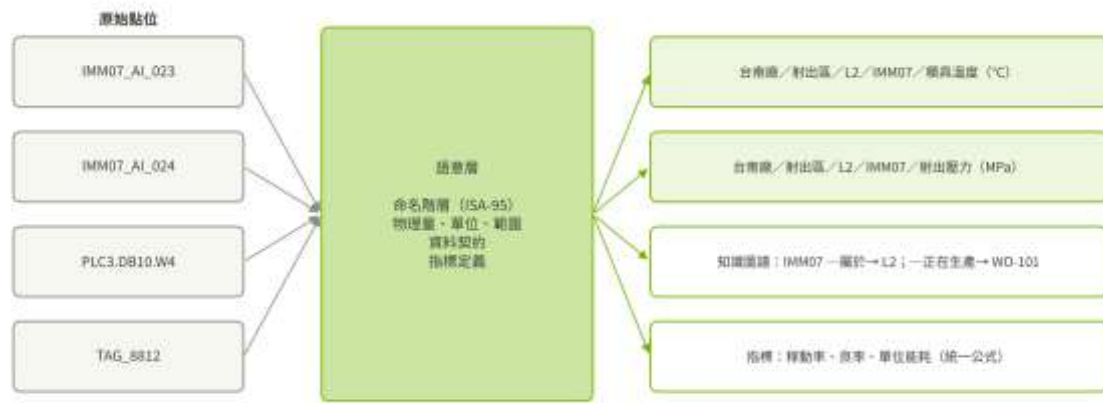


圖 23-6 語意層：從原始點位到可理解、可查詢的資料

有了語意層，分析人員可以問「台南廠所有射出機的模具溫度」，而不必記得幾百個點位名稱；第 28 章的企業 AI 助理，也能透過知識圖譜理解「7 號機昨天為什麼停機」這類問題，把它轉成對資料的查詢。語意層的建立需要大量的領域知識與整理工作，常是製造資料平台中最耗時、卻也最有價值的部分。

23.5 MLOps、ModelOps 與模型漂移監測

23.5.1 從專案到產品

第 22 章的工業 ML 管線指出，模型上線之後的工作與上線之前一樣多。MLOps (機器學習維運) 借用軟體工程的 DevOps 精神，把模型的開發、測試、部署、監控與更新自動化、標準化，讓模型像產品一樣被持續維護。ModelOps 則更廣，泛指企業中各種模型 (不只是機器學習，也包括物理模型、最佳化模型與規則) 的生命週期管理，與第 20 章的模型治理密切相關。

表 23-6 MLOps 的主要實務

實務	內容	本書相關章節
版本控制	程式、資料、特徵、模型與設定都有版本	第 20 章
特徵管理	訓練與上線使用同一套特徵定義與計算程式，避免不一致	本章 23.3 節
自動化訓練與測試	資料或程式變更時，自動重訓並以固定的測試集與門檻驗收	第 22 章
模型登錄	記錄每個版本的血緣、評估結果、核准狀態與負責人	第 20 章
部署策略	影子部署、金絲雀部署、冠軍挑戰者、可回復	本節

實務	內容	本書相關章節
監控	資料品質、資料漂移、預測分布、實際誤差、延遲與資源	本節、第 22 章
重訓與退場	依監控結果觸發重訓、升版或停用	第 20 章

製造環境還有幾個特別的考量：模型常部署在邊緣的工業電腦上，網路與運算有限；模型的輸出可能直接影響設備或放行決策，必須有安全限制與人工覆核；真實的標籤（量測與檢驗結果）常常延遲數小時到數天才回來；而且製程會因保養、換模、換料而突然改變。

23.5.2 部署策略

新模型不應該直接取代線上的模型。常用的部署策略包括：

- 影子部署：新模型與線上模型並行，接收相同的輸入、產生預測，但它的預測只被記錄、不影響實際決策。累積足夠的比較資料後，再決定是否升版。
- 冠軍挑戰者：線上的模型稱為冠軍，新模型稱為挑戰者；挑戰者必須在相同的資料上明顯勝過冠軍，才能取而代之。
- 金絲雀部署：先讓新模型只負責一小部分的產線或產品，確認沒有問題後再逐步擴大。
- 可回復：保留前一個版本，發生問題時能迅速回復。

23.5.3 漂移監測與自動化的升版流程

第 22 章介紹了以 PSI 監控輸入資料的漂移。當真實標籤可以取得時（例如虛擬量測的抽測值），更直接的方法是監控預測誤差本身。Page-Hinkley 檢定是一種經典的序列變化偵測方法：它累積「觀測值相對於目前平均的偏離」，當累積量相對於歷史最小值上升超過門檻時，就判定平均值發生了上升。

下面的程式把漂移偵測、重訓與影子部署串成一個流程：一個線上的膜厚預測模型（冠軍）原本表現良好，第 300 筆起因為換模具，第 1 個特徵的影響變強（概念漂移）。程式以 Page-Hinkley 檢定監控冠軍的絕對誤差，警報後蒐集 100 筆有量測的資料訓練挑戰者，再以影子模式比較兩者，挑戰者的誤差低於冠軍的 80% 才升版：

```

import numpy as np
# MLOps：線上模型（冠軍）與新模型（挑戰者）以影子模式並行，比較後再升版；並以 Page-Hinkley 檢定
監測預測誤差漂移
rng = np.random.default_rng(6)
N = 600
x = rng.normal(0, 1, (N, 3))
coef = np.array([4.0, -2.0, 1.0]); y = 500 + x @ coef + rng.normal(0, 1.0, N)
y[300:] += 3.0 * x[300:, 0] # 第 300 筆起：換模具，第 1 個特徵
的影響變強（概念漂移）
champ = lambda X: 500 + X @ coef # 冠軍：上線中的舊模型
def page_hinkley(err, delta=0.2, lam=15.0): # 對絕對誤差的平均上升做序列檢定
    m, s, smin = 0.0, 0.0, 0.0
    for i, e in enumerate(err):
        m += (e - m) / (i + 1); s += e - m - delta; smin = min(smin, s)
        if s - smin > lam: return i
    return None
err = np.abs(champ(x) - y)
print(f"冠軍模型 MAE：前 300 筆 {err[:300].mean():.2f}，後 300 筆 {err[300:].mean():.2f}")
alarm = page_hinkley(err); print(f"Page-Hinkley 於第 {alarm} 筆發出漂移警報（漂移自第 300 筆
開始）")
lab = np.arange(alarm + 1, alarm + 101) # 警報後蒐集 100 筆有量測的資料重訓
挑戰者
A = np.c_[np.ones(len(lab)), x[lab]]; w = np.linalg.lstsq(A, y[lab], rcond=None)[0]
chall = lambda X: w[0] + X @ w[1:]
shadow = np.arange(alarm + 101, N) # 影子模式：挑戰者只預測、不影響生產
mc, mch = np.abs(champ(x[shadow]) - y[shadow]).mean(), np.abs(chall(x[shadow]) -
y[shadow]).mean()
print(f"影子期間 {len(shadow)} 筆：冠軍 MAE {mc:.2f}，挑戰者 MAE {mch:.2f} → {'升版挑戰者' if
mch < 0.8 * mc else '維持冠軍'})
print("挑戰者學到的係數：", np.round(w[1:], 2))

```

執行結果：冠軍模型在前 300 筆的平均絕對誤差約 0.77，後 300 筆升到約 2.61。Page-Hinkley 檢定在第 316 筆發出警報，距離漂移開始只有 16 筆。以警報後的 100 筆資料訓練的挑戰者，在接下來 183 筆的影子期間平均絕對誤差約 0.87，冠軍約 2.62，挑戰者勝出，升版。挑戰者學到的第 1 個特徵係數約 7.13，正確反映了換模具後該特徵的影響由 4 變為約 7。

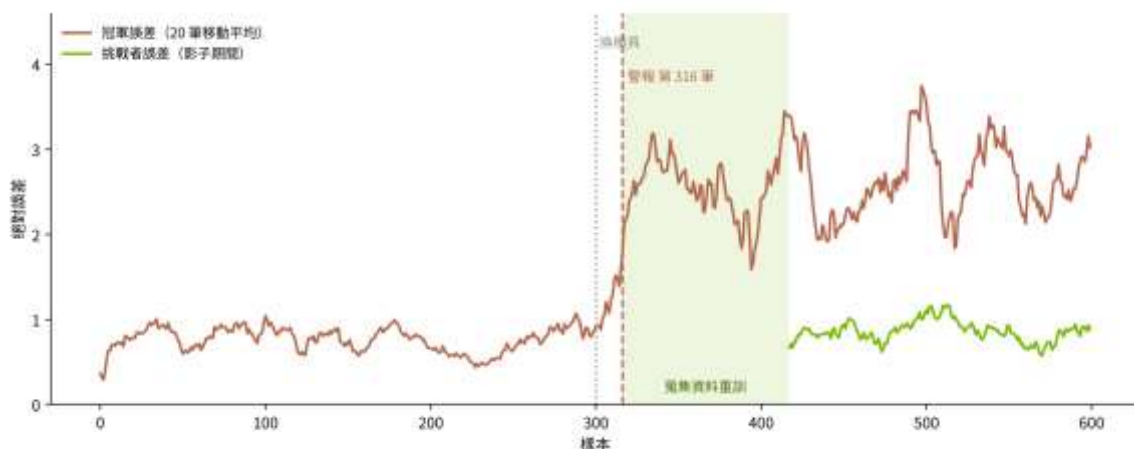


圖 23-7 漂移偵測、挑戰者訓練與影子部署

這個流程有幾個實務上的細節值得討論。第一，偵測的靈敏度與誤報之間有取捨：Page-Hinkley

的門檻越低，偵測越快，但正常的波動也越容易觸發警報。第二，從警報到升版之間，冠軍仍在線上，誤差偏高；若誤差的代價很大，應在警報時就啟動保護措施，例如增加實際量測的頻率，或暫時改由人工判斷，就像第 21 章虛擬量測在不可信時改用實測。第三，升版的決策應該記錄在模型登錄表中：哪一個版本、用了哪些資料、在什麼期間、以什麼指標勝出、由誰核准（第 20 章）。第四，換模具是一個已知的事件；如果 MES 的事件能自動通知模型平台，就能在漂移發生之前主動準備，這正是 23.1 節情境化的價值。

23.5.4 資料工程與 MLOps 的組織

技術之外，資料工程與 MLOps 更是組織的課題。常見的分工包括：

- 資料工程師：建立與維護資料管線、資料平台與資料品質監控。
- 資料科學家：與領域專家合作，定義問題、建立與評估模型。
- 機器學習工程師：把模型部署到生產環境，建立監控與自動化流程。
- 領域專家與製程工程師：提供製程知識、定義標籤、判讀結果，並對決策負責。
- IT 與 OT 人員：負責網路、系統與設備端的整合與資安（第 35 章）。

在中小企業，這些角色常常由少數人身兼數職。重要的不是職稱，而是每一個環節都有人負責：資料壞了誰修？模型失準誰知道？升版誰核准？

案例

案例 23-1 教學情境：八成的時間在整理資料

背景：叫獸開講中的碩士生（綜合情境），第一個預測專案有八成以上的時間花在找資料與整理資料：感測器卡值、單位改變、時鐘不同步、點位名稱看不懂、不良品紀錄對不上。

做法：專案後，工廠與學校合作建立資料品質的自動檢查、時間同步、點位命名規則與資料契約，並把不良品紀錄改為與工單和模具編號連結。

成果：下一個專案的資料準備時間大幅縮短，模型的上線評估也更可信。

啟示：資料工程是工業 AI 的地基；投資在資料品質與情境化，比投資在更複雜的模型更有效。

討論：（1）你的資料中最常見的品質問題是什麼？（2）資料品質的檢查應該在哪一層進行？

案例 23-2 產業：歷史資料庫的壓縮設定

背景：一家化工廠想以歷史資料分析某反應器的短暫壓力尖峰，卻發現資料庫中幾乎看不到尖峰。

做法：檢查後發現，該點位的壓縮容差設得過大，短暫尖峰被當成雜訊捨棄；團隊依感測器的精度與分析需求重新設定各點位的容差，並為需要高頻分析的點位另設原始資料的保存。

成果：之後的尖峰都被完整記錄，團隊找出尖峰與某個閥門動作的關聯。

啟示：壓縮是取捨；容差應依分析需求逐點設定，高頻分析不能依賴為趨勢監控而壓縮的資料。

討論：（1）如何決定一個點位的壓縮容差？（2）原始資料應該保存多久？

案例 23-3 產業：包絡分析找出軸承早期缺陷

背景：一家造紙廠的關鍵風機，定期以振動均方根值監控，數值一直在警戒值以下，卻在一次停機檢修時發現軸承外環已有剝落。

做法：導入線上振動監測，依軸承型號與轉速計算特徵頻率，並以包絡頻譜定期分析；設定特徵頻率振幅的趨勢警報。

成果：之後在多台設備上，於均方根值仍正常時就發現軸承的早期缺陷，得以在計畫停機時更換。

啟示：結合物理知識的特徵，能比通用的統計量更早發現故障；這是預測維護的基本功。

討論：（1）變轉速的設備如何做包絡分析？（2）特徵頻率的振幅門檻如何設定？

案例 23-4 產業：多變量監控抓出感測器漂移

背景：一家特用化學品廠的產品濃度偶爾偏離規格，但每一個製程變數的單變數管制圖都正常。

做法：以正常操作的資料建立主成分模型，即時監控 T^2 與 SPE；SPE 超限時以貢獻圖指出最可

疑的變數，再由製程工程師判讀。

成果：系統多次在產品偏離規格之前，發現溫度感測器的漂移與冷卻水閥門的異常，讓工程師提早校正與維修。

啟示：多變量監控能看見「關係」的異常；貢獻圖提供根因分析的起點，但最終判斷需要領域專家。

討論：（1）製程改變後，主成分模型應如何更新？（2）如何避免把「新的正常」誤判為異常？

案例 23-5 台灣情境：從點位到語意

背景：台灣許多工廠歷經多年擴建，不同年代、不同廠商的設備各有自己的點位命名，資料分析人員常需要花大量時間對照點位與設備。

做法：常見的做法是依 ISA-95 的設備階層建立統一的命名規則，搭配資產管理殼或統一命名空間描述設備與點位，並以知識圖譜連結設備、產線、工單與產品；新設備的採購規格中要求符合命名與資料契約。

成果：分析人員可以依設備與產線查詢資料，跨廠區的比較與 AI 應用也更容易複製。

啟示：語意層的建立耗時，卻是讓資料能被規模化使用的關鍵；從採購規格就開始要求，比事後整理容易得多。

討論：（1）既有設備的點位如何有效率地整理？（2）命名規則應由誰制定與維護？

案例 23-6 教學情境：模型換了模具就失準

背景：一個學生專題團隊為實驗室的小型射出機建立重量預測模型，前兩週表現很好，換了一副模具後誤差突然變大，但沒有人發現，直到兩週後才有人注意。

做法：團隊加入誤差的漂移檢定與每日報表，並讓模具更換的紀錄自動通知模型；新模具上線時先以影子模式訓練新的模型，確認後再切換。

成果：之後每次換模具，系統都能在數十件之內偵測到漂移，並在一兩天內完成新模型的升版。

啟示：模型上線後需要持續監控；把製程事件與模型平台連起來，能讓維運從被動變主動。

討論：(1) 除了換模具，還有哪些事件應該通知模型？(2) 沒有真實標籤時，如何監控模型？

實作活動

活動 23-1 資料品質檢查器

以 23.1.3 節的程式為基礎，加入「變化率過大」與「與相鄰感測器不一致」兩種檢查，並把清理結果以標記保留，而不是直接刪除；以實驗室的一組感測資料測試。

活動 23-2 壓縮與分析

以 23.2.2 節的程式為基礎，在資料中加入幾個短暫的尖峰，觀察不同容差下尖峰是否被保留；並實作死區壓縮法，比較兩種方法的壓縮比與重建誤差。

活動 23-3 軸承診斷

以 23.3.2 節的程式為基礎，加入內環缺陷（其衝擊振幅隨轉速週期調變），計算內環缺陷頻率，並以包絡頻譜辨識；再改變雜訊大小，觀察包絡分析的極限。

活動 23-4 多變量監控

以 23.3.3 節的程式為基礎，模擬另一種故障：進料流量突然增加 3 個標準差，觀察 T^2 與 SPE 的反應，並說明兩種統計量各自擅長偵測哪種異常。

活動 23-5 資料契約

為你的專題或實驗室設備撰寫一份資料契約，包含欄位、單位、範圍、頻率與負責人，並以 23.4.2 節的程式為基礎實作檢查，把違約事件寫入日誌。

活動 23-6 MLOps 流程

以 23.5.3 節的程式為基礎，加入「警報後立即提高實際量測頻率」的保護機制，計算從漂移發生到挑戰者升版之間的總誤差成本，並比較不同 Page-Hinkley 門檻的影響。相關範例程式將於本書的

實驗倉庫中維護。

延伸閱讀

- Matei Zaharia、Ali Ghodsi、Reynold Xin 與 Michael Armbrust，〈Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics〉，CIDR 2021。
- Robert B. Randall，Vibration-based Condition Monitoring (Wiley, 2011)：振動診斷與包絡分析的經典教材。
- Douglas C. Montgomery，Introduction to Statistical Quality Control：包含多變量製程監控（第 25 章亦將引用）。
- Martin Kleppmann，Designing Data-Intensive Applications (O'Reilly, 2017)：資料系統與串流處理的基礎。
- Chip Huyen，Designing Machine Learning Systems (O'Reilly, 2022)：機器學習系統的部署、監控與維運。

表 23-7 本章思考地圖

本章概念	關鍵提問	後續延伸章節
資料類型與情境化	資料知道自己來自哪台設備、哪張工單嗎？	第 17、20 章
資料品質	有卡值、單位改變、時間不同步嗎？誰負責修？	第 25 章
Historian 與湖倉	壓縮設定合適嗎？原始資料保存了嗎？	第 26、33 章
頻譜與異常偵測	有沒有物理特徵？是單變數還是關係的異常？	第 25、26 章
根因分析	最異常的變數是原因還是結果？	第 25、28 章
資料治理與語意層	資料有契約、定義與負責人嗎？	第 28、35 章
MLOps 與漂移	上線後如何偵測漂移、升版與回復？	第 20、36 章

章末小結

- 製造資料包括時序、事件、交易與主資料、品質、影像與文件。情境化為資料加上資產、生產與品質的背景；資料品質問題包括缺值、卡值、超出範圍、重複亂序、單位改變與時鐘不同步。範例中 603 筆資料清理後剩 563 筆，並依時間對齊三張工單；卡值最危險，因為看起來完全正常。
- 製造資料平台由邊緣、歷史資料庫、串流平台、資料湖與資料倉儲、資料湖倉分層組成。旋轉門壓縮只保留轉折點，範例中以容差 0.2 保留 15 點、壓縮 240 倍；固定取樣則在階躍處產生較大誤差。高頻分析不能依賴為趨勢而壓縮的資料。
- 時序特徵包括時域、形狀、趨勢、頻域、時頻與事件特徵。包絡分析在原始頻譜看不到的情況下找出軸承外環缺陷頻率（理論 86.7 Hz、量測 87.0 Hz）。主成分分析的 SPE 偵測到單變數管制幾乎看不出的感測器漂移（超限約 65%，單變數約 6%），貢獻圖指向夾套溫度（約 74%）；根因分析需要時間先後、分組比較、因果結構與領域專家。
- 資料治理涵蓋角色、定義、品質、契約、血緣、存取與保存。資料契約把問題擋在源頭，範例中攔下了範圍檢查抓不到的單位改變；語意層以 ISA-95 階層、屬性、關係與指標讓資料可理解、可查詢，是工業知識圖譜與企業 AI 的基礎。
- MLOps 涵蓋版本、特徵、自動化訓練、模型登錄、部署策略、監控與重訓。範例中 Page-Hinkley 檢定在漂移後 16 筆發出警報，挑戰者在影子期間以約 0.87 對 2.62 的誤差勝出並升版。製造的特殊考量包括邊緣部署、延遲的標籤、安全與人工覆核，以及把製程事件與模型平台連結。

習題

一、選擇題

1. 為資料加上設備、工單、產品與批號等背景資訊，稱為？（A）壓縮 （B）情境化 （C）正規化 （D）加密
2. 感測器故障使數值長時間完全不變，稱為？（A）缺值 （B）卡值 （C）亂序 （D）重複

3. 專門儲存與查詢大量工業時序資料的系統是？(A) ERP (B) 歷史資料庫 (C) CRM (D) PLM
4. 只保留必要轉折點、誤差在容差內的時序壓縮法是？(A) 旋轉門壓縮 (B) JPEG (C) 霍夫曼編碼 (D) 主成分分析
5. 結合資料湖的彈性與資料倉儲管理能力的架構稱為？(A) 資料湖倉 (B) 資料集市 (C) 資料孤島 (D) 資料契約
6. 找出軸承衝擊重複頻率的振動分析方法是？(A) 包絡分析 (B) 蒙地卡羅 (C) 線性規劃 (D) 有限元素
7. 偵測變數之間相關結構被破壞的多變量統計量是？(A) 平均值 (B) SPE (C) 中位數 (D) 全距
8. 生產者與使用者對資料欄位、單位、範圍與頻率的約定稱為？(A) 資料契約 (B) 服務契約 (C) 主資料 (D) 配方
9. 新模型與線上模型並行預測、但不影響實際決策的部署方式是？(A) 影子部署 (B) 藍綠部署 (C) 直接替換 (D) 離線部署
10. 以累積偏離量偵測平均值上升、源自 Page 於 1954 年提出之累積和概念的序列檢定是？(A) t 檢定 (B) Page-Hinkley 檢定 (C) 卡方檢定 (D) F 檢定

二、簡答題

11. 說明情境化的三個層次，並各舉一例。
12. 為什麼卡值是最危險的資料品質問題？如何偵測？
13. 說明旋轉門壓縮的原理，以及容差設定的取舍。
14. 說明串流處理中事件時間與處理時間的差異，以及晚到資料的處理。
15. 為什麼軸承缺陷在原始頻譜中不易發現？包絡分析如何解決？
16. 比較 T^2 與 SPE 各自擅長偵測的異常。
17. 為什麼貢獻圖中最異常的變數不一定是根因？

18. 說明影子部署與冠軍挑戰者的流程與目的。

三、計算題

19. 軸承有 9 顆滾珠，滾珠直徑 7.9 mm，節圓直徑 34.5 mm，接觸角 0，轉速 1,500 rpm，求 BPFO。

20. 3,600 點的資料經旋轉門壓縮後保留 15 點，壓縮比為多少？

21. 某工廠有 20,000 個點位，每秒記錄一次，每筆 8 位元組，一年的原始資料量約為多少？若平均壓縮 50 倍呢？

22. 以 99% 管制界限監控 1,000 筆正常資料，預期會有幾筆誤報？

23. 以 1 秒的資料做 FFT，頻率解析度為多少？若改用 0.5 秒呢？

24. 取樣頻率 20 kHz，可分析的最高頻率為多少？

25. 資料契約的新鮮度要求為 5 秒，訊息的時間戳記為 80 秒、到達時為 101 秒，是否違約？延遲多少？

26. 影子期間冠軍的平均絕對誤差 2.62、挑戰者 0.87，升版規則為「挑戰者小於冠軍的 80%」，是否升版？

四、申論與設計題

27. 為一座有 50 台設備的工廠設計資料平台：說明邊緣、歷史資料庫、串流與湖倉的分工，以及資料品質與情境化的做法。

28. 評論以下說法：「只要把所有資料都存進資料湖，之後要分析什麼都可以。」

29. 為一個上線的品質預測模型設計 MLOps 流程，包括監控指標、漂移偵測、重訓、部署策略、核准與回復。

五、是非題

30. 卡值在數值上通常看起來正常，因此需要特別的檢查。

31. 為趨勢監控而壓縮的歷史資料，適合用來做高頻振動分析。

- 32. 單變數管制圖都正常，就表示製程沒有異常。
- 33. 資料契約只檢查數值範圍就足夠了。
- 34. 影子部署期間，新模型的預測不影響實際的生產決策。
- 35. 偵測漂移的門檻越低越好，因為可以越早發現問題。

六、進階挑戰題

- 36. 以 Kafka 或 MQTT 搭配 Python，建立一個串流管線：即時接收模擬的感測資料，進行資料契約檢查、每分鐘的視窗統計與異常警報，並處理晚到的資料。
- 37. 以公開的軸承振動資料集，實作包絡分析與第 22 章的一維 CNN 兩種診斷方法，以「依軸承分組」的方式評估，比較兩者的準確率與可解釋性。
- 38. 以 MLflow 等開源工具建立模型登錄表，把 23.5.3 節的冠軍挑戰者流程自動化，包括漂移警報、重訓、影子評估、核准與回復。

七、補充題

- 39. 列出表 23-5 中資料治理的七個要素。
- 40. 列出本章介紹的四種模型部署策略。

提示與參考解答

- 1. B。
- 2. B。
- 3. B。
- 4. A。
- 5. A。
- 6. A。
- 7. B。
- 8. A。

9. A。

10. B。

11. 參考：資產情境（資料來自哪台設備、什麼感測器與單位）、生產情境（當時的工單、產品、批號）、品質情境（這段時間產品的品質結果）。

12. 參考：卡值在數值上合理，不會觸發範圍檢查，卻讓分析誤以為製程穩定。可檢查數值長時間完全不變，或與相鄰感測器、預期變化的不一致。

13. 參考：從最後保留的點出發維護上下兩扇門，涵蓋所有新點；門交錯時保留前一點重新開始。容差大則壓縮比高但遺失細節，容差小則保留細節但壓縮有限。

14. 參考：事件時間是資料發生的時間，處理時間是被處理的時間。晚到資料應依事件時間放回正確的視窗，並設定可接受的延遲上限。

15. 參考：早期衝擊能量小，激發的是高頻共振，在原始頻譜中被轉速頻率與雜訊掩蓋。包絡分析先帶通濾出共振頻帶、取包絡，再對包絡做頻譜，使衝擊的重複頻率顯現。

16. 參考：T² 偵測在主成分空間中遠離正常中心的異常，例如負荷或操作點異常；SPE 偵測變數間相關結構被破壞，例如感測器漂移或新的故障模式。

17. 參考：異常會沿製程傳播，受影響的結果變數也可能偏離很大；應結合時間先後、因果結構與領域知識判斷。

18. 參考：影子部署讓新模型與線上模型並行預測但不影響決策；冠軍挑戰者要求新模型在相同資料上明顯勝過線上模型才升版。目的是在不冒風險的情況下以真實資料驗證新模型。

19. $9 \div 2 \times 25 \times (1 - 7.9 \div 34.5) \approx 86.7 \text{ Hz}$ 。

20. $3,600 \div 15 = 240$ 倍。

21. $20,000 \times 86,400 \times 365 \times 8 \approx 5.0 \times 10^{12}$ 位元組，約 5 TB；壓縮 50 倍約 100 GB。

22. $1,000 \times 1\% = 10$ 筆。

23. 1 Hz；2 Hz。

24. 奈奎斯特頻率 10 kHz（實務上會再留一些餘裕）。

25. 延遲 $101 - 80 = 21$ 秒，超過 5 秒，違約。
26. $0.8 \times 2.62 = 2.10$ ，0.87 小於 2.10，升版。
27. 評分重點：分層分工合理；邊緣緩衝與時間同步；壓縮依需求設定並保存原始高頻資料；資料品質自動檢查與標記；以 ISA-95 階層與 MES 事件情境化。
28. 評分重點：能指出沒有情境、定義、品質與治理的資料湖會變成「資料沼澤」；說明情境化、語意層與資料契約的必要。
29. 評分重點：監控資料品質、漂移、預測分布與實際誤差；有明確的漂移檢定與重訓觸發條件；採影子或金絲雀部署；有核准、登錄、回復與負責人。
30. ✓。
31. X。壓縮可能已捨棄高頻細節，應另行保存原始資料。
32. X。變數間的關係可能異常，需要多變量監控。
33. X。還需檢查欄位、型別、單位、命名與新鮮度等。
34. ✓。
35. X。門檻過低會增加誤報，需在偵測速度與誤報之間取捨。
36. 評分重點：管線可運作，契約檢查與視窗統計正確，能依事件時間處理晚到資料，警報合理。
37. 評分重點：特徵頻率計算正確，分組評估合理，能比較兩種方法的準確率與可解釋性。
38. 評分重點：模型登錄與流程自動化完整，包含血緣、評估、核准與回復。
39. 角色與責任、定義與目錄、品質、資料契約、血緣與版本、存取與資安、保存與刪除。
40. 影子部署、冠軍挑戰者、金絲雀部署、可回復。

第 24 章

機器視覺與 AI 品質檢測

編著 李世淵 (機器人叫獸) | 助教 李天宇、李宇晴

本章重點

- 光學先於演算法：照明、相機、鏡頭與像素解析度的設計
- CNN、Vision Transformer 與 YOLO；IoU 與 NMS 後處理
- 分類、偵測、分割與只用良品的異常檢測
- 資料蒐集、標註標準、資料增強與合成影像
- 精確率、召回率、mAP，以及以漏檢與誤判成本定門檻

叫獸開講：AI 看得見，是因為光打對了

有一家做金屬外殼的工廠，花了不少錢導入「AI 瑕疵檢測」。供應商在展示時，模型對樣品照片的判斷幾乎百分之百正確。但上線之後，誤判一大堆：好好的外殼被判成刮傷，真正有刮傷的反而漏掉。工廠找我去看。

我先沒看模型，而是去看相機。問題很快就清楚了：相機裝在產線旁邊，照明是廠房的日光燈，加上窗戶透進來的陽光；早上、中午、下午，外殼表面的反光完全不同；外殼在傳送帶上的位置也不固定，有時偏左、有時偏右。而供應商展示用的樣品照片，是在暗房裡、用固定的燈光、把外殼擺正拍的。模型學到的是「暗房裡的外殼」，不是「產線上的外殼」。

我們做了三件事。第一，加裝遮光罩與低角度的環形光源，讓刮痕在影像中變成明顯的亮線，而且不受廠房燈光影響；第二，加一個定位治具，讓外殼每次都在相同的位置；第三，在產線上重新蒐集資料，並在訓練時加入亮度與位置的隨機變化。三週之後，誤判率降到原本的十分之一以下。

工廠的主管問我：「所以 AI 不重要嗎？」我說：「AI 很重要，但它只能看見相機拍到的東西。光打對了、相機擺對了，AI 才有機會看見瑕疵；資料對了，AI 才學得到真正的瑕疵。」

這一章，我們從工業影像的光學、照明與相機配置談起；介紹 CNN、Vision Transformer 與 YOLO 等視覺模型；說明分類、偵測、分割與異常檢測四種任務；討論資料蒐集、標註、增強與合成影像；最後以精確率、召回率、mAP 與漏檢、誤判的成本，評估視覺檢測系統是否真的帶來價值。

（說明：本節情境由作者協助企業導入視覺檢測的經驗綜合改寫，人物與細節已調整。）

學習目標

讀完本章後，你應該能夠：

1. 依缺陷大小與視野計算相機解析度、放大率與景深，並依瑕疵特性選擇照明方式。
2. 以傳統影像處理完成背景校正與分割，理解照明不均對演算法的影響。
3. 說明 CNN、Vision Transformer 與 YOLO 的原理與差異，並實作 IoU 與非最大值抑制。
4. 區分分類、偵測、分割與異常檢測，並以只用良品訓練的方法建立異常檢測與熱圖。

5. 說明資料蒐集、標註、增強與合成影像的實務，並以程式示範資料增強對穩健性的影響。
6. 以精確率、召回率、AP 與 mAP 評估視覺模型，並以漏檢與誤判成本設計包含人工複檢的檢測流程。

核心概念

表 24-1 本章核心名詞中英對照

中文名詞	英文	一句話說明
視野	Field of View, FOV	相機一次能拍到的物方範圍
解析度	Resolution	每個像素對應的物方尺寸，決定可見的最小缺陷
景深	Depth of Field	物體前後仍能清楚成像的距離範圍
遠心鏡頭	Telecentric Lens	放大率不隨距離改變、適合精密量測的鏡頭
暗場照明	Dark-field Illumination	低角度光源，使刮痕與凸起在暗背景中發亮
背光照明	Backlight	光源在物體後方，取得清晰的輪廓
平場校正	Flat-field Correction	修正照明與感測器不均勻的處理
Vision Transformer	Vision Transformer, ViT	把影像切成區塊、以注意力機制處理的視覺模型
YOLO	You Only Look Once	一次前向運算就同時輸出位置與類別的即時偵測模型
IoU	Intersection over Union	預測框與真實框的交集除以聯集
非最大值抑制	Non-Maximum Suppression, NMS	去除重疊的重複偵測框
語意分割	Semantic Segmentation	逐像素判斷類別，得到缺陷的精確形狀
異常檢測	Anomaly Detection	只學習良品的樣貌，把不像良品的判為異常
平均精確度	Average Precision, AP	精確率—召回率曲線下的面積
漏檢與誤判	Escape / Overkill	不良品被判為良品 / 良品被判為不良品

24.1 工業影像、光學、照明與相機配置

24.1.1 機器視覺系統的組成

一套機器視覺檢測系統，通常包含：

- 照明：讓瑕疵在影像中變得明顯，是成敗的第一關。
- 鏡頭：決定視野、放大率、景深與畸變。
- 相機：面掃描或線掃描、黑白或彩色、解析度與幀率。
- 觸發與同步：以光電開關或編碼器觸發拍攝，與輸送速度同步（第 10 章）。
- 運算與軟體：傳統影像處理或深度學習模型，在工業電腦或邊緣裝置上執行（第 11 章）。
- 與產線整合：判定結果送給 PLC 剔除不良品，並寫入 MES 與製造履歷（第 17 章）。

許多視覺專案的失敗，不是因為演算法不好，而是因為影像本身就拍不出瑕疵。業界常說「好的照明勝過聰明的演算法」：在影像中對比清楚的瑕疵，用簡單的方法就能檢出；對比不清楚的瑕疵，再強的模型也只能猜。

24.1.2 照明

表 24-2 常用的照明方式與適用情境

照明方式	原理	適合檢測	例子
明場（同軸、正面）	光線從正面照射，反射進入鏡頭	平坦表面上的污點、印刷	印刷品、晶圓表面
暗場（低角度）	光線以低角度掠射，只有凹凸處散射進鏡頭	刮痕、凹痕、凸起、毛邊	金屬外殼、玻璃
背光	光源在物體後方，取得剪影	外形尺寸、孔位、透明物的異物	沖壓件輪廓、瓶裝液體
圓頂（漫射）	從各方向均勻照射，消除反光	曲面、高反光物體	焊點、金屬罐
結構光	投射條紋或雷射線	三維形狀、高度、平整度	焊道、翹曲
特殊波段	紅外線、紫外線、X 光	內部缺陷、螢光標記	電池、鑄件、PCB 焊點

選擇照明的關鍵是：瑕疵與背景在光學上有什麼差異？刮痕會改變表面的法線方向，所以低角度的暗場照明能讓它「亮起來」；異物的顏色與背景不同，就以適當的波長加強對比；內部的氣孔，則需要 X 光或電腦斷層（第 16 章）。

24.1.3 解析度、放大率與景深

相機的解析度要夠看見最小的缺陷。常用的經驗法則是：最小缺陷至少要跨越 3 個像素，才能被穩定地偵測。若視野寬度為 FOV，缺陷大小為 d，長邊所需的像素數約為：

$$\text{所需像素數} \approx \text{FOV} \div d \times 3$$

光學放大率為感測器尺寸與視野的比值；景深則與光圈、容許模糊圈與放大率有關，放大率越大、光圈越大（f 值越小），景深越淺。下面的程式以一台約 500 萬像素（2,448 × 2,048、像素 3.45 μm）的工業相機為例，評估幾種情境：

```
import math
# 相機選型的基本計算：視野、解析度、最小可檢缺陷、工作距離與景深
def design(fov_mm, defect_mm, px_per_defect=3, pixel_um=3.45, sensor_px=(2448, 2048),
N=8, coc_px=2):
    need = math.ceil(fov_mm / defect_mm * px_per_defect)           # 長邊所需像素數
    res = fov_mm / sensor_px[0]                                     # 每像素代表的物方尺
    寸 (mm/px)
    mag = sensor_px[0] * pixel_um / 1000 / fov_mm                 # 光學放大倍率 = 感測
    器寬 / 視野
    c = coc_px * pixel_um / 1000                                   # 容許模糊圈 (mm, 感
    測器端)
    dof = 2 * N * c * (1 + mag) / mag**2                           # 近似景深 (mm)，適
    用於近距離
    return need, res, mag, dof
for fov, defect in [(100, 0.2), (100, 0.1), (50, 0.05), (300, 0.5)]:
    need, res, mag, dof = design(fov, defect)
    ok = "足夠" if need <= 2448 else "不足"
    print(f"視野 {fov} mm、缺陷 {defect} mm：需 {need} 像素（{ok}），解析度 {res*1000:.0f}
    μm/px，放大率 {mag:.3f}，f/8 景深約 {dof:.1f} mm")
```

執行結果：

表 24-3 相機配置的評估（2,448 × 2,048 像素、f/8）

視野	最小缺陷	所需像素	是否足夠	解析度	放大率	景深
100 mm	0.2 mm	1,500	足夠	41 μm / px	0.084	約 16.8 mm
100 mm	0.1 mm	3,000	不足	41 μm / px	0.084	約 16.8 mm

視野	最小缺陷	所需像素	是否足夠	解析度	放大率	景深
50 mm	0.05 mm	3,000	不足	20 μm / px	0.169	約 4.5 mm
300 mm	0.5 mm	1,800	足夠	123 μm / px	0.028	約 143.2 mm

同一台相機，在 100 mm 的視野下可以看見 0.2 mm 的缺陷，但看不見 0.1 mm 的缺陷；要檢出 0.1 mm，可以換更高解析度的相機、縮小視野分成多張拍攝，或改用線掃描相機。第三列的情境也顯示了放大率與景深的取舍：視野縮小到 50 mm，景深只剩約 4.5 mm，若工件的高度變化超過這個範圍，影像就會模糊。這時可以縮小光圈（加大 f 值，但需要更強的照明），或採用遠心鏡頭與固定的治具。

24.1.4 傳統影像處理：先把影像整理乾淨

在深度學習普及之前，機器視覺主要依靠傳統影像處理：濾波、門檻分割、形態學運算、邊緣偵測與模板比對。這些方法至今仍然重要：它們快速、可解釋、不需要大量資料，在照明與位置受控的場合非常可靠；即使使用深度學習，也常以它們做前處理與後處理。

下面的程式模擬叫獸開講中的問題：一片金屬表面有一條細刮痕，反射率比周圍低 25%，但照明不均，左上亮、右下暗。程式比較兩種做法：直接以 Otsu 法自動選一個全域門檻；先以形態學的閉運算估計背景（閉運算會「填平」細小的暗紋，留下緩慢變化的照明），把影像除以背景做平場校正，再以固定門檻分割：

```
import numpy as np
from scipy import ndimage as ndi
# 傳統影像處理：照明不均時，全域門檻會失敗；先以形態學估計背景再相除（平場校正），刮痕就能被正確分割
rng = np.random.default_rng(0)
H = W = 240; yy, xx = np.mgrid[0:H, 0:W]
light = 0.55 + 0.45 * np.exp(-((xx - 60)**2 + (yy - 70)**2) / (2 * 110**2)) # 照明不均（左上亮、右下暗）
surface = 0.8 + 0.03 * rng.normal(0, 1, (H, W))
truth = np.zeros((H, W), bool)
for t in np.linspace(0, 1, 400): # 一條細刮痕（寬約 3 像素，反射率降 25%）
    r, c = int(40 + 160 * t), int(30 + 180 * t + 8 * np.sin(6 * t)); truth[r-1:r+2, c-1:c+2] = True
img = light * surface * np.where(truth, 0.75, 1.0)
def otsu(x):
    h, e = np.histogram(x, 256); c = (e[:-1] + e[1:]) / 2; w = np.cumsum(h); m = np.cumsum(h * c)
    between = (m[-1] * w / w[-1] - m)**2 / (w * (w[-1] - w) + 1e-9); return
```

```

c[np.argmax(between)]
def score(pred):
    tp = (pred & truth).sum(); return tp / max(pred.sum(), 1), tp / truth.sum(), 2 * tp /
(pred.sum() + truth.sum())
g = img < otsu(img)
print("全域 Otsu 門檻：精確率 %.2f，召回率 %.2f，Dice %.2f" % score(g))
bg = ndi.grey_closing(img, size=(15, 15)) # 閉運算去除細暗紋，估計背景
flat = img / bg # 平場校正
f = ndi.binary_opening(flat < 0.88, structure=np.ones((2, 2))) # 固定門檻後，以開運算去除零
星雜點
print("背景估計+相除+固定門檻：精確率 %.2f，召回率 %.2f，Dice %.2f" % score(f))

```

執行結果：全域 Otsu 門檻的召回率雖然是 1.00，精確率卻只有 0.03，Dice 係數約 0.06；它把整片照明較暗的右下角都當成了缺陷。背景估計並平場校正之後，精確率約 0.94、召回率 1.00、Dice 約 0.97，刮痕被乾淨地分割出來（最大連通區域約 965 像素，真實刮痕 941 像素）。

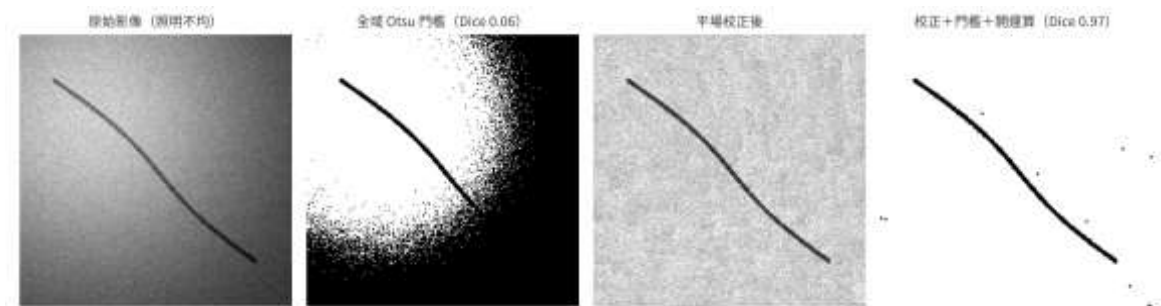


圖 24-1 照明不均時的全域門檻與平場校正

這個例子說明了兩件事：一是照明不均是視覺檢測最常見的敵人，最好的解法是從硬體上改善照明；二是即使硬體無法完美，理解影像的物理（反射率與照明相乘），也能以簡單的處理大幅改善結果。深度學習模型同樣會受照明影響，24.4 節將以資料增強處理這個問題。

24.2 CNN、Vision Transformer 與 YOLO

24.2.1 CNN 與遷移學習

第 22 章以一維 CNN 辨識振動訊號，影像使用的是二維卷積：卷積核在影像上滑動，淺層學到邊緣與紋理，深層學到形狀與部件。2012 年 AlexNet 在大規模影像辨識競賽中大幅勝出之後，CNN 成為電腦視覺的主流；之後的 VGG、ResNet 等架構，以更深的網路與殘差連接持續提升準確率。

製造業很少從零開始訓練大型 CNN，因為資料不夠。常用的做法是遷移學習：先使用在大型公開影像資料集上預訓練的模型，保留它學到的通用特徵，再以少量的產線影像微調最後幾層。這讓數

百張影像就能訓練出可用的模型。

24.2.2 Vision Transformer

2020 年，Dosovitskiy 等人提出 Vision Transformer (ViT，論文於 2021 年在 ICLR 正式發表)：把影像切成固定大小的區塊(例如 16×16 像素)，每個區塊當成一個「字」，再以第 22 章介紹的 Transformer 與注意力機制處理。ViT 能捕捉影像中距離很遠的區塊之間的關係，在大量資料預訓練後表現優異；但在資料較少時，CNN 的「局部性」先驗通常較有優勢。近年的許多視覺基礎模型，以大量無標籤影像自監督預訓練，再遷移到各種下游任務，也開始被應用於工業檢測。

24.2.3 YOLO 與即時物件偵測

物件偵測要同時回答「是什麼」與「在哪裡」。早期的方法先產生大量候選區域、再逐一分類，速度較慢。Redmon 等人提出 YOLO (2015 年公開預印本，2016 年發表於 CVPR) (You Only Look Once)，把偵測視為一次的迴歸問題：網路一次前向運算，就在整張影像的網格上同時輸出方框的位置、大小、信心分數與類別，速度足以即時處理影片。YOLO 系列之後由多個團隊持續改進，例如 Ultralytics 等公司陸續推出新版本，在產線的即時偵測中被廣泛使用。

表 24-4 常見視覺模型的比較

模型類型	代表	輸出	優點	限制
CNN 分類	ResNet 等	影像的類別	成熟、遷移學習容易	不知道缺陷在哪裡
Vision Transformer	ViT 等	影像的類別或特徵	全域關係、適合大規模預訓練	小資料時較吃力、運算量大
單階段偵測	YOLO 系列等	方框、類別、分數	快速、適合即時	極小或細長缺陷較難
雙階段偵測與實例分割	Faster R-CNN、Mask R-CNN 等	方框與遮罩	精確	較慢
語意分割	U-Net 等	逐像素類別	缺陷形狀與面積	標註成本高
異常檢測	PaDiM、PatchCore 等	異常分數與熱圖	只需良品資料	不知道缺陷類別

24.2.4 偵測的後處理：IoU 與 NMS

偵測模型通常會對同一個物體輸出好幾個重疊的方框。後處理以非最大值抑制 (NMS) 去除重複：依信心分數排序，保留分數最高的方框，刪除與它重疊程度 (IoU) 超過門檻的其他方框，再對剩下的方框重複這個步驟。IoU 是兩個方框交集面積除以聯集面積，0 表示完全不重疊，1 表示完全重合；它也是評估偵測是否「框對了」的標準。

24.5 節的程式會實作 IoU、NMS 與平均精確度，並以模擬的偵測結果說明 NMS 對評估的影響。

24.3 分類、偵測、分割與異常檢測

24.3.1 四種任務

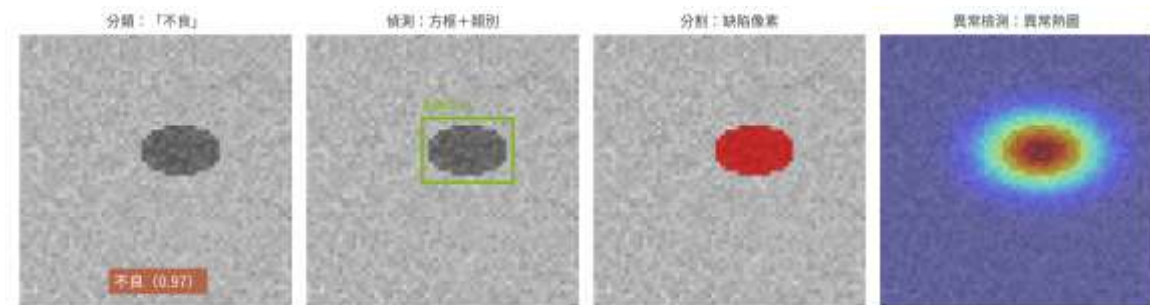


圖 24-2 分類、偵測、分割與異常檢測的輸出

- 分類：判斷整張影像是良品還是不良品，或屬於哪一種缺陷。最簡單，但不提供位置。
- 偵測：以方框標出每個缺陷的位置與類別。適合計數、定位與後續的重工。
- 分割：逐像素標出缺陷的範圍。適合需要量測缺陷面積、長度或形狀的情況，例如塗層剝落的面積比例。
- 異常檢測：只用良品學習「正常的樣子」，任何不像良品的區域都被標為異常。適合缺陷種類多、樣本少、甚至未知的情况。

選擇哪一種任務，取決於檢測的目的與可用的資料：要剔除不良品，分類可能就夠了；要讓人員知道去哪裡修，就需要偵測或分割；要在新產品導入、還沒有足夠缺陷樣本時就開始檢測，異常檢測是最實際的起點。

24.3.2 異常檢測：只用良品學習

製造中最常見的困境是：良品很多，不良品很少，而且下一個出現的缺陷可能是從來沒見過的。

2019 年 Bergmann 等人在 CVPR 發表的 MVTEC AD 資料集，專門為工業異常檢測設計，訓練集只有良品，測試集包含各種真實的缺陷，成為這個領域最常用的基準。之後，PaDiM (Defard 等，2020 年發表於 arXiv、ICPR 2021) 與 PatchCore (Roth 等，CVPR 2022) 等方法相繼提出。它們的共同想法是：以預訓練網路擷取影像各個局部區塊的特徵，建立良品特徵的統計模型；測試時，計算每個區塊與良品的差距，得到異常熱圖。

下面的程式以簡化的方式實作這個想法：影像是 64×64 的規則織紋，切成 8×8 的區塊，每個區塊計算 4 個簡單特徵（平均、標準差、水平與垂直梯度能量），代替預訓練網路的特徵；以 200 張良品影像，為每一個區塊位置建立一個多變量高斯分布；測試時，以馬氏距離作為每個區塊的異常分數，最大值作為影像的異常分數。測試集包含良品、斑點缺陷與線狀缺陷：

```
import numpy as np
from sklearn.metrics import roc_auc_score
# 只用良品學習的異常檢測：以良品影像的局部區塊建立高斯分布（PaDiM 的簡化版），以馬氏距離產生異常熱圖
rng = np.random.default_rng(5)
S, P = 64, 8 # 影像 64x64，區塊 8x8
def texture(defect=None):
    yy, xx = np.mgrid[0:S, 0:S]
    img = 0.5 + 0.15 * np.sin(xx / 2.0 + rng.normal(0, 0.2)) + 0.05 * rng.normal(0, 1, (S, S)) # 規則織紋
    m = np.zeros((S, S), bool)
    if defect == "spot": cy, cx = rng.integers(10, 54, 2); m = (yy - cy)**2 + (xx - cx)**2 < 9; img[m] -= 0.15
    if defect == "line": r = rng.integers(8, 56); m[r:r+1, 20:44] = True; img[m] += 0.20
    return img, m
def patches(img): # 每個區塊取 4 個特徵：平均、標準差、水平與垂直梯度能量
    out = []
    for i in range(0, S, P):
        for j in range(0, S, P):
            p = img[i:i+P, j:j+P]
            out.append([p.mean(), p.std(), np.abs(np.diff(p, axis=1)).mean(), np.abs(np.diff(p, axis=0)).mean()])
    return np.array(out) # (64 區塊, 4)
good = np.array([patches(texture())[0] for _ in range(200)]) # 200 張良品
mu = good.mean(0); cov = np.array([np.cov(good[:, k].T) + 1e-4 * np.eye(4) for k in range(good.shape[1])])
icov = np.linalg.inv(cov)
def anomaly_map(img):
    d = patches(img) - mu; return np.sqrt(np.einsum("ki,kij,kj->k", d, icov, d)).reshape(S // P, S // P)
test = [texture() for _ in range(100)] + [texture("spot") for _ in range(50)] + [texture("line") for _ in range(50)]
y = np.r_[np.zeros(100), np.ones(100)]
```

```

sc = np.array([anomaly_map(im).max() for im, _ in test])
print(f"影像層級 AUROC {roc_auc_score(y, sc):.3f}")
thr = np.percentile(sc[:100], 99)
print(f"以良品 99 百分位為門檻：斑點檢出 {np.mean(sc[100:150] > thr):.0%}，線狀檢出 {np.mean(sc[150:] > thr):.0%}，良品誤判 {np.mean(sc[:100] > thr):.0%}")
hit = [] # 定位：異常最高的區塊是否落在缺陷上
for im, m in test[100:]:
    a = anomaly_map(im); i, j = np.unravel_index(a.argmax(), a.shape)
    hit.append(m[i:P:(i+1)*P, j:P:(j+1)*P].any())
print(f"最高異常區塊與真實缺陷重疊的比例 {np.mean(hit):.0%}")

```

執行結果：模型從未見過任何缺陷，影像層級的 AUROC 仍達約 0.965。若以良品分數的第 99 百分位為門檻（良品誤判約 1%），斑點缺陷的檢出率約 66%，線狀缺陷約 82%。在缺陷影像中，異常分數最高的區塊有約 97% 落在真實的缺陷上，熱圖能正確指出缺陷的位置。

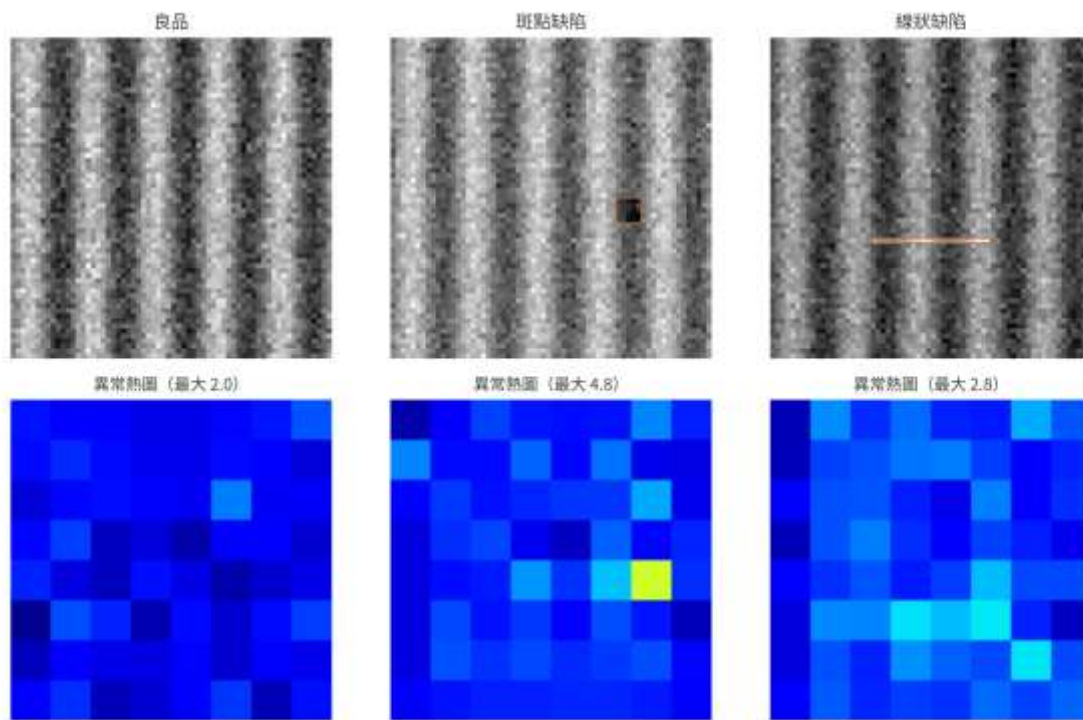


圖 24-3 只用良品訓練的異常檢測：測試影像與異常熱圖

這個結果也顯示了異常檢測的限制：斑點只比背景暗 0.15、半徑只有 3 像素，有三分之一沒有被檢出。要提高檢出率，可以改善照明讓缺陷對比更明顯，或改用預訓練深度網路的特徵（這正是 PaDiM、PatchCore 等方法的做法），或在累積到足夠的缺陷樣本後，改以監督式的偵測或分割模型。實務上常見的路徑是：新產品導入時先以異常檢測上線，同時蒐集被標出的異常影像，由人員確認與標註，逐步建立監督式模型。

24.4 資料蒐集、標註、增強與合成影像

24.4.1 資料蒐集

視覺模型的表現，取決於訓練資料是否代表產線上真正會遇到的情況。資料蒐集時要注意：

- 在產線上、以最終的光學配置拍攝：叫獸開講中的模型在暗房的樣品上學習，到了產線就失準。
- 涵蓋各種變化：不同批次的材料、不同的模具與機台、不同的時段與季節、正常範圍內的位置與角度變化。
- 缺陷的多樣性：每一種缺陷的不同大小、位置與嚴重程度，以及「邊界」的案例（剛好合格與剛好不合格）。
- 依時間與批次保留測試資料：與第 22 章相同，評估資料要模擬上線情況，避免同一批產品同時出現在訓練與測試中。

24.4.2 標註

標註是視覺專案中最耗時、也最容易被低估的工作。分類只需要給每張影像一個標籤；偵測要畫方框；分割要逐像素描出缺陷的輪廓，成本高出許多。更困難的是標註的一致性：什麼程度的刮痕算是缺陷？不同的檢驗員常有不同的判斷，甚至同一個人在不同的時間判斷也不同。

改善標註品質的做法包括：制定圖例清楚的缺陷判定標準（限度樣本）；讓多位人員獨立標註同一批影像，計算一致性並討論分歧；以模型找出「標註與預測差異大」的影像，優先複查。標註標準本身，常常就是品質規格需要釐清的地方。

24.4.3 資料增強

資料增強在訓練時對影像做隨機的變換，讓模型見識更多的變化：翻轉、旋轉、平移、縮放、亮度與對比的改變、模糊與雜訊等。增強的原則是：只使用「真實產線上可能發生、而且不改變標籤」的變換。例如，零件的方向若是固定的，就不應該隨意旋轉；若缺陷的定義與顏色有關，就不應該大幅改變色調。

下面的程式說明增強的效果。一個圓形零件可能有一個缺口；訓練資料都在實驗室以固定的亮度

與位置拍攝，產線上的亮度則在 0.6~1.4 倍之間變化，位置也會偏移最多 3 個像素。程式以邏輯斯迴歸訓練兩個模型：一個只用實驗室的影像，另一個在訓練時加入亮度 (0.5~1.5 倍) 與位置 (最多 4 像素) 的隨機變化：

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from scipy import ndimage as ndi
# 資料增強：訓練影像都在固定的亮度與位置拍攝；產線上的亮度與位置會變動。以增強模擬這些變化，提升穩健性
rng = np.random.default_rng(11)
S = 24
def part(defect, gain=1.0, shift=(0, 0)):
    yy, xx = np.mgrid[0:S, 0:S]
    img = 0.6 * np.exp(-((xx - 12)**2 + (yy - 12)**2) / 60) + 0.05 * rng.normal(0, 1, (S, S))
    # 圓形零件
    if defect: img[(yy - 10)**2 + (xx - 14)**2 < 4] -= 0.25
# 缺口
    return ndi.shift(img * gain, shift, order=1, mode="nearest")
def dataset(n, augment=False, field=False):
    X, y = [], []
    for _ in range(n):
        d = rng.random() < 0.5
        if field: g, s = rng.uniform(0.6, 1.4), rng.uniform(-3, 3, 2) # 產線：亮度與位置都會變
        elif augment: g, s = rng.uniform(0.5, 1.5), rng.uniform(-4, 4, 2) # 增強：訓練時隨機改變
        else: g, s = 1.0, (0, 0) # 實驗室：條件固定
        X.append(part(d, g, s).ravel()); y.append(d)
    return np.array(X), np.array(y)
Xf, yf = dataset(600, field=True)
for name, aug in [("未增強", False), ("亮度+位移增強", True)]:
    X, y = dataset(400, augment=aug)
    m = LogisticRegression(max_iter=3000, C=0.5).fit(X, y)
    X1, y1 = dataset(300)
    print(f"{name}：實驗室條件準確率 {m.score(X1, y1):.1%}，產線條件準確率 {m.score(Xf, yf):.1%}")
```

執行結果：未增強的模型在實驗室條件下準確率 100.0%，到了產線條件只剩約 56.3%，幾乎等於亂猜；加入亮度與位移增強的模型，在實驗室條件下約 93.0%，在產線條件下約 85.0%。增強讓模型在實驗室的表現略為下降，卻大幅提升了在真實條件下的穩健性。

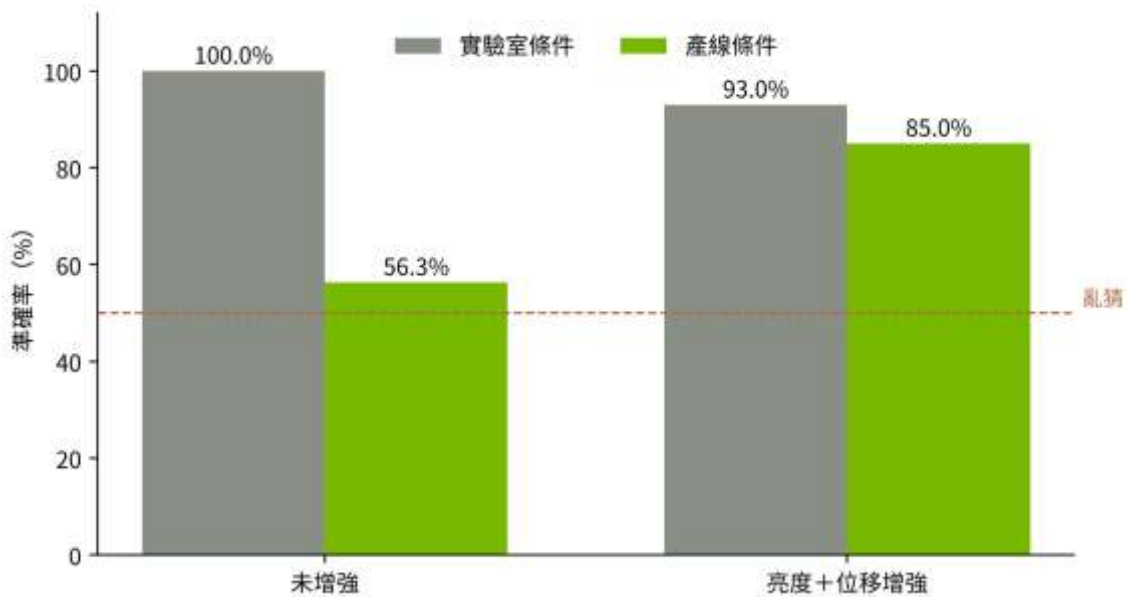


圖 24-4 資料增強：實驗室與產線條件下的準確率

這個簡單的模型直接以像素作為特徵，對位置與亮度特別敏感；CNN 的卷積與池化對平移有一定的容忍度，但同樣會受亮度與照明的影響。不過，增強終究只是模擬，最好的做法仍是從源頭控制變化：穩定的照明、固定的治具，再以增強處理剩下無法消除的變化。

24.4.4 合成影像

第 21 章介紹了合成資料與領域隨機化。在視覺檢測中，合成影像特別有用：以零件的 CAD 模型在渲染軟體中產生影像，隨機改變光源、材質與缺陷的位置、形狀與大小，可以在短時間內產生大量帶有精確標註的缺陷影像。另一種做法是「複製貼上」：把真實缺陷從一張影像中剪下，經過變形與融合，貼到其他良品影像上。近年也有以生成式模型（第 14 章）產生缺陷影像的研究。

合成影像的注意事項與第 21 章相同：要以真實影像確認模型的表現，避免模型學到合成影像的「習慣」；合成資料最適合用來補足罕見的缺陷類型，而不是取代真實資料。

24.5 Precision、Recall、mAP 與漏檢、誤判成本

24.5.1 混淆矩陣與兩種錯誤

檢測系統的錯誤只有兩種：把不良品判為良品，稱為漏檢（escape，偽陰性）；把良品判為不良品，稱為誤判或過殺（overkill，偽陽性）。兩者的代價完全不同：漏檢讓不良品流到客戶手中，可

能造成退貨、索賠、召回甚至安全事故；誤判讓良品被報廢或重工，浪費材料與產能。

精確率 (precision) 是判為不良的件數中真正不良的比例，回答「警報可信嗎」；召回率 (recall) 是所有不良品中被抓到的比例，回答「抓得完嗎」。在不良率很低的產線上，準確率 (accuracy) 幾乎沒有意義：不良率 0.5% 時，一個「全部判良品」的模型準確率高達 99.5%，卻一件不良也抓不到。

表 24-5 檢測評估的常用指標

指標	定義	回答的問題
精確率	$TP \div (TP + FP)$	判為不良的，有多少真的不良？
召回率 (檢出率)	$TP \div (TP + FN)$	不良品有多少被抓到？
漏檢率	$FN \div (TP + FN)$	不良品有多少流出去？
誤判率 (過殺率)	$FP \div (FP + TN)$	良品有多少被誤殺？
F1	精確率與召回率的調和平均	兩者的綜合表現
AUROC	ROC 曲線下面積	與門檻無關的分辨能力
AP、mAP	PR 曲線下面積；對各類別平均	偵測模型的整體表現

24.5.2 偵測的評估：AP 與 mAP

偵測任務的評估多了一個問題：方框框得準不準。一個預測方框與真實缺陷的 IoU 達到門檻 (常用 0.5)，而且該缺陷還沒有被其他方框配對，才算真陽性；同一個缺陷的重複方框算偽陽性。把所有預測依信心分數由高到低排列，逐一累積精確率與召回率，得到 PR 曲線，曲線下的面積就是平均精確度 (AP)。AP@0.5 表示以 IoU 0.5 為門檻；COCO 資料集常用的 mAP@0.5:0.95，則是在 IoU 0.5 到 0.95、每隔 0.05 的十個門檻上平均，對定位精度要求更嚴格。mAP 是各類別 AP 的平均。

```
import numpy as np
# 物件偵測的後處理與評估：IoU、非最大值抑制 (NMS) 與平均精確度 (AP@0.5)
def iou(a, b):
    # 方框格式 (x1, y1, x2, y2)
    ix = max(0, min(a[2], b[2]) - max(a[0], b[0])); iy = max(0, min(a[3], b[3]) - max(a[1], b[1]))
    inter = ix * iy; ua = (a[2]-a[0])*(a[3]-a[1]) + (b[2]-b[0])*(b[3]-b[1]) - inter
    return inter / ua
def nms(boxes, scores, thr=0.5):
    order = np.argsort(-scores); keep = []
```

```

while len(order):
    i = order[0]; keep.append(i)
    order = np.array([j for j in order[1:] if iou(bboxes[i], bboxes[j]) < thr])
    return keep
def average_precision(dets, gts, thr=0.5):
    # dets: [(影像, 分數, 方
    框)]; gts: {影像: [方框]}
    dets = sorted(dets, key=lambda d: -d[1]); used = {k: [False] * len(v) for k, v in
    gts.items()}
    tp = []; npos = sum(len(v) for v in gts.values())
    for img, s, b in dets:
        ious = [iou(b, g) for g in gts.get(img, [])]; j = int(np.argmax(ious)) if ious
    else -1
        hit = j >= 0 and ious[j] >= thr and not used[img][j]
        if hit: used[img][j] = True
        tp.append(hit)
    tp = np.array(tp); ctp = np.cumsum(tp); prec = ctp / np.arange(1, len(tp) + 1); rec =
    ctp / npos
    ap = 0.0
    # 以所有點插值的方式計算
    PR 曲線下的面積
    mrec = np.r_[0, rec, 1]; mpre = np.r_[0, prec, 0]
    for i in range(len(mpre) - 2, -1, -1): mpre[i] = max(mpre[i], mpre[i + 1])
    idx = np.where(mrec[1:] != mrec[:-1])[0]; ap = np.sum((mrec[idx + 1] - mrec[idx]) *
    mpre[idx + 1])
    return ap, prec, rec
rng = np.random.default_rng(3)
gts, dets = {}, []
for k in range(50):
    # 50 張影像，每張 0~2 個
    缺陷
    g = []
    for _ in range(rng.integers(0, 3)):
        x, y = rng.uniform(20, 200, 2); w, h = rng.uniform(10, 40, 2); g.append((x, y, x
    + w, y + h))
    gts[k] = g
    for b in g:
        # 模型對每個缺陷產生 1~3
    個重疊候選框
        if rng.random() < 0.9:
            for _ in range(rng.integers(1, 4)):
                j = rng.normal(0, 3, 4); dets.append((k, float(np.clip(rng.normal(0.8,
    0.12), 0, 1)), tuple(np.array(b) + j)))
            for _ in range(rng.integers(0, 2)):
                # 誤報
                x, y = rng.uniform(20, 200, 2); dets.append((k, float(np.clip(rng.normal(0.45,
    0.15), 0, 1)), (x, y, x + 20, y + 20)))
    print(f"真實缺陷 {sum(len(v) for v in gts.values())} 個；原始候選框 {len(dets)} 個")
    ap_raw, _, _ = average_precision(dets, gts)
    kept = []
    for k in gts:
        d = [x for x in dets if x[0] == k]
        if d: kept += [d[i] for i in nms(np.array([x[2] for x in d]), np.array([x[1] for x in
    d]))]
    ap_nms, prec, rec = average_precision(kept, gts)
    print(f"NMS 後剩 {len(kept)} 個；AP@0.5 : NMS 前 {ap_raw:.3f}，NMS 後 {ap_nms:.3f}；最高召回
    率 {rec[-1]:.2f}")
    print(f"範例 IoU：完全重疊 {iou((0,0,10,10),(0,0,10,10)):.2f}，錯開一半
    {iou((0,0,10,10),(5,0,15,10)):.2f}")

```

執行結果：50 張影像中共有 47 個真實缺陷，模型產生 106 個候選框。NMS 後剩 76 個；AP@0.5 從 NMS 前的 0.539 提升到 0.687，最高召回率約 0.81（約一成的缺陷模型根本沒有產生方框，後處理也救不回來）。完全重疊的兩個方框 IoU 為 1.00，錯開一半時只有 0.33，低於 0.5 的門

檻。

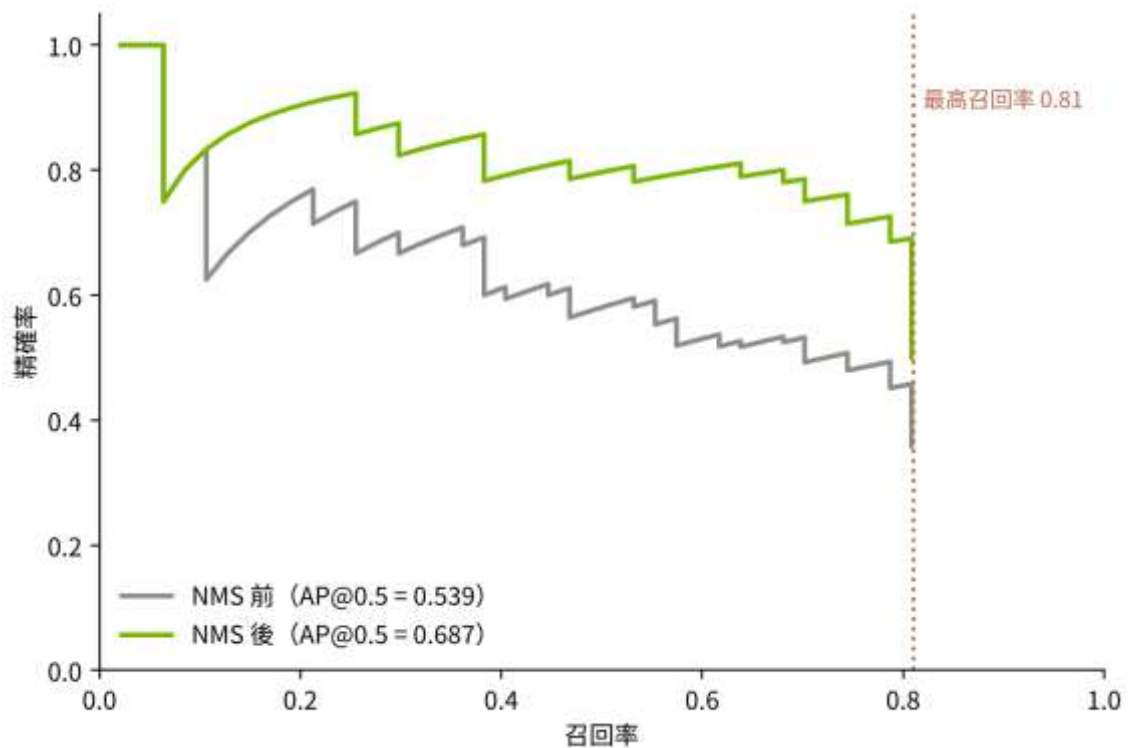


圖 24-5 NMS 前後的精確率 - 召回率曲線

NMS 前，同一缺陷的重複方框都被算成偽陽性，壓低了精確率；NMS 後重複框被移除，曲線明顯上移。這說明了兩件事：第一，偵測模型的成績與後處理的設定密切相關，比較不同模型時要用相同的後處理與評估程式；第二，NMS 的 IoU 門檻也是取舍，門檻太低可能把兩個靠得很近的真實缺陷合併成一個。

24.5.3 從指標到成本：門檻怎麼定

模型輸出的是分數，門檻決定了漏檢與誤判的平衡。門檻不應該由資料科學家憑感覺決定，而應該由漏檢與誤判的成本決定（第 22 章的成本敏感門檻）。實務上常用「雙門檻」：分數很低的直接判良品，分數很高的直接判不良，中間的灰色地帶送人工複檢。人工只需要看少量可疑件，可以專注、不易疲勞，漏看率遠低於全數目檢。

```
import numpy as np
# 延續 24.3.2 節程式中的 texture() 與 anomaly_map()
# 漏檢與誤判的成本：以異常檢測的分數，比較單一門檻與「雙門檻+人工複檢」的品質成本
test = [texture() for _ in range(900)] + [texture("spot") for _ in range(50)] +
[texture("line") for _ in range(50)]
y = np.r_[np.zeros(900), np.ones(100)] # 不良率 10%
sc = np.array([anomaly_map(im).max() for im, _ in test])
```

```

C_ESC, C_OK, C_REV = 20000, 800, 30 # 漏檢流出、良品誤判報廢、
人工複檢 (元/件)
def single(t):
    p = sc >= t; esc = np.sum(~p & (y == 1)); over = np.sum(p & (y == 0))
    return esc, over, (esc * C_ESC + over * C_OK) / len(y)
def dual(lo, hi, miss=0.05): # 分數介於 lo~hi 送人工
    複檢 (只看少量可疑件，漏看率 5%)
    rev = (sc >= lo) & (sc < hi); bad = sc >= hi
    esc = np.sum((sc < lo) & (y == 1)) + miss * np.sum(rev & (y == 1)); over = np.sum(bad
    & (y == 0))
    return esc, over, rev.sum(), (esc * C_ESC + over * C_OK + rev.sum() * C_REV) / len(y)
ts = np.quantile(sc, np.linspace(0.01, 0.99, 197))
best = min(ts, key=lambda t: single(t)[2]); e, o, c = single(best)
print(f"單一門檻 (成本最佳): 漏檢 {e} 件, 誤判 {o} 件, 平均成本 {c:.0f} 元/件")
lo = np.percentile(sc[:900], 60); hi = np.percentile(sc[:900], 99.5)
e2, o2, r2, c2 = dual(lo, hi)
print(f"雙門檻+人工複檢: 漏檢約 {e2:.1f} 件, 誤判 {o2} 件, 複檢 {r2} 件 ({r2/len(y):.0%}),
    平均成本 {c2:.0f} 元/件")
print(f"對照: 全數人工目檢 (長時間疲勞, 漏看率 15%) 平均成本 {(0.15*100*C_ESC +
    1000*C_REV)/1000:.0f} 元/件")

```

執行結果：在 1,000 件 (不良率 10%) 的測試中，成本最佳的單一門檻漏檢 8 件、誤判 88 件，平均成本約 230 元/件；雙門檻 + 人工複檢漏檢約 6.2 件、誤判只有 5 件，送複檢 380 件 (38%)，平均成本約 140 元/件；全數人工目檢 (漏看率 15%) 約 330 元/件。

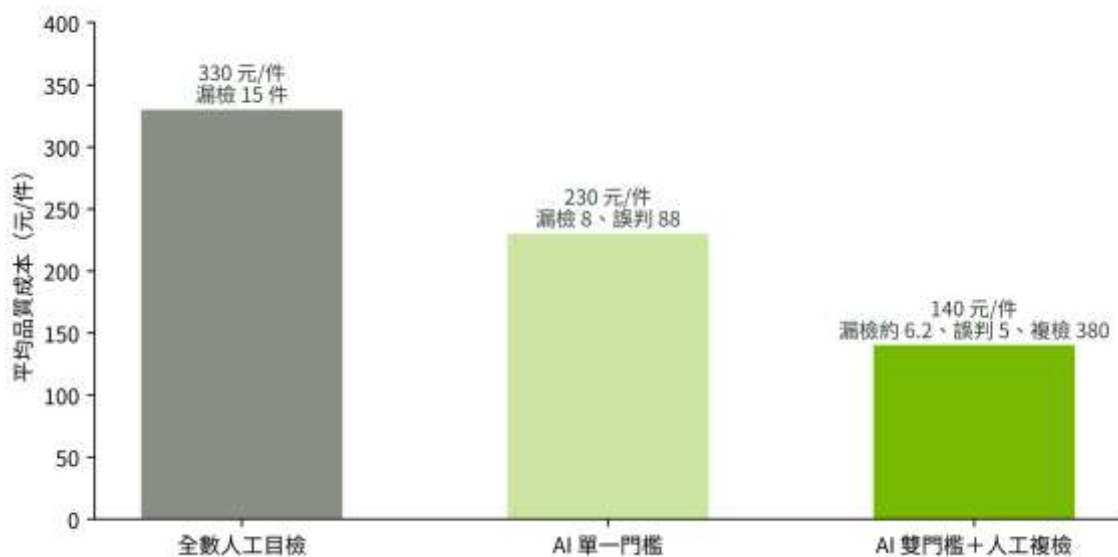


圖 24-6 三種檢測策略的平均品質成本

表 24-6 三種檢測策略的比較 (模擬)

策略	漏檢 (件)	誤判 (件)	人工複檢 (件)	平均成本 (元/件)
全數人工目檢	15	—	1,000	330
AI 單一門檻	8	88	0	230

策略	漏檢 (件)	誤判 (件)	人工複檢 (件)	平均成本 (元/件)
AI 雙門檻 + 人工複檢	約 6.2	5	380	140

這個結果有幾點值得注意。第一，即使是 AUROC 0.965 的模型，單一門檻仍然要在漏檢與誤判之間付出不小的代價；第二，雙門檻把人工用在最需要判斷的地方，成本最低，但 38% 的複檢比例仍然偏高，代表模型還有改善空間，複檢的結果也應該回饋為新的標註資料；第三，這裡的成本數字都是假設，實際導入時應以工廠自己的報廢、重工、客訴與人力成本計算。數字會改變最佳門檻，但「以成本定門檻」的方法不變。

24.5.4 驗收與上線後的監控

視覺檢測系統的驗收，應該以產線條件下、足夠數量的真實樣本為準，而不是展示用的樣品。常見做法包括：以已知良品與不良品組成的「標準樣本組」反覆測試，評估重複性；與人工檢驗並行一段時間，比較兩者的檢出結果（類似第 23 章的影子部署）；依缺陷類型分別統計召回率，避免罕見但嚴重的缺陷被平均掩蓋。量測系統分析（MSA）的觀念也適用於視覺檢測，包括重複性、再現性，以及屬性一致性分析（第 25 章）。

上線之後，要監控的包括：影像本身的統計（亮度、對比、清晰度）是否漂移，這往往是光源老化、鏡頭髒污或相機位移的早期徵兆；判定比例與分數分布是否改變；複檢與客訴的回饋是否顯示新的缺陷類型。新產品、新材料或新供應商導入時，應視為模型可能失效的時機，重新驗證。

案例

案例 24-1 教學情境：展示很完美，上線很慘

這是本章開頭的情境。展示樣品在暗房中以固定燈光拍攝，產線卻受日光與位置變化影響。改善從光學開始：遮光罩、低角度環形光源、定位治具，再以產線資料重新訓練並加入增強。

啟示：先確認相機拍到了瑕疵，再談模型；驗收要用產線條件。

案例 24-2 產業：AOI 與人工複判

依公開報導，印刷電路板與半導體封裝產業長期使用自動光學檢測（AOI），但傳統規則式 AOI 誤報偏高，需要大量人力複判。近年許多廠商以深度學習模型協助複判，過濾明顯的誤報，只把可疑件交給人員。

啟示：AI 不一定要取代原有的檢測，可以從降低複判負擔開始。

案例 24-3 產業：只有良品的異常檢測

新產品導入初期幾乎沒有不良樣本，難以訓練分類或偵測模型。以良品影像訓練的異常檢測（如 PaDiM、PatchCore 一類方法）可以先上線標示「與良品不同」的區域，累積的異常樣本再經人員確認，逐步建立有標註的缺陷資料集。

啟示：異常檢測與監督式模型可以接力，而不是二選一。

案例 24-4 產業：標註標準不一致

一個刮傷偵測專案中，三位標註人員對「多淺的刮痕算缺陷」看法不同，模型的表現始終無法提升。團隊先以實體樣品與照片建立缺陷判定標準，抽樣檢查標註者之間的一致性，再重新標註，模型的召回率才明顯改善。

啟示：標註的一致性往往比模型架構更影響成績；標準要先於標註。

案例 24-5 台灣情境：中小企業的第一套視覺檢測

一家生產五金零件的中小企業，以一台工業相機、一組背光與開源的影像處理程式，先做尺寸與有無的檢查，取代部分人工；累積經驗與影像後，再針對外觀缺陷導入深度學習模型。投資分階段進行，每一步都以實際的漏檢與人力成本評估。

啟示：從傳統影像處理能穩定解決的問題開始，再逐步導入 AI。

案例 24-6 教學情境：光源老化與模型漂移

某檢測站上線半年後，誤判逐漸增加。檢查發現 LED 光源亮度下降約兩成，影像整體變暗。團隊在監控中加入影像亮度與對比的統計管制，並定期以標準樣本校正。

啟示：模型漂移不一定來自產品，可能來自光學系統本身；要監控影像，不只監控判定結果。

實作活動

活動 24-1 相機配置計算

給定視野、最小缺陷與檢測速度，以 24.1 節的程式計算所需的像素數、像素解析度、放大率與景深，選擇合適的相機與鏡頭，並說明照明方式。

活動 24-2 照明與前處理

以手機或網路攝影機，在不同照明（正面光、低角度光、背光）下拍攝同一個有刮痕的物體，比較缺陷的對比；再以平場校正與形態學處理分割缺陷。

活動 24-3 遷移學習分類

以公開的工業缺陷資料集，使用預訓練的 CNN 進行遷移學習，比較只訓練最後一層與微調整個模型的差異，並以混淆矩陣說明結果。

活動 24-4 異常檢測

以 MVTec AD 等公開資料集的一個類別，只用良品影像訓練異常檢測模型，產生異常熱圖並計算影像層級的 AUROC，觀察哪些缺陷容易漏掉。

活動 24-5 資料增強實驗

設計三種增強策略（亮度、位移、旋轉），以 24.4 節的方法比較模型在不同測試條件下的準確率，討論哪些增強是合理的、哪些可能把缺陷「增強」掉。

活動 24-6 以成本定門檻

給定漏檢、誤判與複檢的成本，以 24.5 節的程式找出單一門檻與雙門檻的最佳設定，並分析成本假設改變時最佳門檻如何變化。

延伸閱讀

- Paul Bergmann 等，〈MVTec AD — A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection〉，CVPR 2019。
- Karsten Roth 等，〈Towards Total Recall in Industrial Anomaly Detection〉，CVPR 2022 (PatchCore)。
- Alexey Dosovitskiy 等，〈An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale〉，ICLR 2021 (Vision Transformer)。
- Joseph Redmon 等，〈You Only Look Once: Unified, Real-Time Object Detection〉，CVPR 2016。
- Carsten Steger、Markus Ulrich 與 Christian Wiedemann，Machine Vision Algorithms and Applications (第 2 版，Wiley-VCH，2018)：工業機器視覺的光學、相機與演算法。

表 24-7 本章思考地圖

本章概念	關鍵提問	後續延伸章節
光學與照明	缺陷在影像中有足夠的對比嗎？	第 25 章
相機配置	像素解析度、景深與速度都夠嗎？	第 6、25 章
視覺模型	需要 CNN、ViT 還是 YOLO？邊緣算得動嗎？	第 14、33 章
任務選擇	分類、偵測、分割還是異常檢測？	第 25 章
資料與標註	資料來自產線嗎？標註一致嗎？	第 23、28 章
評估與成本	門檻依成本決定了嗎？罕見缺陷抓得到嗎？	第 25、36 章
上線監控	影像與分數分布有沒有漂移？	第 23、36 章

章末小結

- 工業視覺的成敗首先取決於光學：照明決定缺陷的對比，相機與鏡頭決定像素解析度、視野與景深。範例中 100 mm 視野檢出 0.2 mm 缺陷需約 1,500 像素，2,448 像素的相機足夠；若要檢出 0.1 mm 則不足。照明不均時全域門檻失效 (Dice 0.06)，平場校正後 Dice 提升到 0.97。
- CNN 以卷積萃取局部特徵，搭配遷移學習可在少量資料下訓練；Vision Transformer 以自注意力建立全域關係，通常需要更多資料或預訓練；YOLO 一類的單階段偵測模型適合即時檢測，需搭配 IoU 與 NMS 的後處理。
- 分類回答「有沒有」，偵測回答「在哪裡」，分割回答「哪些像素」，異常檢測只用良品學習「與良品不同」。範例中以良品區塊的高斯分布建立異常熱圖，影像層級 AUROC 約 0.965，熱圖最高點落在真實缺陷的比例約 97%。
- 資料要來自產線條件，標註要先有一致的標準；資料增強模擬亮度與位置的變化，範例中把產線條件下的準確率從約 56% 提升到約 85%；合成影像適合補足罕見缺陷，但要以真實影像驗證。
- 評估要看精確率、召回率與 AP，不能只看準確率；範例中 NMS 使 AP@0.5 從 0.539 提升到 0.687。門檻應由漏檢與誤判的成本決定：雙門檻 + 人工複檢的平均成本約 140 元/件，低於單一門檻的 230 元與全數人工目檢的 330 元。上線後要監控影像統計、分數分布與回饋。

習題

一、選擇題

1. 使表面刮痕在影像中變成明顯亮線、適合凸顯表面紋理的照明方式是？(A) 背光 (B) 低角度照明 (C) 同軸照明 (D) 圓頂照明
2. 要清楚呈現物體外形輪廓與尺寸，最適合的照明是？(A) 背光 (B) 低角度照明 (C) 環境光 (D) 頻閃
3. 兩個方框交集面積除以聯集面積稱為？(A) AP (B) IoU (C) F1 (D) Dice
4. 去除同一物體重複方框的後處理方法是？(A) NMS (B) PCA (C) SVM (D) FFT

5. 以自注意力處理影像區塊的模型是？ (A) CNN (B) Vision Transformer (C) 決策樹 (D) k-means
6. 只用良品影像訓練、找出與良品不同區域的任務是？ (A) 分類 (B) 偵測 (C) 異常檢測 (D) 迴歸
7. 輸出每個像素類別的任務是？ (A) 分類 (B) 偵測 (C) 分割 (D) 排序
8. 不良品被判為良品稱為？ (A) 誤判 (B) 漏檢 (C) 過殺 (D) 重工
9. 判為不良的件數中真正不良的比例稱為？ (A) 召回率 (B) 精確率 (C) 準確率 (D) 特異度
10. 以 IoU 0.5 到 0.95 多個門檻平均的偵測指標是？ (A) AP@0.5 (B) mAP@0.5:0.95 (C) AUROC (D) F1

二、簡答題

11. 為什麼說「AI 只能看見相機拍到的東西」？舉例說明照明對檢測的影響。
12. 說明平場校正的原理，以及它解決的問題。
13. 比較 CNN 與 Vision Transformer 的特性與資料需求。
14. 說明 NMS 的步驟，以及 IoU 門檻設定的取舍。
15. 為什麼新產品導入初期適合使用異常檢測？它的限制是什麼？
16. 說明資料增強的目的，並舉一個不合理的增強例子。
17. 在不良率很低的產線上，為什麼準確率不是好的指標？
18. 說明雙門檻 + 人工複檢的做法與優點。

三、計算題

19. 視野 80 mm，最小缺陷 0.1 mm，要求缺陷至少涵蓋 3 個像素，相機在該方向至少需要多少像素？
20. 相機 2,448 像素對應 100 mm 視野，像素解析度為多少 $\mu\text{m}/\text{像素}$ ？

21. 方框 A 為 (0,0,10,10) · B 為 (5,5,15,15) · 求 IoU。
22. 100 件不良品中檢出 92 件，同時有 23 件良品被判為不良，求精確率與召回率。
23. 接上題，若良品共 900 件，誤判率為多少？F1 為多少？
24. 不良率 0.5%，模型全部判為良品，準確率與召回率各為多少？
25. 漏檢成本 20,000 元，誤判成本 800 元。某門檻漏檢 8 件、誤判 88 件，1,000 件的平均成本為多少？
26. 輸送帶速度每秒 0.5 m，相機曝光時間 100 μ s，像素解析度 41 μ m，影像拖影約為多少像素？

四、申論與設計題

27. 為一條生產手機金屬外殼的產線設計外觀檢測系統：說明照明、相機、模型任務、資料蒐集與標註、評估指標與上線監控。
28. 評論以下說法：「只要模型夠強，照明與相機隨便裝就可以了。」
29. 設計一套視覺檢測系統的驗收程序，說明樣本、指標、與人工檢驗並行的方式與合格標準。

五、是非題

30. 照明不均時，單一的全域門檻可能把大片背景誤判為缺陷。
31. 準確率 99.5% 的模型一定適合上線。
32. NMS 的 IoU 門檻越低，越不會漏掉靠得很近的兩個缺陷。
33. 異常檢測只需要良品影像就可以訓練。
34. 資料增強可以完全取代穩定的照明與治具。
35. 光源老化可能導致視覺檢測模型的表現下降。

六、進階挑戰題

36. 以開源的 YOLO 套件，在公開的表面缺陷資料集上訓練偵測模型，報告 AP@0.5 與 mAP@0.5:0.95，並分析不同 NMS 門檻的影響。
37. 實作 PatchCore 或 PaDiM，在 MVTec AD 的三個類別上評估影像層級與像素層級的 AUROC，

並與本章的簡化版比較。

38. 以 Blender 等軟體產生帶有隨機缺陷的合成影像，與真實影像混合訓練，比較只用真實資料與加入合成資料的召回率。

七、補充題

39. 列出表 24-5 中的七個檢測評估指標。
40. 列出本章介紹的四種視覺任務。

提示與參考解答

1. B。
2. A。
3. B。
4. A。
5. B。
6. C。
7. C。
8. B。
9. B。
10. B。
11. 參考：模型只能從影像學習，缺陷若在影像中沒有對比，任何模型都看不見。例如刮痕在正面光下幾乎不可見，在低角度照明下成為明顯亮線；反光、陰影也可能被誤認為缺陷。
12. 參考：以大尺度的形態學運算或空白影像估計背景亮度，再以原影像除以背景，消除照明的不均勻，使同一門檻在整張影像都適用。
13. 參考：CNN 以局部卷積與權重共享，具有平移與局部性的歸納偏置，小資料也能訓練；ViT 以自注意力建立全域關係，彈性較大，但通常需要大量資料或預訓練模型。

14. 參考：依分數排序，保留最高者，刪除與它 IoU 超過門檻的方框，重複直到處理完。門檻低則刪得多，可能合併相鄰的真實缺陷；門檻高則可能留下重複框。
15. 參考：初期不良樣本極少，難以訓練監督式模型；異常檢測只需良品。限制包括難以區分缺陷類型、對良品的正常變異敏感、細微缺陷可能漏掉。
16. 參考：模擬產線上真實會出現的變化，提升穩健性。不合理的例子：對有方向性的零件做大角度旋轉、把亮度調到使缺陷消失，或裁切掉缺陷。
17. 參考：不良極少時，全部判良品就能得到很高的準確率，卻一件不良也抓不到；應看召回率、精確率與成本。
18. 參考：低分直接判良、高分直接判不良，中間送人工複檢。人工只看少量可疑件，可專注且不易疲勞，同時兼顧漏檢與誤判，複檢結果也可回饋為新的標註資料。
19. $80 \div 0.1 \times 3 = 2,400$ 像素。
20. $100 \text{ mm} \div 2,448 \approx 0.0408 \text{ mm}$ ，約 $41 \mu\text{m}$ /像素。
21. 交集 $5 \times 5 = 25$ ，聯集 $100 + 100 - 25 = 175$ ， $\text{IoU} \approx 0.14$ 。
22. 精確率 $92 \div (92 + 23) = 0.80$ ；召回率 $92 \div 100 = 0.92$ 。
23. 誤判率 $23 \div 900 \approx 2.6\%$ ； $F1 = 2 \times 0.80 \times 0.92 \div (0.80 + 0.92) \approx 0.86$ 。
24. 準確率 99.5%；召回率 0。
25. $(8 \times 20,000 + 88 \times 800) \div 1,000 = (160,000 + 70,400) \div 1,000 \approx 230$ 元/件。
26. 曝光期間移動 $0.5 \text{ m/s} \times 100 \mu\text{s} = 50 \mu\text{m}$ ，約 $50 \div 41 \approx 1.2$ 像素。
27. 參考：低角度或圓頂照明搭配遮光罩、治具定位；依缺陷大小選相機；初期以異常檢測、累積後改用偵測或分割；在產線蒐集資料並建立標註標準；以召回率、誤判率與成本評估，雙門檻 + 複檢；上線後監控影像統計與分數分布。
28. 參考：不同意。照明與相機決定缺陷是否可見，模型無法憑空看見影像中沒有的資訊；不穩定的成像會讓模型學到錯誤的特徵，也使上線後容易漂移。
29. 參考：以產線條件下的標準樣本組（含各類缺陷）測試重複性；與人工並行數週；依缺陷類型統

計召回率與誤判率；合格標準依成本與客戶要求訂定，並規定重新驗證的時機。

30. 是。

31. 否，不良率低時準確率可能誤導，應看召回率與成本。

32. 否，門檻越低刪除越多，越可能把相鄰的兩個缺陷合併。

33. 是。

34. 否，增強只能模擬部分變化，應先從源頭控制。

35. 是。

36. 提示：固定訓練設定，只改變 NMS 的 IoU 門檻，觀察 AP 與相鄰缺陷的合併情形。

37. 提示：以預訓練 CNN 的中間層特徵取代原始像素，像素層級以熱圖與標註遮罩計算。

38. 提示：以相同數量的真實測試影像評估，注意合成比例過高時的表現。

39. 精確率、召回率、漏檢率、誤判率、F1、AUROC、AP (mAP) 。

40. 分類、偵測、分割、異常檢測。

第 25 章

智慧品質：SPC、DOE 與閉環控制

編著 李世淵（機器人叫獸） | 助教 李天宇、李宇晴

本章重點

- TQM、精實、六標準差與 QMS：品質從檢驗走向預測閉環
- 管制圖、西方電氣規則、Cp / Cpk 與 EWMA；量具 GR&R
- 實驗設計、反應曲面與 Run-to-Run 控制
- 智慧品質與 DFSQ 六階段：定義、量測、理解、預測、控制、學習
- 良率模型、晶圓圖與半導體、電子組裝的製程內檢測

叫獸開講：管制圖上的那條線

有一次我到一家做汽車零件的工廠，品保課長很自豪地帶我看牆上的管制圖：每天一張，點子都在管制界限內，「我們的製程很穩定」。我問他：「這張圖的管制界限是怎麼算的？」他翻了翻資料夾，說是三年前的顧問算的。我再問：「那這三年換過刀具、換過量具、換過操作員嗎？」他沉默了一下。

我們把最近一個月的資料重新整理：先做量具的重複性與再現性分析，發現那支用了很多年的游標卡尺，量測變異幾乎佔了零件變異的一半——管制圖上的點子跳動，有一大半是量具在跳，不是製程在跳。換成數位測微器、加上治具之後，重新計算管制界限，原本「很穩定」的製程，立刻出現了連續八點在中心線同一側的訊號：刀具磨耗造成的緩慢偏移，一直藏在量測雜訊裡。

課長問：「那以前的管制圖都白畫了嗎？」我說：「不是白畫，是沒有閉環。管制圖告訴你製程變了，接下來要有人找原因、要有人調整、要有人確認調整有效，而且量測本身要可信。」後來，他們把量測資料直接從測微器上傳，管制圖自動更新，偏移警報直接通知機台旁的技術員，並由系統建議刀具補償量。

這一章，我們從品質管理的歷史與體系談起；接著介紹統計製程管制、製程能力與量測系統分析；再談實驗設計、反應曲面與半導體常用的 Run-to-Run 控制；然後把這些方法整合成「智慧品質」的閉環架構；最後以良率分析，以及半導體與電子組裝的製程內檢測案例作結。

（說明：本節情境由作者輔導企業品質改善的經驗綜合改寫，人物與細節已調整。）

學習目標

讀完本章後，你應該能夠：

1. 說明 TQM、精實、六標準差與品質管理系統的核心精神與關係。
2. 建立 X-R 管制圖、判讀西方電氣規則，計算 C_p 、 C_{pk} ，並比較 Shewhart 與 EWMA 管制圖對小偏移的偵測能力。
3. 以變異數分析法進行量具重複性與再現性分析，判斷量測系統是否可信。

4. 設計與分析因子實驗，以反應曲面法找出最佳製程條件。
5. 說明 Run-to-Run 控制的原理，並以 EWMA 控制器補償製程漂移。
6. 說明智慧品質的閉環架構，並以良率模型與晶圓圖分析半導體與電子組裝的品質問題。

核心概念

表 25-1 本章核心名詞中英對照

中文名詞	英文	一句話說明
全面品質管理	Total Quality Management, TQM	全員參與、以顧客為中心、持續改善的管理哲學
六標準差	Six Sigma	以資料與 DMAIC 流程降低變異的改善方法
品質管理系統	Quality Management System, QMS	以文件化流程確保品質的管理架構，如 ISO 9001
統計製程管制	Statistical Process Control, SPC	以統計方法監控製程是否只受共同原因影響
管制圖	Control Chart	依時間繪製統計量與管制界限，偵測特殊原因
製程能力指標	Process Capability Index, Cp / Cpk	比較規格寬度與製程變異的指標
EWMA 管制圖	Exponentially Weighted Moving Average	累積過去資訊、對小偏移較敏感的管制圖
量測系統分析	Measurement System Analysis, MSA	評估量測變異是否小到足以判斷製程
量具重複性與再現性	Gage R&R, GR&R	同一人重複量測與不同人之間的量測變異
實驗設計	Design of Experiments, DOE	有系統地同時改變多個因子，找出影響與交互作用
反應曲面法	Response Surface Methodology, RSM	以二次模型描述反應，尋找最佳操作條件
Run-to-Run 控制	Run-to-Run Control, R2R	依批後量測，逐批調整下一批的製程參數
晶圓圖	Wafer Map	以晶粒位置呈現良否，揭示空間型態
製程內檢測	In-process Inspection	在製程中途檢測，及早攔截不良

25.1 TQM、精實、六標準差與 QMS

25.1.1 品質觀念的演進

工業化初期，品質靠的是最終檢驗：做完了再挑出不良品。1924 年，任職於西方電氣公司（其工程部門隨後併入新成立的貝爾實驗室）的 Walter Shewhart 在一份內部備忘錄中提出管制圖，主張以統計方法區分製程的「共同原因」與「特殊原因」變異，品質從「挑出來」走向「做出來」。二次大戰後，W. Edwards Deming 與 Joseph Juran 到日本講授品質管理，日本企業發展出全公司品質管制與品管圈；Armand Feigenbaum 則提出全面品質管制的概念。這些思想在 1980 年代回流美國，形成全面品質管理（TQM）。

表 25-2 主要品質管理方法的比較

方法	核心精神	代表工具	與智慧製造的關係
TQM	全員參與、顧客導向、持續改善	PDCA、品管圈、方針管理	提供文化與組織基礎
精實生產	消除浪費、流動、拉式、內建品質	價值流圖、防呆、標準作業、自動化	第 8 章；即時異常停線是閉環的原型
六標準差	以資料降低變異、專案式改善	DMAIC、SPC、DOE、MSA	提供統計方法與改善流程
QMS (ISO 9001、IATF 16949 等)	以文件化流程確保一致性與可追溯	程序、稽核、矯正預防措施	數位化後成為 eQMS，與 MES 整合 (第 17 章)
智慧品質	以感測、資料與 AI 形成即時閉環	線上 SPC、虛擬量測、R2R、AI 檢測	本章 25.4 節

25.1.2 精實與六標準差

精實生產源自豐田生產方式（第 8 章），其中「自動化」強調機器在異常時自動停止、不把不良品往下傳，正是內建品質的思想。六標準差則由摩托羅拉在 1980 年代中期發展（一般歸功於工程師 Bill Smith，約 1986 年），後經奇異等企業推廣；它以「每百萬機會的缺陷數」衡量品質，並以

DMAIC (定義、量測、分析、改善、管制) 作為改善專案的流程。新產品開發則常用「六標準差設計」(DFSS)，在設計階段就把品質放進產品與製程。

兩者各有所長：精實擅長消除流程中的浪費與等待，六標準差擅長以統計方法降低變異，因此許多企業把它們合稱「精實六標準差」。

25.1.3 品質管理系統

品質管理系統以文件化的流程，確保產品與服務的一致性。ISO 9001 是最普遍的通用標準，強調流程方法、風險導向思維與持續改善；汽車業有 IATF 16949，並配合 APQP、PPAP、FMEA、SPC、MSA 等核心工具；醫療器材有 ISO 13485；航太有 AS9100。製藥與醫療器材的電子紀錄與電子簽章，則受各國法規規範 (第 17 章)。

傳統 QMS 大量依賴紙本與事後稽核；數位化之後，檢驗紀錄、不合格品處理、矯正預防措施與文件管理都在電子品質管理系統 (eQMS) 中完成，並與 MES、ERP、PLM 連結，品質資料可以即時追溯到料號、批號、設備與參數。

25.2 SPC、管制圖與製程能力指標

25.2.1 共同原因與特殊原因

任何製程都有變異。共同原因變異來自許多微小、隨機的因素，是製程固有的；特殊原因變異來自可以找出的原因，例如刀具磨耗、原料換批、量具故障。只受共同原因影響的製程稱為「統計管制狀態」。SPC 的目的不是消除所有變異，而是及早發現特殊原因，並避免對共同原因過度反應——對隨機變動也去調整機台，反而會讓變異變大。

25.2.2 管制圖的種類與判讀

管制圖以中心線與上下管制界限 (通常為 $\pm 3\sigma$) 呈現統計量隨時間的變化。依資料型態，常用的管制圖如下：

表 25-3 常用管制圖

資料型態	管制圖	適用情境
計量值，子群 $n = 2 \sim 10$	X-R	機械加工尺寸，每次抽 5 件
計量值，子群 $n > 10$	X-S	自動量測、樣本數多
計量值，單筆	I-MR	化工批次、破壞性試驗
計數值，不良率	p、np	外觀檢驗的不良比例
計數值，缺點數	c、u	每片 PCB 的焊點缺點數
小幅偏移	EWMA、CUSUM	緩慢漂移、高價值製程
多變量	Hotelling T^2 、SPE	多個相關變數（第 23 章）

除了超出管制界限，管制圖也以「規則」判斷非隨機的型態。西方電氣公司在 1956 年出版的《統計品質管制手冊》中整理了一組常用規則，例如：一點超出 3σ ；連續 3 點中有 2 點超過同側 2σ ；連續 5 點中有 4 點超過同側 1σ ；連續 8 點落在中心線同一側。規則越多，對偏移越敏感，但誤報也越多。

25.2.3 製程能力指標

管制界限是製程「自己的聲音」，規格界限是顧客的要求，兩者不可混淆。製程能力指標比較兩者：

$$C_p = (USL - LSL) \div 6\sigma; C_{pk} = \min(USL - \mu, \mu - LSL) \div 3\sigma$$

C_p 只看變異， C_{pk} 同時考慮平均值是否偏離中心。汽車業的實務中，量產製程常要求 C_{pk} 至少 1.33，關鍵特性或新製程則常要求 1.67； C_{pk} 1.33 相當於最近的規格界限距平均值 4σ 。計算能力指標的前提是製程處於管制狀態、資料近似常態，且量測系統可信。

下面的程式模擬一個軸徑加工製程：先以 25 組、每組 5 件的資料建立 X-R 管制圖並計算能力指標；再監控 40 組資料，其中第 21 組起平均值因刀具磨耗偏移 0.75σ 。最後以模擬比較 Shewhart 管制圖與 EWMA 管制圖偵測不同大小偏移所需的平均組數（平均連串長度，ARL）：

```
import numpy as np
# SPC：X-R 管制圖、西方電氣規則、製程能力，以及 EWMA 對小幅偏移的偵測速度
rng = np.random.default_rng(1)
```

```

mu0, sd, n = 10.00, 0.02, 5                                # 軸徑 10.00 mm，規格
10.00 ± 0.08 mm
LSL, USL = 9.92, 10.08
base = rng.normal(mu0, sd, (25, n))                        # 第一階段：25 組，建立
管制界限
xbar, R = base.mean(1), np.ptp(base, 1); xbb, Rb = xbar.mean(), R.mean()
A2, D3, D4, d2 = 0.577, 0, 2.114, 2.326                  # n = 5 的管制圖常數
UCL, LCL = xbb + A2 * Rb, xbb - A2 * Rb
sigma = Rb / d2
Cp = (USL - LSL) / (6 * sigma); Cpk = min(USL - xbb, xbb - LSL) / (3 * sigma)
def we_rules(x, c, s):                                     # 西方電氣規則 1、2、4
    (簡化)
    z = (x - c) / s; alarms = []
    for i in range(len(z)):
        if abs(z[i]) > 3: alarms.append((i, "規則 1：超出 3σ"))
        elif i >= 2 and (np.sum(z[i-2:i+1]) > 2) >= 2 or np.sum(z[i-2:i+1] < -2) >= 2):
    alarms.append((i, "規則 2：3 點中 2 點超過 2σ"))
        elif i >= 7 and (np.all(z[i-7:i+1]) > 0) or np.all(z[i-7:i+1] < 0)):
    alarms.append((i, "規則 4：連續 8 點在同一側"))
    return alarms
mon = rng.normal(mu0, sd, (40, n)); mon[20:] += 0.75 * sd  # 第二階段：第 21 組起平均偏移
0.75σ (刀具磨耗)
xm = mon.mean(1); s_x = sigma / np.sqrt(n)
def ewma_arl(shift, lam=0.2, L=2.86, reps=4000):          # 以模擬估計 EWMA 與
Shewhart 的平均連串長度
    out = []
    for kind in ("shewhart", "ewma"):
        runs = []
        for _ in range(reps):
            z, t = 0.0, 0
            while True:
                t += 1; x = rng.normal(shift, 1)
                if kind == "shewhart":
                    if abs(x) > 3: break
                else:
                    z = lam * x + (1 - lam) * z
                    if abs(z) > L * np.sqrt(lam / (2 - lam)): break
                    if t > 3000: break
            runs.append(t)
        out.append(np.mean(runs))
    return out
print(f"管制界限：中心線 {xbb:.4f}，UCL {UCL:.4f}，LCL {LCL:.4f}； $\bar{R}$  {Rb:.4f}，估計  $\sigma$ 
{sigma:.4f}")
print(f"製程能力：Cp {Cp:.2f}，Cpk {Cpk:.2f}")
a = we_rules(xm, xbb, s_x); print("第一個警報：", (a[0][0] + 1, a[0][1]) if a else "無",
f"；共 {len(a)} 個警報")
for sh in (0.5, 1.0, 2.0):
    s, e = ewma_arl(sh * np.sqrt(n)); print(f"平均偏移 {sh}σ (樣本平均
{sh*np.sqrt(n):.2f}σ $\bar{x}$ )：Shewhart ARL {s:.1f} 組，EWMA ARL {e:.1f} 組")
s, e = ewma_arl(0.0, reps=1500); print(f"未偏移時的誤報 ARL：Shewhart {s:.0f}，EWMA
{e:.0f}")

```

執行結果：管制界限中心線約 9.9991 mm，UCL 10.0238、LCL 9.9745，估計 σ 約 0.0184 mm；Cp 約 1.45、Cpk 約 1.44。偏移從第 21 組開始，第一個警報出現在第 28 組，觸發的是「連續 8 點在同一側」的規則，而不是超出管制界限——小偏移通常要靠型態規則或 EWMA 才能及早發現。ARL 的比較顯示：平均偏移 0.5 σ 時，Shewhart 圖平均要 33.5 組才發出警報，EWMA 只要約

8.2 組；偏移 2σ 時，Shewhart 約 1.1 組，反而比 EWMA 的 1.6 組快。兩者在未偏移時的誤報 ARL 都約為 380 組。

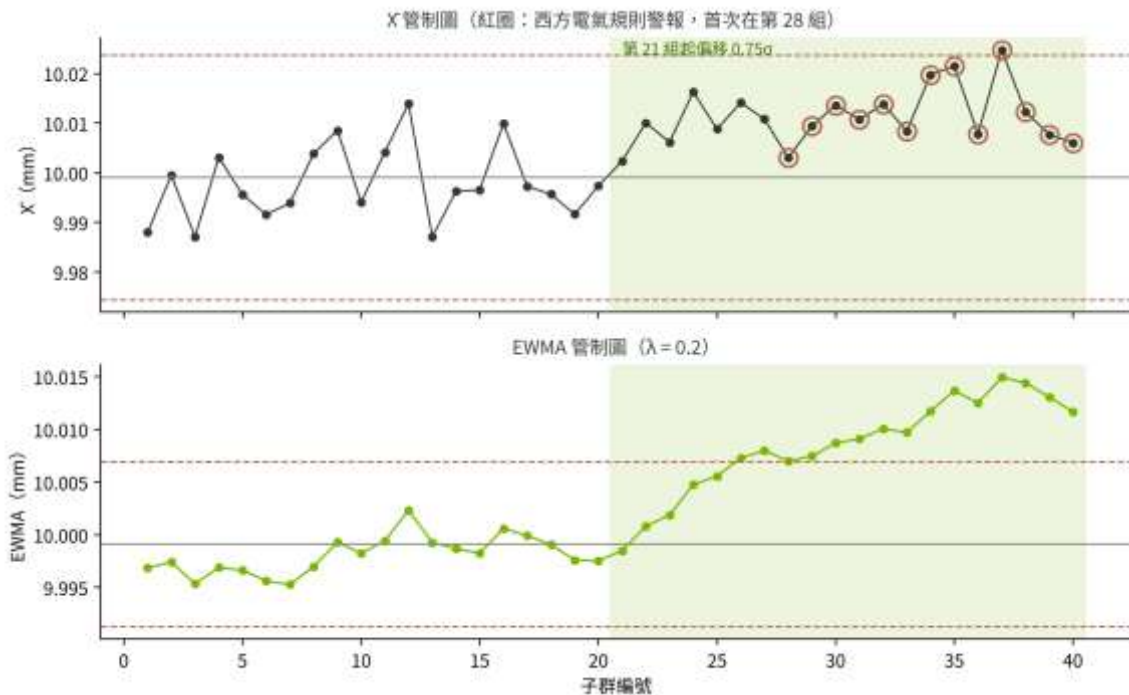


圖 25-1 $\bar{X}$ 管制圖與 EWMA 管制圖對小幅偏移的反應

結論是：大偏移用 Shewhart 圖，小而緩慢的偏移用 EWMA 或 CUSUM；在自動化產線上，兩者可以同時執行。

25.2.4 量測系統分析

管制圖與能力指標都建立在量測資料上。若量測系統的變異太大，管制圖上看到的主要是量具的雜訊。量測系統分析 (MSA) 評估量測的偏差、線性、穩定性、重複性與再現性；其中最常用的是量具重複性與再現性 (GR&R) 分析：選約 10 個涵蓋製程範圍的零件，由 2~3 位量測員各量 2~3 次，以變異數分析拆解出零件、量測員、交互作用與重複量測的變異。

依汽車業 MSA 手冊的常用準則，%GR&R (量測變異佔總變異的比例) 低於 10% 可接受，10%~30% 視應用與成本有條件接受，超過 30% 不可接受；可區分類別數 (ndc) 應至少為 5。

```
import numpy as np
# 量測系統分析 (MSA)：以變異數分析 (ANOVA) 法計算量具重複性與再現性 (GR&R)
rng = np.random.default_rng(2)
p, o, r = 10, 3, 3                                     # 10 個零件 × 3 位量測員
                                                         # 3 次重複
def study(sd_part, sd_rep, sd_op):
```

```

part = rng.normal(0, sd_part, p); op = rng.normal(0, sd_op, o)
y = part[:, None, None] + op[None, :, None] + rng.normal(0, sd_rep, (p, o, r))
gm = y.mean(); mp = y.mean((1, 2)); mo = y.mean((0, 2)); mpo = y.mean(2)
SSp = o * r * np.sum((mp - gm)**2); SSo = p * r * np.sum((mo - gm)**2)
SSpo = r * np.sum((mpo - mp[:, None] - mo[None, :] + gm)**2); SSe = np.sum((y -
mpo[:, :, None])**2)
MSp, MSo = SSp / (p - 1), SSo / (o - 1); MSpo = SSpo / ((p - 1) * (o - 1)); MSe = SSe / (p * o * (r - 1))
v_rep = MSe; v_po = max((MSpo - MSe) / r, 0); v_op = max((MSo - MSpo) / (p * r), 0)
v_part = max((MSp - MSpo) / (o * r), 0); v_grr = v_rep + v_op + v_po; v_tot = v_grr + v_part
grr = np.sqrt(v_grr / v_tot); ndc = int(1.41 * np.sqrt(v_part / v_grr))
return np.sqrt(v_rep / v_tot), np.sqrt((v_op + v_po) / v_tot), grr, ndc
for name, args in [("原本的量具 (游標卡尺)", (0.010, 0.006, 0.004)), ("改善後 (數位測微器 + 治具)", (0.010, 0.0015, 0.0008))]:
    rep, rpd, grr, ndc = study(*args)
    verdict = "可接受" if grr < 0.10 else ("有條件接受" if grr < 0.30 else "不可接受")
    print(f"{name}: 重複性 {rep:.0%}, 再現性 {rpd:.0%}, %GR&R {grr:.0%} ({verdict}), 可區分類別數 {ndc}")

```

執行結果：原本以游標卡尺量測，重複性約 43%、再現性約 24%，%GR&R 約 49%，不可接受，可區分類別數只有 2——也就是量具只能分出「大」與「小」兩類零件。改用數位測微器並加上治具後，%GR&R 降到約 17%，可區分類別數 8，屬於有條件接受。

這正是本章開頭的情境：在改善量測之前，任何管制圖與能力分析都不可信。在智慧工廠中，量測儀器直接上傳資料，可以定期自動執行穩定性與 GR&R 檢查；視覺檢測（第 24 章）等屬性判定，則以屬性一致性分析評估判定者之間與判定者自己的一致性。

25.3 實驗設計、反應曲面與 Run-to-Run 控制

25.3.1 為什麼要做實驗設計

現場最常見的試驗方法是「一次改一個因子」：固定其他條件，只調溫度，找到最好的溫度後，再調壓力。這種方法的問題是：看不到交互作用。若溫度高時壓力的影響變大，一次一因子的試驗可能停在錯誤的條件，而且實驗次數多。實驗設計（DOE）同時、有系統地改變多個因子，以較少的實驗次數估計主效應與交互作用。

表 25-4 常用的實驗設計

設計	用途	實驗次數 (k 個因子)	說明
2^k 全因子	估計所有主效應與交互作用	2^k	因子不多時的首選
2^{k-p} 部分因子	篩選重要因子	2^{k-p}	犧牲高階交互作用，換取較少次數
Plackett-Burman	大量因子的篩選	約 $k+1$	只估計主效應
中心複合設計 (CCD)	配適二次反應曲面	$2^k + 2k + \text{中心點}$	反應曲面法的標準設計
Box-Behnken	配適二次曲面	較 CCD 少	不需要極端的角落條件
田口方法	穩健設計	直交表	以訊號雜訊比降低對雜訊因子的敏感度

反應曲面法由 Box 與 Wilson 在 1951 年提出，通常分為兩個階段：先以因子設計找出重要因子與改善方向，再在最佳區域附近以中心複合設計配適二次模型，求出最佳條件。中心點的重複實驗可以估計純誤差，也可以檢查曲率。

25.3.2 範例：射出成形的強度最佳化

下面的程式模擬射出成形的拉伸強度，因子為料溫 A、射出壓力 B 與保壓時間 C，以編碼值 -1 、 $+1$ 表示低、高水準。第一階段做 2^3 全因子實驗 (8 次)，估計各效應；第二階段只保留重要的 A、B，以中心複合設計 (8 個設計點 + 4 個中心點) 配適二次模型，並求出曲面的最高點：

```
import numpy as np, itertools
# 實驗設計：23 全因子實驗估計主效應與交互作用，再以中心複合設計 (CCD) 配適反應曲面、找出最佳條件
rng = np.random.default_rng(5)
def process(T, P, t):                                     # 射出成形的拉伸強度
    (MPa)，未知的真實製程；T、P、t 為編碼值 -1~+1
    return 52 + 3.0*T + 1.8*P + 0.3*t + 1.5*T*P - 2.2*T**2 - 1.4*P**2 + rng.normal(0,
    0.5)
X = np.array(list(itertools.product([-1, 1], repeat=3)))    # 8 組實驗
y = np.array([process(*x) for x in X])
names = ["溫度 A", "壓力 B", "保壓時間 C", "AB", "AC", "BC", "ABC"]
cols = [X[:, 0], X[:, 1], X[:, 2], X[:, 0]*X[:, 1], X[:, 0]*X[:, 2], X[:, 1]*X[:, 2],
X[:, 0]*X[:, 1]*X[:, 2]]
effects = {nm: 2 * np.mean(c * y) for nm, c in zip(names, cols)} # 效應 = 高水準平均 -
低水準平均
a = np.sqrt(2)                                             # 第二階段：A、B 兩因子
```

的中心複合設計 (C 固定在低水準)

```
pts = [(-1,-1),(1,-1),(-1,1),(1,1),(-a,0),(a,0),(0,-a),(0,a)] + [(0,0)]*4
Z = np.array(pts); yc = np.array([process(T, P, -1) for T, P in Z])
D = np.c_[np.ones(len(Z)), Z[:,0], Z[:,1], Z[:,0]*Z[:,1], Z[:,0]**2, Z[:,1]**2]
b = np.linalg.lstsq(D, yc, rcond=None)[0]
H = np.array([[2*b[4], b[3]], [b[3], 2*b[5]]]); opt = np.linalg.solve(H, -b[1:3])
pred = b @ np.r_[1, opt, opt[0]*opt[1], opt[0]**2, opt[1]**2]
print("23 全因子效應：" + " , ".join(f"{k} {v:+.2f}" for k, v in effects.items()))
print("二次模型係數 (常數、A、B、AB、A2、B2)：" + " , ".join(f"{v:.2f}" for v in b))
print(f"最佳條件 (編碼值) A = {opt[0]:.2f}, B = {opt[1]:.2f}, 預測強度 {pred:.1f} MPa；中心點  
平均 {yc[-4:].mean():.1f} MPa")
```

執行結果：溫度 A 的效應約 +6.23 MPa，壓力 B 約 +3.56 MPa，AB 交互作用約 +2.39 MPa，而保壓時間 C 只有約 +0.46 MPa，可以在下一階段固定。二次模型的 A^2 、 B^2 係數皆為負值（約 -1.90、-1.37），表示曲面有最高點；最佳條件約在 $A = 1.16$ 、 $B = 1.03$ （編碼值），預測強度約 54.0 MPa，比中心點的 51.4 MPa 高。這組模擬的真實最佳點約在 $A \approx 1.10$ 、 $B \approx 1.23$ ，最高約 54.5 MPa，估計相當接近。

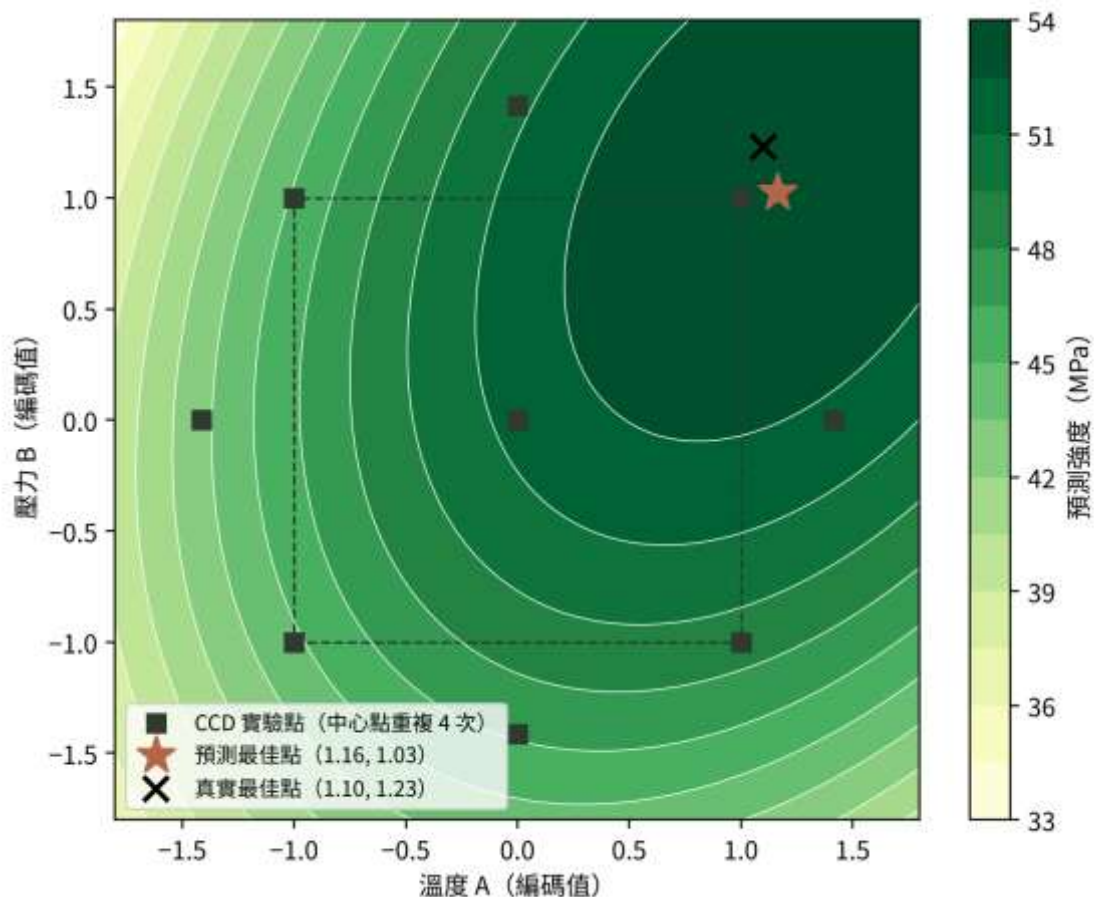


圖 25-2 中心複合設計的實驗點與配適的反應曲面

兩點值得注意。第一，最佳點落在原本試驗範圍（ ± 1 ）的稍外側，實務上應在該處做確認實驗，而不是直接相信外插的預測；第二， 2^3 全因子實驗只有兩個水準，看不到曲率，若沒有中心點，

可能誤以為「溫度越高越好」。

25.3.3 Run-to-Run 控制

半導體製程的許多步驟無法在加工中即時量測，只能在一批（一片或一盒晶圓）完成後量測。Run-to-Run (R2R) 控制依批後量測的結果，調整下一批的配方參數，以補償設備的漂移與換批的偏移。最常用的是以 EWMA 估計製程的偏移或增益，再依模型計算下一批的設定：

$$\text{估計值}_{k+1} = \lambda \times (\text{本批觀測值}) + (1 - \lambda) \times \text{估計值}_k$$

λ 越大，對新資訊反應越快，但也越容易被量測雜訊影響； λ 越小越平滑，但追蹤漂移較慢。

R2R 控制在 1990 年代於半導體業發展成熟，常見於化學機械研磨 (CMP)、微影的疊對與曝光劑量、蝕刻與薄膜沉積。

下面的程式模擬 CMP：研磨墊逐漸磨耗，使移除率從約 40 Å/s 緩慢下降，並有週期性的擾動；目標移除量為 2,000 Å。開迴路使用固定的研磨時間；R2R 則以 EWMA 估計移除率，逐批調整研磨時間：

```
import numpy as np
# Run-to-Run 控制：CMP 研磨的移除率隨研磨墊磨耗而漂移；以 EWMA 估計移除率，逐批調整研磨時間
target, N = 2000.0, 100                                # 目標移除量 (Å)、批數
def rate(k): return 40.0 - 0.06 * k + 0.8 * np.sin(k / 6) # 真實移除率 (Å/s)：磨耗
造成的漂移+週期擾動
def run(lam=None, seed=4):
    rng = np.random.default_rng(seed); est, out = 40.0, [] # 相同的雜訊序列，方便公平
    比較
    for k in range(N):
        t = target / est # 依估計的移除率決定研磨時
        間
        y = rate(k) * t + rng.normal(0, 25) # 批後量測 (含量測雜訊)
        out.append(y - target)
        if lam is not None: est = lam * (y / t) + (1 - lam) * est # EWMA 更新；lam 為
        None 時不更新 (開迴路)
    return np.array(out)
for name, lam in [("開迴路 (固定研磨時間)", None), ("R2R λ = 0.1", 0.1), ("R2R λ = 0.3",
0.3), ("R2R λ = 0.7", 0.7), ("R2R λ = 1.0", 1.0)]:
    e = run(lam)
    print(f"{name}：平均偏差 {e.mean():+.0f} Å，RMSE {np.sqrt(np.mean(e**2)):.0f} Å，超出
    ±100 Å 的批數 {np.sum(abs(e) > 100)}/{N}")
```

執行結果：開迴路的平均偏差約 -146 Å，RMSE 約 174 Å，100 批中有 70 批超出 ±100 Å；R2R ($\lambda = 0.3$) 的平均偏差約 -11 Å，RMSE 約 33 Å，沒有任何一批超出範圍。 $\lambda = 0.1$ 時追蹤太慢，RMSE 約 47 Å； $\lambda = 1.0$ 時完全相信上一批，RMSE 約 39 Å，受量測雜訊影響。

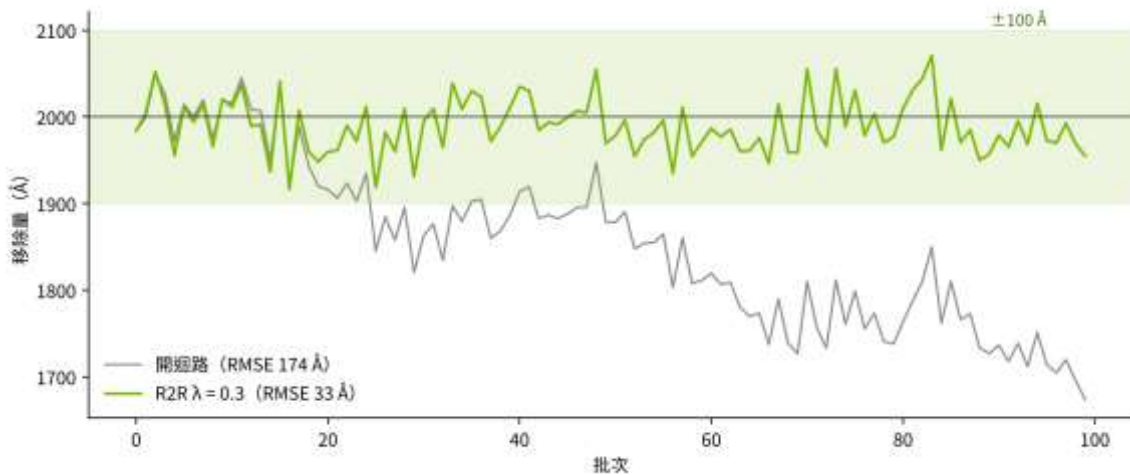


圖 25-3 CMP 的開迴路與 Run-to-Run 控制

R2R 與 SPC 是互補的：R2R 補償可預期的漂移，SPC 監控控制後的殘差是否出現特殊原因。若一味以 R2R 補償，可能把設備的真正問題（例如研磨墊異常磨耗）掩蓋起來，因此控制器的調整量本身也應該納入監控，並設定上下限。第 21 章的虛擬量測可以提供每一片的預測值，讓 R2R 從「逐批」走向「逐片」。

25.4 Smart Quality 與 DFSQ 六階段

25.4.1 從檢驗、管制到預測

品質管理的重心一直在往上游移動：最早是最終檢驗，接著以 SPC 在製程中管制，再以 DOE 與 DFSS 在設計階段預防。智慧製造把這個趨勢再推進一步：感測器與資料平台（第 23 章）讓每一件產品都有完整的製程資料，AI 模型可以在量測之前預測品質，控制器可以在不良發生之前調整。這種以資料與模型形成即時閉環的品質管理，常被稱為「智慧品質」（Smart Quality）或「品質 4.0」（Quality 4.0）。

表 25-5 品質管理的四個世代

世代	重點	資料	時間尺度
檢驗	挑出不良品	抽樣或全檢結果	事後
管制	偵測製程的特殊原因	抽樣的關鍵特性	每小時至每班
預防	在設計與開發時就控制變異	實驗與歷史資料	專案

世代	重點	資料	時間尺度
預測與閉環	預測每一件的品質並即時調整	全數的製程與品質資料	每件、每批，即時

25.4.2 DFSQ 六階段

為了把前面的方法串成一套可以落地的流程，本書整理出「智慧品質設計」(Design for Smart Quality, DFSQ) 的六個階段。它借用 DMAIC 與 DFSS 的精神，並納入資料與 AI 的閉環；這是本書為教學整理的架構，並非國際標準，各企業可依自己的成熟度調整。學術界也有類似的提法，例如以「製程監控用於品質」(process monitoring for quality) 為核心的多步驟框架。

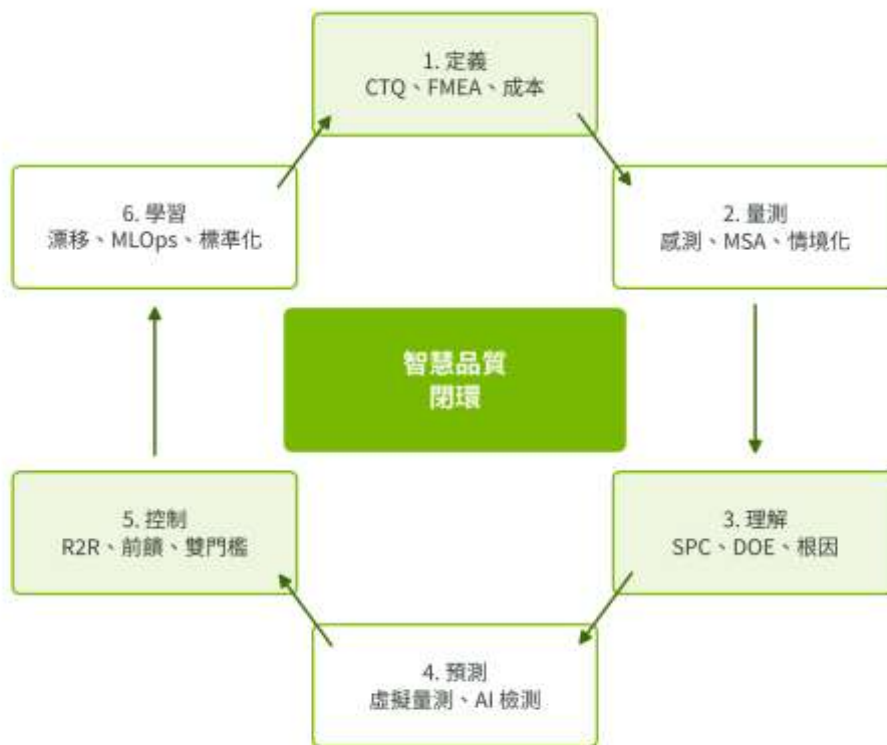


圖 25-4 DFSQ 六階段：從定義品質到持續學習的閉環

表 25-6 DFSQ 六階段

階段	主要問題	方法與工具	產出
1. 定義	顧客在意的品質特性是什麼？不良的代價多大？	VOC、CTQ、FMEA、成本分析	關鍵品質特性與目標

階段	主要問題	方法與工具	產出
2. 量測	量得到嗎？量得準嗎？資料連得起來嗎？	感測器規劃、MSA、資料情境化	可信、可追溯的資料
3. 理解	哪些製程參數影響品質？有交互作用嗎？	SPC、DOE、相關與根因分析	製程與品質的關係模型
4. 預測	能在量測之前知道品質嗎？	虛擬量測、AI 品質預測、視覺檢測	每件的品質預測與信心
5. 控制	預測不好時怎麼調整？誰批准？	R2R、前饋與回饋、雙門檻、人工覆核	閉環控制與處置規則
6. 學習	模型與控制還有效嗎？經驗如何保存？	漂移監測、MLOps、標準化、知識管理	持續更新的模型與標準

這六個階段的重點在於「閉環」：第 6 階段的發現會回到第 1 至第 5 階段，例如新的缺陷類型會改變 CTQ 的定義、量測需求與模型。許多導入失敗的原因，是只做了第 4 階段——建了一個預測模型，卻沒有可信的量測（第 2 階段）、沒有處置規則（第 5 階段），也沒有上線後的監控（第 6 階段）。

25.4.3 前饋、回饋與品質的閉環

品質閉環有兩種基本形式。回饋控制依本批或前幾批的結果調整下一批，例如 25.3.3 節的 R2R；前饋控制則依上游的量測，調整下游的參數，例如依前一站量到的膜厚，調整本站的研磨時間；依進料的硬度，調整熱處理的溫度。兩者常結合使用：前饋補償已知的上游變異，回饋修正模型的誤差與漂移。

閉環的對象也不只是機台參數。品質預測可以決定一件產品是否需要抽檢、是否送去複檢（第 24 章）、是否降級使用；也可以觸發保養工單（第 26 章），或回饋到產品設計（第 20 章的數位主線）。不論哪一種閉環，都要清楚定義：誰有權限自動執行、什麼情況需要人工批准、異常時如何回到安全的設定。

25.5 良率分析、製程內檢測與半導體、電子組裝案例

25.5.1 良率與良率模型

良率是半導體與電子產業最重要的指標之一。一片晶圓上有數百到數千顆晶粒，良率決定了每顆晶粒的成本。最簡單的良率模型假設缺陷隨機、均勻地落在晶圓上，晶粒良率服從 Poisson 分布：

$$Y = e^{(-A \cdot D_0)}$$

其中 A 是晶粒面積， D_0 是缺陷密度。但實際的缺陷往往會群聚：某些晶圓、某些區域特別多，其他地方很少。群聚使更多缺陷集中在同一顆已經不良的晶粒上，良率反而比 Poisson 模型的預測高。Stapper 等人以負二項模型描述這種群聚：

$$Y = (1 + A \cdot D_0 \div \alpha)^{-\alpha}$$

α 是群聚參數， α 越小越群聚； α 趨近無限大時，負二項模型會回到 Poisson 模型。

25.5.2 晶圓圖與空間型態

除了良率的數字，晶圓圖的空間型態往往直接指向原因：中心的團塊可能與化學機械研磨或旋塗有關；邊緣環常與蝕刻、薄膜或晶邊處理有關；刮痕型態指向搬運；規則的重複圖形則可能來自光罩或曝光。公開的 WM-811K 晶圓圖資料集，常被用來研究晶圓圖型態的自動分類。

下面的程式模擬 300 片晶圓、每片 648 顆晶粒：缺陷密度在晶圓之間與晶圓之內都有變化（群聚），另有約 10% 的晶圓有邊緣的製程問題。程式比較實際良率與兩種模型的預測，並以「邊緣不良率減去內圈不良率」找出邊緣環型態的晶圓：

```
import numpy as np
# 良率分析：缺陷群聚使 Poisson 模型低估良率；負二項模型較貼近實際。再以徑向良率找出「邊緣環」型態的晶圓
rng = np.random.default_rng(5)
yy, xx = np.mgrid[-15:15, -15:15] + 0.5; r = np.hypot(xx, yy); die = r < 14.5 # 每格一個晶粒，約 660 顆
A = 0.25 # 晶粒面積 (cm²)
D0, alpha = 2.0, 1.5 # 平均缺陷密度 (個/cm²)、群聚參數 (越小越群聚)
maps, edge = [], []
for w in range(300):
    D = rng.gamma(alpha, D0 / alpha) # 各晶圓 (與區域) 的缺陷密度不同 → 群聚
    lam = np.where(die, D * A * rng.gamma(4, 1/4, die.shape), 0) # 晶圓內的區域性變化
    bad = rng.poisson(lam) > 0
    e = rng.random() < 0.1 # 10% 的晶圓有邊緣製程問題
```

題（如蝕刻不均）

```
if e: bad |= (r > 12.5) & die & (rng.random(die.shape) < 0.7)
maps.append(bad & die); edge.append(e)
maps, edge = np.array(maps), np.array(edge)
Y = 1 - maps.sum((1, 2)) / die.sum()
ring = (r > 12.5) & die; inner = (r < 10) & die
gap = maps[:, ring].mean(1) - maps[:, inner].mean(1) # 邊緣不良率 - 內圈不良率
flag = gap > 0.25
norm = ~edge
print(f"晶粒數 {die.sum()}; 平均良率 {Y.mean():.1%} (無邊緣問題的晶圓 {Y[norm].mean():.1%})")
print(f"以平均缺陷密度 {D0} 個/cm2 預測: Poisson 模型 {np.exp(-A*D0):.1%}, 負二項模型 ( $\alpha =$  {alpha}) {(1 + A*D0/alpha)**-alpha:.1%}")
print(f"邊緣環偵測: 實際 {edge.sum()} 片, 標記 {flag.sum()} 片, 其中正確 {np.sum(flag & edge)} 片")
miss = edge & ~flag; print(f"漏掉的 {miss.sum()} 片, 平均良率 {Y[miss].mean():.1%} (偵測到的 {Y[flag & edge].mean():.1%})")
```

執行結果：平均良率約 64.1%，沒有邊緣問題的晶圓約 65.0%。以平均缺陷密度 2.0 個/cm² 預測，Poisson 模型只有約 60.7%，低估了良率；負二項模型 ($\alpha = 1.5$) 約 65.0%，與沒有邊緣問題的晶圓一致。邊緣環的偵測：實際 31 片，標記 29 片，全部正確；漏掉的 2 片平均良率只有約 25%，整片都是缺陷，邊緣的型態被掩蓋了。

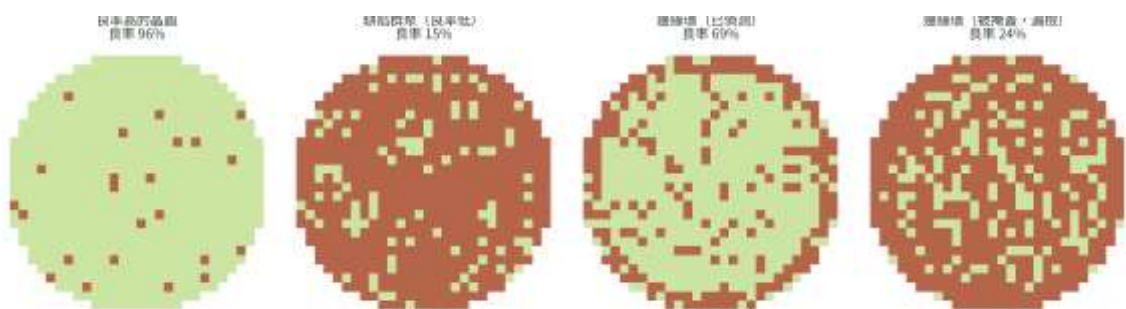


圖 25-5 模擬的晶圓圖：隨機、群聚與邊緣環

漏掉的晶圓提醒我們：簡單的統計規則在型態不明顯時會失效；實務上會以 CNN 等模型分類晶圓圖（第 24 章），並把多種型態的混合納入考慮。更重要的是，晶圓圖要和製程履歷連結：同一型態的晶圓，是否都經過同一台機台、同一個腔體、同一批化學品？這就是第 23 章的資料情境化與根因分析。

25.5.3 製程內檢測與半導體品質

半導體製造有數百道步驟，等到最後的電性測試才發現問題，損失已經非常大。因此晶圓廠在製程中安排大量的量測與檢測：膜厚、線寬、疊對誤差等量測，以及以光學或電子束掃描的缺陷檢測。

缺陷檢測找到的缺陷，再以掃描式電子顯微鏡複檢與分類。這些資料與設備的感測資料、R2R 控制、虛擬量測（第 21 章）、晶圓圖與最終測試資料串連，構成良率管理系統。

表 25-7 半導體與電子組裝的製程內檢測

產業	檢測站	檢測內容	閉環的對象
半導體	量測站（膜厚、線寬、疊對）	關鍵尺寸與疊對誤差	R2R 調整曝光、蝕刻、研磨
半導體	缺陷檢測與複檢	微粒、圖形缺陷	設備保養、製程調整
半導體	晶圓允收測試與晶圓測試	電性參數、晶粒良否	良率分析、製程回饋
電子組裝	錫膏檢測（SPI）	錫膏體積、面積、高度、偏移	回饋印刷機（壓力、速度、偏移、清潔）
電子組裝	迴焊前後的自動光學檢測（AOI）	缺件、偏移、極性、焊點	回饋置件機與迴焊爐
電子組裝	自動 X 光檢測（AXI）	BGA 等隱藏焊點的空洞與短路	迴焊曲線、錫膏與鋼板設計
電子組裝	電路測試與功能測試	電性與功能	追溯到料號、批號與站點

25.5.4 電子組裝的閉環

表面黏著技術（SMT）產線中，業界常引用的經驗是：相當大比例的焊接缺陷源自錫膏印刷。因此錫膏檢測（SPI）是閉環品質的關鍵：SPI 量測每一個焊墊的錫膏體積與偏移，若整片板子有系統性的偏移，就回饋印刷機修正對位；若體積逐漸不足，就提示清潔鋼板。SPI、AOI 與印刷機、置件機之間的資料交換，近年多以 IPC-CFX（設備與系統之間的資料交換）進行，並以 IPC-HERMES-9852 規範相鄰設備之間的電路板交接與板件資訊傳遞（第 11 章），讓不同廠牌的設備可以形成閉環。

這也是第 24 章的視覺檢測與本章 SPC、R2R 的交會點：AOI 的判定結果可以用 p 管制圖監控；SPI 的錫膏體積可以用 $\bar{X}$ 管制圖與 EWMA 監控；印刷機的偏移修正則是一種 R2R 控制。當 AOI 的誤判太多時，作業員會習慣性地按「通過」，閉環就失效了——檢測的可信度與量測系統分析一樣，是閉環的前提。

案例

案例 25-1 教學情境：三年沒更新的管制圖

這是本章開頭的情境。管制界限三年未更新，量具的 %GR&R 約五成，管制圖看到的大多是量具雜訊。改善量測後重新建立管制界限，刀具磨耗的偏移立即被型態規則抓到，並由系統建議補償量。

啟示：先確認量測可信，再談管制；管制圖要有人負責、能閉環。

案例 25-2 產業：Run-to-Run 控制在晶圓廠

依公開文獻與報導，R2R 控制在 1990 年代起被晶圓廠廣泛採用，應用於微影、蝕刻、CMP 與薄膜等製程，並與設備故障偵測與分類（FDC）、虛擬量測整合為先進製程控制（APC）系統。

啟示：當製程無法即時量測時，逐批的回饋與前饋控制是品質閉環的主力。

案例 25-3 產業：以 DOE 取代試誤

一家塑膠射出廠過去以老師傅的經驗試模，每個新模具要試十幾次。導入 DOE 後，先以部分因子實驗篩選出料溫、射出速度與保壓壓力三個重要因子，再以反應曲面找出最佳條件，試模次數與時間都明顯減少，且留下可以傳承的模型。

啟示：DOE 把經驗變成可驗證、可傳承的知識。

案例 25-4 產業：SPI 與印刷機的閉環

依設備廠商的公開資料，SPI 可以把量到的錫膏偏移回饋給印刷機，自動修正對位，並依體積趨勢提示鋼板清潔。導入時要先確認 SPI 的量測重複性，否則回饋會把雜訊放大。

啟示：自動閉環之前，要先確認量測與控制模型可信，並設定調整上限。

案例 25-5 台灣情境：從紙本檢驗到 eQMS

一家中部的精密零件廠，原本以紙本記錄檢驗結果，月底才彙整 Cpk 報表。導入電子檢驗紀錄與量具連線後，每一筆量測即時上傳，管制圖自動更新、異常即時通知；客戶稽核時，也能依批號追溯到原料、機台與量測員。

啟示：數位化的第一個效益往往是即時與可追溯，而不是 AI。

案例 25-6 教學情境：補償掩蓋了問題

某產線的 R2R 控制器運作良好，產品都在規格內。但工程師發現，控制器的補償量在三週內持續增加，接近上限；追查後發現是設備的某個零件異常磨耗。若只看產品品質，問題要到補償用盡、產品超規時才會被發現。

啟示：除了監控產品，也要監控控制器的調整量；閉環會掩蓋問題，也能提早揭露問題。

實作活動

活動 25-1 管制圖與規則

以 25.2 節的程式，改變偏移的大小與發生時間，比較 Shewhart 圖加上西方電氣規則與 EWMA 圖的偵測時間與誤報次數。

活動 25-2 量具 GR&R

以游標卡尺與測微器，由三位同學各量 10 個零件 3 次，以 ANOVA 法計算 %GR&R 與可區分類別數，並與 25.2.4 節的程式結果比較。

活動 25-3 紙飛機 DOE

以紙飛機的機翼長度、紙張種類、迴紋針數量為因子，進行 2^3 全因子實驗，分析飛行距離的主效應與交互作用，並說明一次一因子實驗可能錯過什麼。

活動 25-4 R2R 控制器調參

以 25.3.3 節的程式，加入量測延遲（量測結果晚一批才回來）與換墊造成的階躍變化，重新尋找最佳的 λ ，並設計控制器調整量的監控規則。

活動 25-5 DFSQ 規劃

選一個熟悉的產品（例如手機外殼、PCB 或飲料瓶），依 DFSQ 六階段寫出關鍵品質特性、量測方式、可能的影響因子、預測與控制的做法，以及上線後的監控。

活動 25-6 晶圓圖分類

以公開的晶圓圖資料集，先以 25.5 節的徑向統計方法分類邊緣環，再訓練簡單的 CNN，比較兩者對各種型態的召回率。

延伸閱讀

- Douglas C. Montgomery，Introduction to Statistical Quality Control（Wiley）：管制圖、製程能力與 MSA 的標準教材。
- Douglas C. Montgomery，Design and Analysis of Experiments（Wiley）：實驗設計與反應曲面法。
- George E. P. Box 與 K. B. Wilson，〈On the Experimental Attainment of Optimum Conditions〉，Journal of the Royal Statistical Society, Series B，1951。
- AIAG，Measurement Systems Analysis（MSA）Reference Manual：汽車業量測系統分析的參考手冊。
- James Moyne、Enrique del Castillo 與 Arnon M. Hurwitz 編，Run-to-Run Control in Semiconductor Manufacturing（CRC Press，2001）。

表 25-8 本章思考地圖

本章概念	關鍵提問	後續延伸閱讀
品質管理體系	品質靠檢驗、管制、預防還是預測？	第 8、17 章

本章概念	關鍵提問	後續延伸章節
SPC 與能力	管制界限是製程的聲音還是規格？偏移有多小？	第 23 章
MSA	量具的變異佔多少？量測可信嗎？	第 24 章
DOE 與 RSM	有交互作用嗎？最佳點有確認實驗嗎？	第 21、26 章
R2R 與前饋	補償量有監控嗎？有上限嗎？	第 21、26 章
DFSQ 閉環	六個階段哪一個最弱？誰批准自動調整？	第 28、36 章
良率與晶圓圖	缺陷是隨機還是群聚？型態指向哪個製程？	第 27、28 章

章末小結

- 品質管理從最終檢驗演進到統計管制、設計預防與預測閉環。TQM 提供全員參與與持續改善的文化，精實強調內建品質與消除浪費，六標準差以 DMAIC 與統計方法降低變異，QMS 以文件化流程確保一致與可追溯，數位化後成為 eQMS。
- SPC 區分共同原因與特殊原因。範例中 X-R 管制圖的 C_{pk} 約 1.44； 0.75σ 的偏移在 8 組後由「連續 8 點同側」規則抓到。偏移 0.5σ 時 EWMA 的 ARL 約 8.2 組，遠快於 Shewhart 的 33.5 組；大偏移則以 Shewhart 較快。
- 量測系統要先可信：範例中游標卡尺的 %GR&R 約 49%、可區分類別數 2，改用測微器與治具後降到約 17%、可區分類別數 8。
- DOE 以較少的實驗估計主效應與交互作用，反應曲面法以二次模型找出最佳條件；範例中預測的最佳點 ($A \approx 1.16$ 、 $B \approx 1.03$ ，強度約 54.0 MPa) 接近真實最佳點，但落在試驗範圍外緣，需要確認實驗。R2R 以 EWMA 補償 CMP 的漂移，RMSE 從約 174 Å 降到約 33 Å，超規批數從 70 降到 0。
- 智慧品質以資料與模型形成閉環；本書整理的 DFSQ 六階段為定義、量測、理解、預測、控制與學習。良率分析中，缺陷群聚使 Poisson 模型低估良率（60.7% 對實際約 65%），負二項模型較準確；晶圓圖型態與製程履歷結合，可指向問題來源。電子組裝以 SPI、AOI、AXI 與印刷機、置件機形成閉環。

習題

一、選擇題

1. 1920 年代提出管制圖的是？(A) Deming (B) Shewhart (C) Juran (D) Taguchi
2. 六標準差改善專案常用的流程是？(A) PDCA (B) DMAIC (C) 5S (D) SMED
3. 只受隨機、微小因素影響的變異稱為？(A) 特殊原因 (B) 共同原因 (C) 量測偏差 (D) 系統誤差
4. 同時考慮變異與平均值偏離中心的指標是？(A) Cp (B) Cpk (C) R⁻ (D) ARL
5. 對小而緩慢的偏移較敏感的管制圖是？(A) X⁻-R (B) p (C) EWMA (D) c
6. 每片 PCB 的焊點缺點數適合用哪一種管制圖？(A) X⁻-R (B) I-MR (C) c 或 u (D) X⁻-S
7. 依汽車業常用準則，%GR&R 低於多少通常可接受？(A) 10% (B) 30% (C) 50% (D) 5%
8. 配適二次反應曲面的標準設計是？(A) Plackett-Burman (B) 中心複合設計 (C) 一次一因子 (D) 2^{k-p}
9. 依批後量測調整下一批配方的控制方式是？(A) PID (B) R2R (C) 開迴路 (D) 手動
10. 在 SMT 產線中，量測錫膏體積與偏移的設備是？(A) AOI (B) SPI (C) AXI (D) ICT

二、簡答題

11. 說明共同原因與特殊原因的差異，以及對共同原因過度反應的後果。
12. 為什麼管制界限與規格界限不可混淆？
13. 比較 Shewhart 管制圖與 EWMA 管制圖的優缺點。
14. 為什麼在計算 Cpk 之前要先做量測系統分析？
15. 說明一次一因子實驗的缺點，以及 DOE 如何改善。
16. 說明 R2R 控制中 λ 的取捨。
17. 為什麼缺陷群聚時，Poisson 模型會低估良率？

18. 說明 DFSQ 六階段，並指出最常被忽略的階段。

三、計算題

19. 規格 10.00 ± 0.08 mm，製程平均 10.02 mm， $\sigma = 0.02$ mm，求 C_p 與 C_{pk} 。

20. 子群大小 5， $\bar{\bar{X}} = 50.0$ ， $R = 2.0$ ， $A_2 = 0.577$ ，求 $\bar{X}$ 圖的管制界限。

21. 接上題， $d_2 = 2.326$ ，估計製程的 σ 。

22. 以 $\pm 3\sigma$ 管制界限，製程未偏移時，平均每幾點會出現一次超出界限的誤報？

23. 2^4 全因子實驗需要幾次實驗？若採 2^{4-1} 部分因子呢？

24. 某 2^2 實驗的結果： $(-, -)$ 20、 $(+, -)$ 30、 $(-, +)$ 24、 $(+, +)$ 40，求 A、B 的主效應與 AB 交互作用。

25. 晶粒面積 0.25 cm^2 ，缺陷密度 2.0 個/ cm^2 ，分別以 Poisson 模型與 $\alpha = 1.5$ 的負二項模型計算良率。

26. R2R 的 EWMA 估計值為 38.0，本批觀測值為 36.0， $\lambda = 0.3$ ，求更新後的估計值。若目標移除量為 2,000 Å，下一批的研磨時間為多少？

四、申論與設計題

27. 為一條 CNC 加工產線設計 SPC 系統：說明量測的選擇、MSA、管制圖類型、規則、警報處置與資料整合。

28. 評論以下說法：「有了 AI 品質預測，就不需要 SPC 與 DOE 了。」

29. 以 DFSQ 六階段，為 SMT 產線的焊接品質設計一個閉環品質系統。

五、是非題

30. 所有點都在管制界限內，就表示製程沒有特殊原因。

31. C_{pk} 不可能大於 C_p 。

32. %GR&R 為 45% 的量具，適合用來判斷製程能力。

33. 只有兩個水準的 2^k 實驗無法偵測曲率。

34. R2R 控制的 λ 越大越好，因為反應越快。

35. 晶圓圖的空間型態可以幫助找出問題的製程站點。

六、進階挑戰題

36. 實作 CUSUM 管制圖，與本章的 EWMA 比較不同偏移大小下的 ARL，並說明參數 k 、 h 的選擇。

37. 以本章的射出成形模擬，改用 Box-Behnken 設計與田口直交表，比較找到的最佳點、實驗次數與穩健性。

38. 建立一個結合虛擬量測（第 21 章）與 R2R 的逐片控制模擬，比較逐批與逐片控制的效果，並分析虛擬量測誤差的影響。

七、補充題

39. 列出 DFSQ 的六個階段。

40. 列出表 25-7 中電子組裝的四種製程內檢測。

提示與參考解答

1. B。

2. B。

3. B。

4. B。

5. C。

6. C。

7. A。

8. B。

9. B。

10. B。

11. 參考：共同原因是製程固有、許多微小隨機因素造成的變異；特殊原因是可找出的原因。對共同原因過度反應（隨機變動就調機）會增加變異。
12. 參考：管制界限由製程資料計算，代表製程本身的變異；規格界限由顧客或設計決定。製程可能在管制內卻不符規格，也可能符合規格卻不在管制內。
13. 參考：Shewhart 圖簡單直觀，對大偏移反應快；EWMA 累積過去資訊，對小而緩慢的偏移較敏感，但對突發大偏移稍慢，也較不直觀。
14. 參考：量測變異會疊加在製程變異上，使估計的 σ 偏大、Cpk 偏低，管制圖也被雜訊主導；量測不可信時，結論都不可信。
15. 參考：一次一因子看不到交互作用，可能停在錯誤的條件，實驗次數也多。DOE 同時改變多個因子，以較少實驗估計主效應與交互作用。
16. 參考： λ 大則對新資訊反應快、追蹤漂移好，但容易被量測雜訊影響； λ 小則平滑，但追蹤慢。應依漂移速度與雜訊大小選擇。
17. 參考：群聚使多個缺陷集中在少數晶粒上，同一晶粒的第二、第三個缺陷不再造成額外的不良，因此不良晶粒數比均勻分布時少。
18. 參考：定義、量測、理解、預測、控制、學習。最常被忽略的是量測（MSA 與資料連結）、控制（處置規則）與學習（上線後監控）。
19. $C_p = 0.16 \div 0.12 \approx 1.33$; $C_{pk} = \min (10.08 - 10.02 , 10.02 - 9.92) \div 0.06 = 0.06 \div 0.06 = 1.00$ 。
20. $UCL = 50.0 + 0.577 \times 2.0 \approx 51.15$; $LCL \approx 48.85$ 。
21. $\sigma \approx 2.0 \div 2.326 \approx 0.86$ 。
22. 超出 $\pm 3\sigma$ 的機率約 0.27%，平均約每 370 點一次。
23. $2^4 = 16$ 次 ; $2^{4-1} = 8$ 次。
24. $A = (30 + 40) \div 2 - (20 + 24) \div 2 = 35 - 22 = 13$; $B = (24 + 40) \div 2 - (20 + 30) \div 2 = 32 - 25 = 7$; $AB = (20 + 40) \div 2 - (30 + 24) \div 2 = 30 - 27 = 3$ 。

25. $A \cdot D_0 = 0.5$ 。Poisson： $e^{(-0.5)} \approx 60.7\%$ ；負二項： $(1 + 0.5 \div 1.5)^{(-1.5)} = (4/3)^{(-1.5)} \approx 65.0\%$ 。
26. $0.3 \times 36.0 + 0.7 \times 38.0 = 37.4 \text{ Å/s}$ ；時間 = $2,000 \div 37.4 \approx 53.5$ 秒。
27. 參考：選擇關鍵尺寸並以自動量測上傳；先做 GR&R；依子群選擇 X-R 或 I-MR，搭配 EWMA 監控刀具磨耗；定義規則與警報的處置流程 (OCAP)；與 MES 連結刀具、工單與量測資料。
28. 參考：不同意。AI 預測需要可信的量測、穩定的製程與理解因果的實驗；SPC 監控預測與控制後的殘差，DOE 提供因果知識與模型的基礎。三者是互補的。
29. 參考：定義焊接缺陷的 CTQ；以 SPI、AOI、AXI 量測並做 MSA；以 DOE 研究印刷參數；以模型預測焊接品質；SPI 回饋印刷機、AOI 回饋置件機，設定調整上限與人工覆核；監控誤判率、控制器調整量與模型漂移。
30. 否，還要檢查非隨機的型態，例如連續 8 點同側。
31. 是， $C_{pk} \leq C_p$ ，平均值在規格中心時兩者相等。
32. 否，量測變異過大，應先改善量測系統。
33. 是，需加入中心點或改用三水準設計。
34. 否， λ 太大容易被量測雜訊影響。
35. 是。
36. 提示： k 常取欲偵測偏移的一半， h 依誤報 ARL 選擇；以相同誤報 ARL 公平比較。
37. 提示：比較實驗次數、預測最佳點與真實最佳點的距離，以及在雜訊因子變動下的表現。
38. 提示：以虛擬量測預測每片的移除量，並加入預測誤差；比較誤差大小對逐片控制效益的影響。
39. 定義、量測、理解、預測、控制、學習。
40. SPI、AOI、AXI、電路與功能測試。

第 26 章

預測維護、遠程運維與製程最佳化

編著 李世淵 (機器人叫獸) | 助教 李天宇、李宇晴

本章重點

- 可靠度、可用度與韋伯壽命分析；維護策略的演進與 RCM
- 狀態監測、故障診斷與剩餘壽命預測：以模擬比較三種維護策略
- 物理模型、資料模型與混合模型的外推，以及不確定性量化
- 遠程運維的架構、資安與服務模式
- MPC、製程最佳化與強化學習的閉環決策及其治理

叫獸開講：壞了才修，還是還沒壞就換？

有一家做鋁擠型的工廠，主軸馬達的軸承每年總會壞個兩、三次。每壞一次，整條線停八個小時，還要緊急叫修、空運零件。廠長受不了，下令「每半年全部換新」。結果一年下來，軸承的費用多了好幾倍，故障卻沒有少多少——有些軸承才裝上去兩個月就壞了，有些換下來的看起來還好好的。

我請維修課把過去五年的故障紀錄整理出來。軸承的壽命分布很分散：有的撐一年多，有的不到三個月。這種情況下，「定期全部換」很浪費，因為大部分換下來的還能用很久；但也擋不住那些提早壞掉的。真正需要的，是知道「這一顆」軸承現在的狀態。

我們在每台馬達上裝了振動感測器，用第 23 章的包絡分析監看軸承的缺陷頻率，再以趨勢外推它還能撐多久。當預測的剩餘壽命接近備料與排程需要的時間，系統就開立保養工單，排在下一次換模的時候一起換。半年後，非計畫停機只剩一次，軸承的用量也比「定期全換」少了一大半。

廠長後來問我：「那供應商說他們可以遠端幫我們看，要不要買？」我說：「看他們能不能告訴你，哪些問題可以遠端解決、哪些要到場、到場時帶什麼零件。遠程運維的價值，不是多一個儀表板，而是少跑幾趟、一次修好。」

這一章，我們從可靠度、可用度與維護策略的演進談起；介紹狀態監測、故障診斷與剩餘壽命預測；討論物理資訊 AI、混合模型與不確定性；說明遠程運維的系統架構與服務模式；最後以模型預測控制、最佳化與強化學習，談設備與製程的閉環決策。

（說明：本節情境由作者協助企業導入預測維護的經驗綜合改寫，人物與細節已調整。）

學習目標

讀完本章後，你應該能夠：

1. 說明可靠度、可用度與可維護度，以韋伯分布分析含設限資料的故障時間，並求出成本最低的更換週期。
2. 比較事後維護、預防維護、狀態維護與預測維護，並以模擬評估不同策略的成本。

3. 說明狀態監測與故障診斷的流程，並以劣化趨勢預測剩餘壽命及其不確定性。
4. 說明物理模型、資料模型與混合模型的優缺點，理解外推與不確定性量化的重要性。
5. 說明遠程運維的架構、資安與服務模式，並評估其對停機與服務成本的影響。
6. 比較 PID、模型預測控制與強化學習在製程與維護決策上的應用與限制。

核心概念

表 26-1 本章核心名詞中英對照

中文名詞	英文	一句話說明
可靠度	Reliability	在規定條件與時間內完成功能而不故障的機率
可用度	Availability	設備在需要時能正常運作的時間比例
可維護度	Maintainability	故障後能多快、多容易修復
平均故障間隔	Mean Time Between Failures, MTBF	可修復系統兩次故障之間的平均運轉時間
平均修復時間	Mean Time To Repair, MTTR	從故障到恢復運轉的平均時間
韋伯分布	Weibull Distribution	以形狀與尺度參數描述壽命的分布
設限資料	Censored Data	觀察結束時尚未故障、只知道壽命超過某值的資料
狀態維護	Condition-based Maintenance, CBM	依設備的實際狀態決定何時維護
預測維護	Predictive Maintenance, PdM	預測故障時間，在故障前安排維護
剩餘壽命	Remaining Useful Life, RUL	從現在到失效還能運轉的時間
故障預測與健康管理	Prognostics and Health Management, PHM	結合監測、診斷、預測與決策的完整體系
物理資訊神經網路	Physics-informed Neural Network, PINN	以物理方程式約束訓練的神經網路
遠程運維	Remote Monitoring and Maintenance	透過網路監控、診斷並支援遠端設備
模型預測控制	Model Predictive Control, MPC	以模型預測未來、在限制下求最佳控制序列
強化學習	Reinforcement Learning, RL	透過與環境互動、依獎勵學習決策策略

26.1 可靠度、可用度、可維護度與維護策略演進

26.1.1 可靠度的基本量

可靠度 $R(t)$ 是設備運轉到時間 t 仍未故障的機率；故障率（危險率） $h(t)$ 是已經運轉到 t 的設備，在下一瞬間故障的條件機率密度。許多設備的故障率隨時間呈「浴缸曲線」：初期因製造缺陷而較高（早夭期），中期大致穩定（隨機故障期），後期因磨耗而上升（磨耗期）。

可修復系統的穩態可用度常以下式估計：

$$\text{可用度 } A = \text{MTBF} \div (\text{MTBF} + \text{MTTR})$$

提高可用度有兩條路：讓設備少壞（提高 MTBF），或壞了修得快（降低 MTTR）。第 8 章的設備綜合效率（OEE）中的「可用率」，就與這兩者密切相關。

26.1.2 韋伯分布與壽命資料分析

韋伯分布以形狀參數 β 與尺度參數 η 描述壽命： β 小於 1 表示早夭型故障，等於 1 表示隨機故障（指數分布），大於 1 表示磨耗型故障； η 是約 63.2% 的設備會故障的時間。分析壽命資料時要特別注意設限資料：觀察結束時還沒壞的設備，它的壽命不是「觀察時間」，而是「大於觀察時間」。若把它們丟掉或當成故障，都會低估壽命。最大概似估計可以正確地處理設限資料。

下面的程式模擬 40 個軸承的壽命試驗，觀察到 5,000 小時為止；以最大概似法估計韋伯參數，再依「預防更換 2,000 元、故障後更換 30,000 元（含停機損失）」計算成本最低的定期更換週期（年齡更換策略）：

```
import numpy as np
from scipy.stats import weibull_min
from scipy.optimize import minimize, minimize_scalar
# 可靠度：以韋伯分布配適故障時間（含設限資料），再求成本最低的定期更換週期（年齡更換策略）
rng = np.random.default_rng(1)
beta_true, eta_true = 2.5, 4000 # 形狀參數（>1 表示磨耗）、尺度參數（小時）
life = weibull_min.rvs(beta_true, scale=eta_true, size=40, random_state=rng)
cens = np.minimum(life, 5000); failed = life < 5000 # 觀察到 5,000 小時為止，之後的為設限資料
def nll(p):
    b, e = np.exp(p)
    return -(np.sum(weibull_min.logpdf(cens[failed], b, scale=e)) +
```

```

np.sum(weibull_min.logsf(cens[~failed], b, scale=e)))
b, e = np.exp(minimize(nll, [0, 8]).x)
mttf = weibull_min.mean(b, scale=e)
Cp, Cf = 2000, 30000                                # 預防更換成本、故障後更換
                                                    成本（含停機）
def cost_rate(T):                                    # 每小時的期望成本
    R = weibull_min.sf(T, b, scale=e)
    t = np.linspace(0, T, 400); mean_cycle = np.trapezoid(weibull_min.sf(t, b, scale=e),
t)
    return (Cp * R + Cf * (1 - R)) / mean_cycle
Topt = minimize_scalar(cost_rate, bounds=(200, 8000), method="bounded").x
print(f"故障 {failed.sum()} 件、設限 {(~failed).sum()} 件；估計  $\beta = \{b:.2f\}$ ， $\eta = \{e:.0f\}$  小
時，MTTF {mttf:.0f} 小時")
print(f"運轉 2,000 小時的可靠度 {weibull_min.sf(2000, b, scale=e):.1%}")
print(f"最佳更換週期 {Topt:.0f} 小時，成本 {cost_rate(Topt):.2f} 元/小時；壞了才換
{Cf/mttf:.2f} 元/小時")
MTTR = 8; print(f"可用度（壞了才換、MTTR {MTTR} 小時）{mttf/(mttf+MTTR):.2%}")

```

執行結果：33 件故障、7 件設限；估計 β 約 2.65、 η 約 3,977 小時，平均壽命約 3,535 小時；運轉 2,000 小時的可靠度約 85.1%。成本最低的定期更換週期約 1,219 小時，每小時成本約 2.65 元，遠低於「壞了才換」的 8.49 元。若只看可用度，壞了才換仍有 99.77%——可用度很高，不代表成本合理，因為每次故障的代價遠大於停機時數本身。

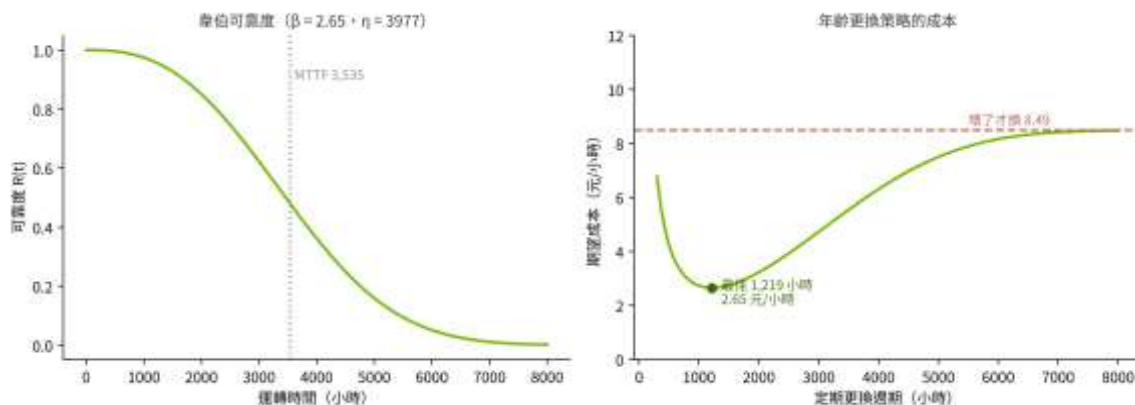


圖 26-1 韋伯可靠度曲線與更換週期的成本曲線

值得注意的是，最佳週期只有平均壽命的三分之一左右：為了避開少數提早故障的軸承，大部分軸承在還能用很久時就被換掉了。這正是本章開頭廠長遇到的困境，也是狀態維護與預測維護要解決的問題。

26.1.3 維護策略的演進

表 26-2 維護策略的演進

策略	決策依據	優點	限制
事後維護 (Run-to-failure)	壞了才修	不需規劃，適合便宜、不重要的設備	非計畫停機、連帶損壞、安全風險
預防維護 (定期)	時間、運轉時數或次數	簡單、可排程	過度維護、擋不住提早故障
狀態維護 (CBM)	量測到的狀態超過門檻	依實際狀態，減少浪費	需要感測與門檻設定
預測維護 (PdM)	預測的剩餘壽命	可配合備料與排程，最大化使用壽命	需要資料、模型與不確定性管理
規範性維護 (Prescriptive)	預測 + 最佳化，建議做什麼、何時做	結合生產排程與成本的整體最佳	需要整合 MES、ERP 與維護系統
以可靠度為中心的維護 (RCM)	依故障模式與後果選擇策略	把資源用在重要的設備與故障模式	需要系統性的分析與跨部門合作

以可靠度為中心的維護 (RCM) 源自航空業，1978 年 Nowlan 與 Heap 的報告指出，許多故障與使用時間並沒有明顯關係，定期翻修不一定能提升可靠度。RCM 的核心問題是：每一種故障模式的後果是什麼？有沒有可偵測的徵兆？若有，就用狀態監測；若無，再考慮定期更換或重新設計。換句話說，預測維護不是取代所有其他策略，而是在「有可偵測的劣化徵兆、故障後果嚴重」的設備上最有價值。

26.2 狀態監測、故障診斷與剩餘壽命預測

26.2.1 狀態監測的流程

ISO 13374 系列 (第 1 部分發布於 2003 年) 等標準把狀態監測的資料處理分成幾個層次：資料擷取、資料處理 (濾波、特徵萃取)、狀態偵測 (與正常基準比較)、健康評估 (診斷故障類型與嚴重度)、預測評估 (剩餘壽命)，以及建議產生 (維護決策)。這與第 23 章的資料平台、第 22 章的機器學習方法一脈相承。

表 26-3 常見的狀態監測技術

技術	監測對象	可偵測的故障
振動分析	旋轉機械	不平衡、對心不良、鬆動、軸承與齒輪缺陷
油液分析	齒輪箱、液壓系統	磨耗顆粒、污染、油品劣化
紅外線熱像	電氣盤、馬達、軸承	接觸不良、過載、潤滑不良
超音波	軸承、閥件、壓縮空氣	早期軸承缺陷、洩漏
馬達電流分析	電動機與負載	轉子斷條、偏心、負載異常
製程與控制器資料	伺服、PLC、CNC	負載上升、循環時間變長、異常停機

在數位化的工廠中，控制器本身就有大量可用的資料：伺服馬達的電流、CNC 主軸的負載、PLC 記錄的循環時間與警報。這些「免費」的資料常常是導入預測維護的第一步，不一定要先加裝昂貴的感測器。

26.2.2 故障診斷

故障診斷回答「是什麼故障、多嚴重」。方法從規則與專家知識（如振動頻譜的故障特徵頻率，第 23 章），到統計與機器學習分類（第 22 章），再到結合兩者的混合方法。實務上的挑戰是：故障樣本很少，而且不同設備、不同工況下的訊號差異很大；因此常先以異常偵測找出「與正常不同」，再由專家確認並逐步累積故障標籤。公開資料集如 NASA 的 C-MAPSS 渦輪發動機模擬資料、各大學的軸承試驗資料，是學習與比較方法的好起點。

26.2.3 剩餘壽命預測

剩餘壽命（RUL）預測的方法大致有三類：以劣化模型（如指數或冪次成長）配適趨勢並外推到失效門檻；以統計模型（如隨機過程、粒子濾波，第 19 章的卡爾曼濾波是其近親）描述劣化的不確定性；以機器學習（如第 22 章的 LSTM、CNN）直接從感測資料學習 RUL。不論哪一種，都要回答兩個問題：預測的不確定性有多大？決策如何使用這個不確定性？

P-F 曲線是常用的概念：從可偵測到潛在故障（P 點）到功能失效（F 點）之間有一段時間，稱為 P-F 間隔。監測的頻率要明顯短於 P-F 間隔，預測的提前量要大於備料、排程與維修所需的前置時

間，否則即使預測準確，也來不及處理。

下面的程式模擬 400 個軸承：每個軸承的缺陷起始時間與劣化速率都不同，振動指標超過 1.0 即故障；每 50 小時量測一次，備料與排程需要 150 小時。比較三種策略：定期更換、狀態門檻（振動超過 0.35 即開工單），以及 RUL 預測（預測剩餘壽命的下界小於前置時間加一個量測間隔時開工單）：

```
import numpy as np
# 剩餘壽命預測：振動指標呈指數成長，超過 1.0 即故障。比較三種維護策略的成本：定期更換、狀態門檻、RUL 預測
rng = np.random.default_rng(2)
dt, lead = 50, 150 # 每 50 小時量測一次；備料與排程需要 150 小時
Cp, Cf = 2000, 30000
def unit():
    t0 = rng.uniform(500, 2500); r = rng.lognormal(np.log(1/700), 0.3) # 缺陷起始時間、劣化速率因零件而異
    return lambda t: 0.1 * np.exp(r * np.maximum(t - t0, 0)) # 劣化指標；1.0 為故障
def fit_rul(ts, ys): # 只用已開始劣化的點 (> 0.12)，配適 log(y) 的直線外推到 1.0；以殘差估計不確定性
    t, y = np.array(ts), np.array(ys); k = y > 0.12; t, y = t[k][-10:], np.log(y[k][-10:])
    if len(t) < 5: return np.inf, np.inf
    A = np.c_[np.ones_like(t), t]; c, *_ = np.linalg.lstsq(A, y, rcond=None)
    if c[1] <= 1e-5: return np.inf, np.inf
    s = np.std(y - A @ c) + 0.02
    rul = (np.log(1.0) - (c[0] + c[1] * t[-1])) / c[1]; low = (np.log(1.0) - 2 * s - (c[0] + c[1] * t[-1])) / c[1]
    return rul, low
def simulate(policy, n=400, T_age=1500, alarm=0.35):
    cost, hours, fails = 0, 0, 0
    for _ in range(n):
        f = unit(); ts, ys = [], []; t = 0
        while True:
            t += dt; y = f(t) * rng.lognormal(0, 0.08) # 量測雜訊
            if f(t) >= 1.0: cost += Cf; fails += 1; hours += t; break
            ts.append(t); ys.append(y)
            if policy == "age" and t >= T_age: cost += Cp; hours += t; break
            trig = (policy == "cbm" and y >= alarm)
            if policy == "rul" and ys[-1] > 0.15: trig = fit_rul(ts, ys)[1] <= lead + dt
            if trig: # 觸發後等待備料與排程，期間持續運轉，可能先壞
                tt = np.arange(t, t + lead + 1, 5); hit = tt[f(tt) >= 1.0]
                if len(hit): cost += Cf; fails += 1; hours += hit[0]
                else: cost += Cp; hours += t + lead
                break
        return cost / hours, fails / n
for name, p in [("定期更換 (1,500 小時)", "age"), ("狀態門檻 (振動 ≥ 0.35)", "cbm"), ("RUL 預測 (下界 ≤ 前置+量測間隔)", "rul")]:
    c, fr = simulate(p); print(f"{name}: 成本 {c:.2f} 元/小時，故障率 {fr:.1%}")
```

執行結果：定期更換（1,500 小時）每小時成本約 1.38 元，故障率約 0.2%；狀態門檻約 0.78 元，故障率 0；RUL 預測約 0.70 元，故障率約 0.5%。

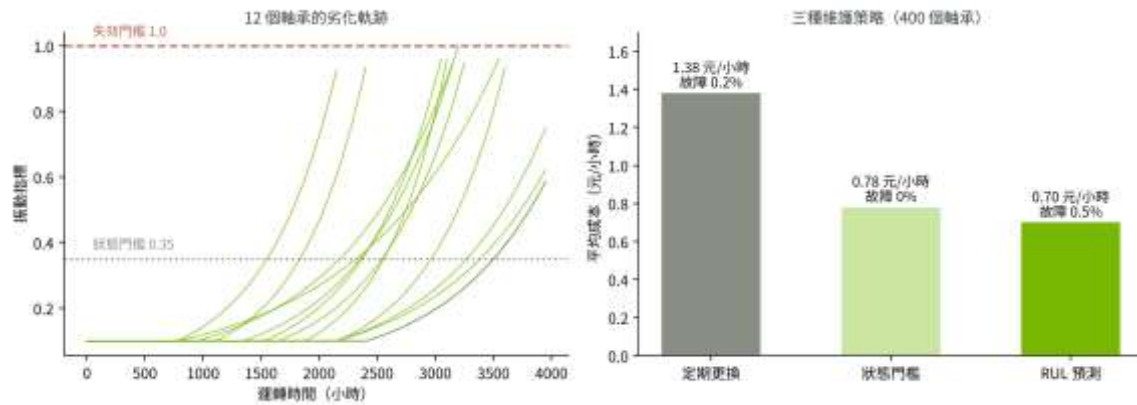


圖 26-2 劣化軌跡、RUL 預測與三種維護策略的成本

狀態維護與預測維護都把成本降到定期更換的一半左右。RUL 預測比固定門檻再省約一成，因為它依每顆軸承自己的劣化速度決定時機，讓劣化慢的軸承用得更久；但它也付出了約 0.5% 的故障率——劣化突然加快或雜訊造成誤判時，預測會來不及。這說明了預測維護的本質：它是在「多用一點壽命」與「承擔一點風險」之間的取舍，決策規則中的不確定性（這裡是預測下界）直接決定了這個平衡。在故障後果嚴重的設備上，應採用更保守的下界，或與狀態門檻並用作為安全網。

26.3 物理資訊 AI、混合模型與不確定性

26.3.1 物理模型與資料模型

物理模型以第一原理或經驗公式描述系統，例如刀具磨耗的 Taylor 公式、軸承壽命的 L10 公式、熱傳與疲勞模型。它的優點是參數有意義、可以外推到沒有資料的條件；缺點是需要專業知識，而且真實系統常有模型沒描述的效應。資料模型（機器學習）不需要知道物理，能擷取複雜的非線性；但它只在訓練資料涵蓋的範圍內可靠，外推時可能完全失準，而且需要大量資料。

表 26-4 物理模型、資料模型與混合模型

類型	做法	優點	限制
物理模型	第一原理、經驗公式	可解釋、可外推、資料需求少	簡化假設、建模成本高
資料模型	機器學習、深度學習	擷取複雜關係、不需領域公式	外推差、需大量資料、難解釋

類型	做法	優點	限制
串聯混合	物理模型的輸出或特徵作為 ML 的輸入	讓 ML 學習較簡單的關係	仍受 ML 外推限制
殘差混合	ML 學習物理模型的誤差	保留物理趨勢、修正系統偏差	殘差模型在外推區仍可能失準
物理資訊學習	以物理方程式作為損失函數或約束	少量資料也能符合物理	訓練較難、需要可微分的方程式

物理資訊神經網路 (PINN) 把偏微分方程式的殘差加入神經網路的損失函數，使預測同時符合資料與物理定律；Raissi、Perdikaris 與 Karniadakis 在 2017 年公開預印本、2019 年發表於 Journal of Computational Physics 的論文，使這個方向廣受注意。在製造中，類似的想法也用於熱變形、切削力、電池老化與結構健康監測。

26.3.2 範例：刀具磨耗的外推

下面的程式模擬刀具磨耗速率，真實關係類似 Taylor 公式，但另有物理模型沒描述的非線性。訓練資料只涵蓋切削速度 80 ~ 180 m/min，測試則分為訓練範圍內與外推 (200 ~ 260 m/min) 兩組；比較隨機森林、物理模型 (取對數後的冪次迴歸) 與殘差混合模型，最後以自助法估計物理模型的參數不確定性：

```
import numpy as np
from sklearn.ensemble import RandomForestRegressor
# 物理資訊與混合模型：刀具磨耗。訓練資料只涵蓋中低切削速度，比較純資料模型、物理模型與混合模型在新條件下的外推能力
rng = np.random.default_rng(3)
def wear_rate(v, f):                                     # 真實磨耗速率
    (μm/min)：類似 Taylor 關係，另有物理模型沒描述的非線性
    return 2e-4 * v**1.9 * f**0.7 * (1 + 0.15 * np.tanh((v - 150) / 30))
def data(n, vlo, vhi):
    v = rng.uniform(vlo, vhi, n); f = rng.uniform(0.1, 0.3, n)
    return np.c_[v, f], wear_rate(v, f) * rng.lognormal(0, 0.08, n)
Xtr, ytr = data(150, 80, 180); Xin, yin = data(200, 80, 180); Xout, yout = data(200, 200, 260)
phys = lambda X, c: np.exp(c[0]) * X[:, 0]**c[1] * X[:, 1]**c[2]
c = np.linalg.lstsq(np.c_[np.ones(len(Xtr)), np.log(Xtr)], np.log(ytr), rcond=None)[0]
# 物理模型：取對數後線性迴歸
rf = RandomForestRegressor(300, min_samples_leaf=3, random_state=0).fit(Xtr, ytr)
res = RandomForestRegressor(300, min_samples_leaf=3, random_state=0).fit(Xtr, np.log(ytr) - np.log(phys(Xtr, c))) # 混合：學物理模型的殘差
hyb = lambda X: phys(X, c) * np.exp(res.predict(X))
def ens(X, B=30):                                       # 以自助法 (bootstrap) 集
```

成估計混合模型的不確定性

```
P = []
for b in range(B):
    i = rng.integers(0, len(Xtr), len(Xtr)); cb =
np.linalg.lstsq(np.c_[np.ones(len(i)), np.log(Xtr[i])], np.log(ytr[i]), rcond=None)[0]
    P.append(phys(X, cb))
return np.array(P)
mape = lambda p, y: np.mean(np.abs(p - y) / y)
print(f"物理模型指數：v^{c[1]:.2f}，f^{c[2]:.2f}（真實 1.9、0.7，另有未建模的非線性）")
for name, f in [("純資料（隨機森林）", rf.predict), ("物理模型", lambda X: phys(X, c)), ("混
合模型", hyb)]:
    print(f"{name}：訓練範圍內誤差 {mape(f(Xin), yin):.1%}，外推（v = 200~260）誤差
{mape(f(Xout), yout):.1%}")
P = ens(Xout); lo, hi = np.percentile(P, [2.5, 97.5], 0)
print(f"只含參數不確定性的 95% 區間，涵蓋外推實測值的比例 {np.mean((yout >= lo) & (yout <=
hi)):.0%}")
s = np.std(np.log(ytr) - np.log(phys(Xtr, c))) # 加入殘差（雜訊與模型誤差）
的不確定性
lo2, hi2 = lo * np.exp(-1.96 * s), hi * np.exp(1.96 * s)
print(f"加入殘差不確定性後，涵蓋比例 {np.mean((yout >= lo2) & (yout <= hi2)):.0%}")
```

執行結果：在訓練範圍內，三種模型的誤差差不多（隨機森林約 10.1%，物理模型約 6.8%，混合模型約 6.6%）；外推時，隨機森林的誤差暴增到約 45.2%，因為決策樹只能輸出訓練過的數值範圍，物理模型約 7.1%，混合模型約 8.5%。

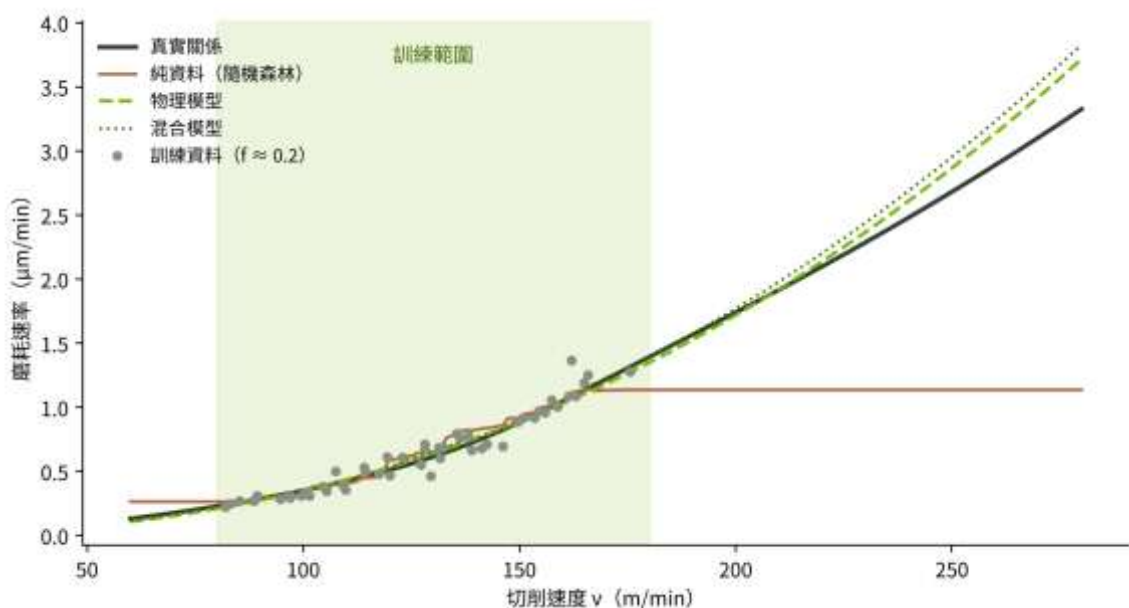


圖 26-3 純資料、物理與混合模型在訓練範圍外的預測

這個結果有兩個值得注意的地方。第一，混合模型在外推時並沒有比物理模型好：殘差模型同樣是隨機森林，在外推區只能沿用邊界的修正量，反而帶進了偏差。混合模型的價值主要在訓練範圍內修正系統偏差；要改善外推，需要更好的物理模型或能合理外推的殘差模型。第二，估計出的速度指數約 2.29，與真實的 1.9 不同——因為物理模型把未建模的非線性也吸收進了參數。參數「有物理意

義」，不代表估計值就是真實的物理常數。

26.3.3 不確定性量化

預測維護的決策需要知道「預測有多可信」。不確定性大致分成兩類：認知不確定性來自資料與模型的不足，可以透過更多資料或更好的模型降低；隨機不確定性來自系統本身的變異與量測雜訊，無法消除。上面的程式顯示：只考慮參數的不確定性，95% 區間只涵蓋約 36% 的外推實測值；加入殘差（雜訊與模型誤差）之後，涵蓋比例約 98%。只報告參數不確定性，會讓人對預測過度自信。

常用的不確定性量化方法包括：貝氏方法與高斯過程、自助法與模型集成、分位數迴歸，以及不需要分布假設的保形預測（conformal prediction）。第 20 章的 VVUQ 強調：模型的可信度必須在「預期使用的範圍」內驗證；外推時，應加大安全裕度，並盡快以新資料確認。

26.4 遠程運維系統架構與服務模式

26.4.1 系統架構

遠程運維讓設備製造商或服務團隊透過網路監控、診斷並支援分散在各地的設備。典型的架構包括：

- 設備端：控制器、感測器與邊緣閘道器，負責資料擷取、前處理與緩衝（第 11 章）。
- 連線：以 OPC UA、MQTT 等協定，經由 VPN 或雲端服務安全地傳輸（第 11 章）。
- 平台：資料湖倉、設備資產模型、數位孿生與分析模型（第 19、23 章）。
- 應用：儀表板、警報、診斷知識庫、遠端協助（如擴增實境的指導）、工單與備品管理。
- 服務流程：分級回應、遠端排除、到場派工、備料、知識回饋與合約管理。

遠程連線也帶來資安風險：遠端存取是工業控制系統常見的入侵途徑。實務上應遵循 IEC 62443 等標準（第 35 章），採用分區與管道、最小權限、多因子驗證、存取紀錄與經客戶核准的連線；許多工廠只允許「設備資料單向送出」，遠端操作則需要現場人員確認。

26.4.2 服務模式

遠程運維改變了設備商與客戶的關係。傳統上，設備商賣設備、賣零件、按次收維修費；有了遠

程監控，設備商可以提供依可用度或產出計價的服務合約，自己承擔維護的風險，也分享效率提升的收益。航空發動機的「按飛行時數計費」是常被引用的例子；在工具機、壓縮機、半導體設備與電梯等產業，也有各種形式的預測維護與遠程服務方案。

表 26-5 遠程運維的服務模式

模式	計價方式	設備商的責任	適用情境
傳統維修	按次、按零件	修好叫修的故障	設備便宜、客戶有維修能力
遠程監控訂閱	每月或每年訂閱	提供監控、警報與診斷建議	客戶自行維修，需要專家支援
保證可用度	依可用度訂定費用與罰則	承擔停機風險，主動維護	關鍵設備、停機代價高
依產出計價	依運轉時數、產量或結果	承擔設備的維護與效能	設備商擁有資料與維護能力

26.4.3 範例：遠程診斷的效益

下面的程式模擬 200 台設備一年的故障處理：傳統做法每次故障都要等工程師到場（24~72 小時），一次修復率 70%；遠程做法先遠端診斷，假設 55% 的問題可以遠端排除，需要到場的則因已確認故障與零件，等待時間較短、一次修復率提高到 90%：

```
import numpy as np
# 遠程運維：200 台設備一年的故障處理。比較傳統到場維修與「遠程診斷+預先備料」的停機時間與服務成本
rng = np.random.default_rng(4)
def year(remote):
    down, cost, visits = 0.0, 0.0, 0
    for m in range(200):
        for _ in range(rng.poisson(3)):
            if remote and rng.random() < 0.55:
                # 每台每年平均故障 3 次
                # 55% 可由遠端排除（參數
                # 調整、軟體、操作指導）
                down += rng.uniform(1, 4); cost += 1500; continue
            wait = rng.uniform(24, 72) if not remote else rng.uniform(12, 48) # 遠程預先
            # 診斷，可排定最近的工程師
            ftf = 0.70 if not remote else 0.90 # 一次修復率：遠程已確認故
            # 障與零件
            fix = rng.uniform(3, 8); down += wait + fix; cost += 12000; visits += 1
            if rng.random() > ftf: down += rng.uniform(24, 72) + fix; cost += 12000;
            visits += 1 # 第二次到場
    return down / 200, cost / 200, visits
for name, r in [("傳統到場維修", False), ("遠程診斷+預先備料", True)]:
    d, c, v = year(r); print(f"{name}: 每台年停機 {d:.0f} 小時，每台年服務成本 {c:,.0f} 元，
    到場次數 {v}")
```

執行結果：傳統到場維修每台每年停機約 218 小時、服務成本約 48,660 元，全年到場 811 次；遠程診斷加預先備料每台每年停機約 55 小時、成本約 18,622 元，到場 271 次。

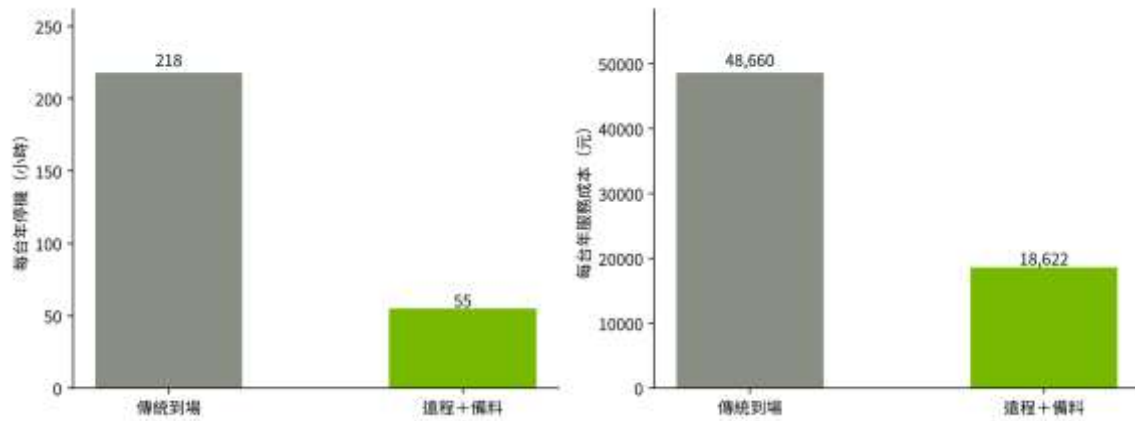


圖 26-4 傳統到場維修與遠程運維的停機與成本

要提醒的是，這個結果高度依賴假設：遠端可排除的比例、一次修復率與等待時間。導入前應以自己的維修紀錄估計這些參數——例如過去的叫修中，有多少是參數設定、軟體或操作問題，這類問題最適合遠端排除。遠程運維的效益主要來自「少跑一趟」與「一次修好」，而不是監控畫面本身。

26.5 MPC、最佳化與強化學習的閉環決策

26.5.1 從 PID 到模型預測控制

PID 控制器簡單、穩健，是工業控制的主力（第 10 章）。但它只看「現在的誤差」，不知道未來，也不直接處理限制：功率有上下限、溫度不可超過安全值、多個變數彼此耦合。模型預測控制（MPC）以製程模型預測未來一段時間的行為，在每個取樣時刻求解一個最佳化問題：在滿足限制的前提下，找出使未來誤差與控制變動最小的控制序列，只執行第一步，下一個時刻再重新計算（滾動時域）。

MPC 在 1970 年代末起源於煉油與石化業，例如 Shell 的動態矩陣控制（DMC）與法國的模型演算法控制，如今廣泛應用於流程工業（第 18 章）、半導體、能源與運動控制。它的代價是需要可信的模型、較大的運算量，以及對模型誤差與擾動的處理。

下面的程式模擬一座加熱爐：一階模型加上 3 步的輸送延遲，加熱功率限制在 0~100%，爐溫不可超過 860 °C；設定點在第 10 步由 800 °C 升到 850 °C，第 70~89 步有控制器不知道的進料降溫擾動。PI 控制器以網格搜尋調參，分別取「誤差最小」與「不超溫中誤差最小」兩組；MPC 則以模型預測 20 步，並以預測誤差估計未知擾動：

```

import numpy as np
from scipy.optimize import minimize
# 模型預測控制 (MPC) 與 PID：加熱爐溫度，加熱功率限制 0~100%，爐溫不可超過 860 °C；設定點由 800
升到 850 °C
a, b, d_ = 0.95, 0.6, 3 # 一階模型 + 3 步輸送延
遲：T[k+1] = a·T[k] + b·u[k-3] + (1-a)·T_amb
Tamb, H = 25.0, 20 # 環境溫度、預測時域
def simulate(ctrl, kp=0.15, ki=0.012, N=120):
    T = 800.0; u0 = (800 - a * 800 - (1 - a) * Tamb) / b; uh = [u0] * (d_ + 1); Ts, us =
    [], []
    integ, dhat, Tpred = 0.0, 0.0, T
    for k in range(N):
        sp = 800 if k < 10 else 850; dist = -4.0 if 70 <= k < 90 else 0.0 # 第 70~89 步
        有進料降溫擾動 (控制器不知道)
        if ctrl == "pid":
            e = sp - T; integ += e; u = u0 + kp * e + ki * integ
        else:
            dhat = 0.7 * dhat + 0.3 * (T - Tpred) # 以預測誤差估計未知擾動
            (偏移估計)
        def J(useq):
            t, h, J = T, list(uh), 0.0
            for i in range(H):
                h.append(useq[i]); t = a * t + b * h[-d_ - 1] + (1 - a) * Tamb + dhat
                J += (t - sp)**2 + 0.05 * (useq[i] - h[-2])**2 + 1e4 * max(t - 860,
0)**2
            return J
        u = minimize(J, np.full(H, uh[-1]), bounds=[(0, 100)] * H, method="L-BFGS-
B").x[0]
        u = float(np.clip(u, 0, 100)); uh.append(u)
        Tpred = a * T + b * uh[-d_ - 1] + (1 - a) * Tamb + (dhat if ctrl == "mpc" else 0)
        T = a * T + b * uh[-d_ - 1] + (1 - a) * Tamb + dist; Ts.append(T); us.append(u)
    return np.array(Ts), np.array(us)
def score(T): return np.sum((T[10:] - 850)**2)
best = min(((kp, ki) for kp in np.linspace(0.05, 0.6, 12) for ki in np.linspace(0.002,
0.04, 12))),
key=lambda p: score(simulate("pid", *p)[0])) # 以網格搜尋調出誤差最小的 PI
參數
safe = min(((kp, ki) for kp in np.linspace(0.05, 0.6, 12) for ki in np.linspace(0.002,
0.04, 12))
            if simulate("pid", kp, ki)[0].max() <= 860), key=lambda p:
score(simulate("pid", *p)[0])) # 加上不超溫的條件
for name, args in [("PI (調到誤差最小)", ("pid", *best)), ("PI (不超溫中最佳)", ("pid",
*safe)), ("MPC", ("mpc",))]:
    T, u = simulate(*args)
    print(f"{name}: 最高爐溫 {T.max():.1f} °C, 誤差平方和 {score(T):.0f}, 擾動期間最大偏差
{np.abs(T[70:100] - 850).max():.1f} °C")
print(f"PI 參數: 誤差最小 kp = {best[0]:.2f}、ki = {best[1]:.3f}; 不超溫 kp =
{safe[0]:.2f}、ki = {safe[1]:.3f}")

```

執行結果：誤差最小的 PI 最高爐溫約 866.1 °C，超過 860 °C 的限制；不超溫中最佳的 PI 誤差平方和約 16,142，擾動期間最大偏差約 20.5 °C；MPC 最高爐溫約 857.1 °C，誤差平方和約 10,004，擾動期間最大偏差約 14.8 °C。

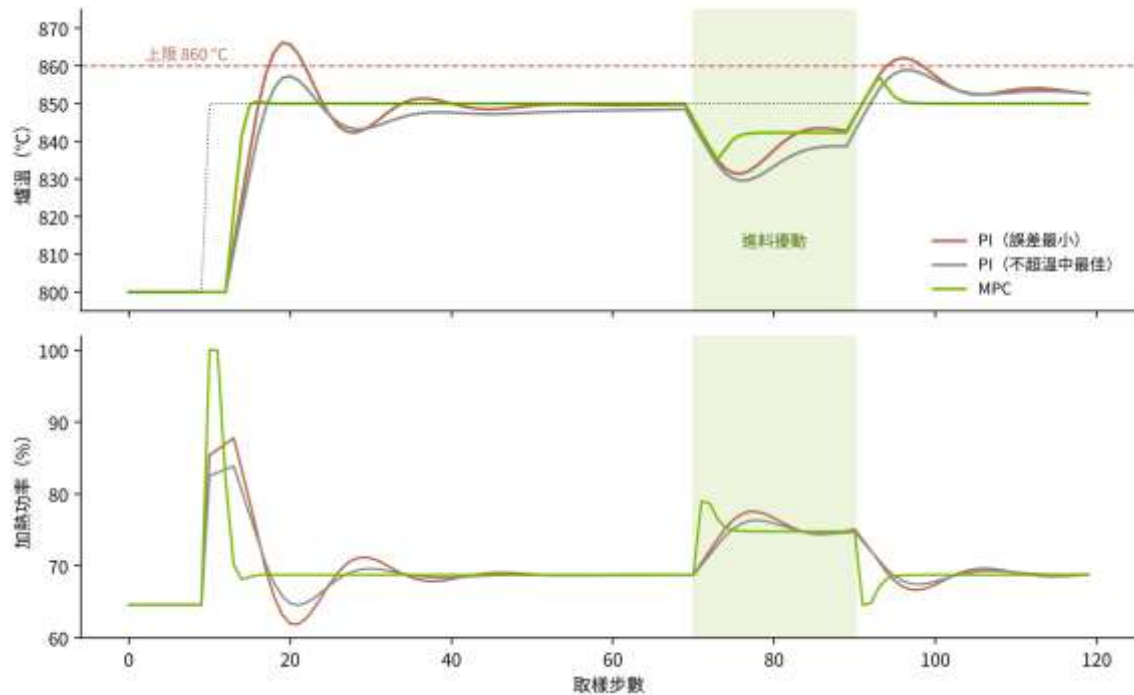


圖 26-5 加熱爐的 PI 與 MPC 控制：設定點變更與擾動

PI 要滿足限制只能放慢反應；MPC 知道延遲與限制，可以在接近 860 °C 之前提早減少功率，因此同時兼顧速度與安全。但這個優勢建立在模型準確的前提下：若真實的延遲或增益與模型差很多，MPC 的表現可能不如一個保守調好的 PI。實務上，MPC 常放在監督層，設定下層 PID 的設定點。

26.5.2 製程最佳化

閉環決策不只在毫秒到秒級的控制迴路，也在更長的時間尺度：即時最佳化 (RTO) 每小時依經濟目標更新設定點；第 25 章的 R2R 逐批調整配方；排程與維護決策以天或週為單位 (第 27 章)。常用的方法包括線性與非線性規劃、混合整數規劃，以及實驗成本高時的貝氏最佳化——以高斯過程描述未知的目標函數，有效率地選擇下一次要試的條件，常用於製程參數與配方的調校。

最佳化的結果要經得起現場的檢驗：目標函數是否反映真正的成本？限制是否完整？模型在最佳點附近是否可信 (第 25 章的確認實驗)？自動執行的權限與人工批准的界線在哪裡？

26.5.3 強化學習

強化學習 (RL) 讓代理人透過與環境互動、依獎勵學習策略，不需要事先知道系統的機率模型。它在棋類與電玩上取得了突破，在工業中則被研究用於製程控制、機器人、排程、能源管理與維護決策。依 DeepMind 與 Google 在 2016 年的公開說明，他們以深度神經網路預測資料中心的能耗與溫

度並建議冷卻設定，使冷卻用電降低約 40%，是常被引用的工業 AI 閉環案例；嚴格來說它以預測模型為主，並非典型的強化學習，但展示了從資料學習控制決策的潛力。

下面的程式把維護決策建成一個馬可夫決策過程：設備有 0~8 的劣化等級，等級 9 為故障；每一期可以選擇運轉（產值 100）或保養（成本 500，回到全新）；故障的代價為維修費與停機損失合計 3,800；等級 5 以上劣化加速。程式比較已知模型時以動態規劃求出的最佳策略、不知道模型時以 Q-learning 學到的策略，以及幾種固定門檻策略：

```
import numpy as np
# 強化學習：以 Q-learning 學習「何時保養」。狀態為劣化等級 0~8，等級 9 為故障；與動態規劃的最佳解及固定門檻策略比較
rng = np.random.default_rng(6)
S, FAIL = 10, 9
P_up = lambda s: [0.6, 0.3, 0.1] if s < 5 else [0.3, 0.4, 0.3] # 劣化增加 0、1、2 級的機率；等級 5 以上加速劣化
def step(s, a):
    if s == FAIL: return 0, -3000 - 8 * 100 # 故障：維修費+停機 8 期的產值損失，回到全新
    if a == 1: return 0, -500 # 保養：成本 500，回到全新
    s2 = min(s + rng.choice(3, p=P_up(s)), FAIL); return s2, 100 # 運轉：產值 100
g = 0.98
def value_iteration(): # 已知機率模型時的最佳策略（動態規劃）
    V = np.zeros(S)
    for _ in range(2000):
        Q = np.zeros((S, 2))
        for s in range(S):
            if s == FAIL: Q[s] = -3800 + g * V[0]; continue
            Q[s, 1] = -500 + g * V[0]
            Q[s, 0] = 100 + g * sum(p * V[min(s + j, FAIL)] for j, p in enumerate(P_up(s)))
        V = Q.max(1)
    return Q.argmax(1)
def q_learning(epochs=60000, alpha=0.05, eps=0.1): # 不知道機率模型，只靠與環境互動學習
    Q = np.zeros((S, 2)); s = 0
    for _ in range(epochs):
        a = rng.integers(2) if rng.random() < eps else int(Q[s].argmax())
        s2, r = step(s, a); Q[s, a] += alpha * (r + g * Q[s2].max() - Q[s, a]); s = s2
    return Q.argmax(1)
def evaluate(policy, T=200000):
    s, tot = 0, 0.0
    for _ in range(T):
        s, r = step(s, policy[s]); tot += r
    return tot / T
opt, ql = value_iteration(), q_learning()
th = lambda k: np.array([1 if i >= k else 0 for i in range(S)])
print("動態規劃最佳策略 (0 運轉、1 保養) :", opt[:FAIL], "; Q-learning :", ql[:FAIL])
for name, pol in [("動態規劃", opt), ("Q-learning", ql), ("壞了才修", th(FAIL)), ("等級 ≥ 3 保養", th(3)), ("等級 ≥ 7 保養", th(7))]:
    print(f"{name} : 平均每期收益 {evaluate(pol):.1f}")
```

執行結果：動態規劃與 Q-learning 得到相同的策略——在劣化等級 7 以上保養，平均每期收益約 55.4~55.6（差異來自模擬的隨機性）；壞了才修約 -151.6，等級 3 就保養約 19.0，而「等級 ≥ 7 保養」的固定門檻與最佳策略一樣。

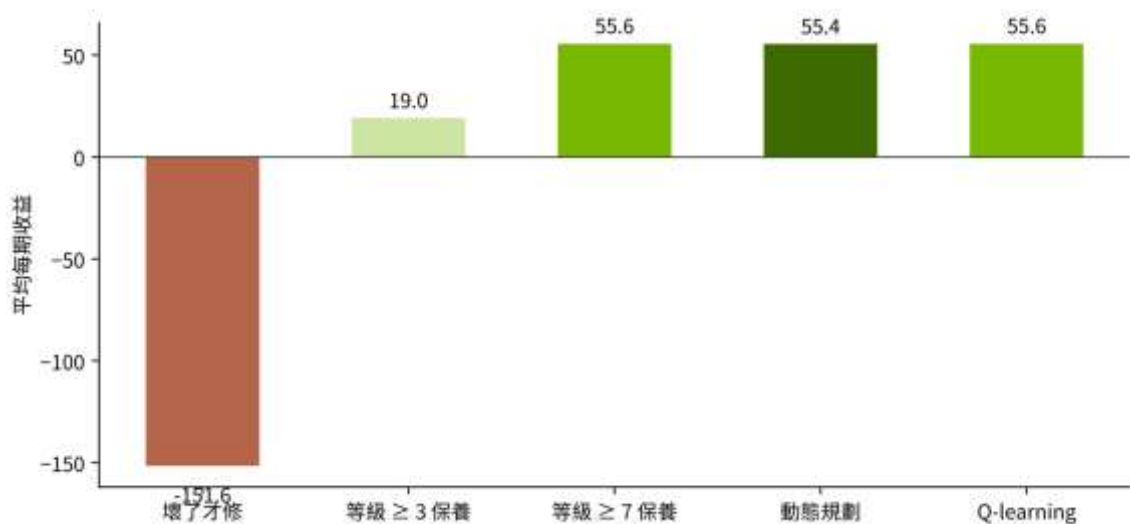


圖 26-6 維護策略的平均每期收益

這個例子說明了強化學習的價值與限制。價值是：即使不知道劣化的機率，只要能與環境互動，就能學到接近最佳的策略。限制是：這裡的 Q-learning 用了 6 萬次互動，相當於讓設備故障了上千次——在真實工廠中不可能這樣學。因此工業上的 RL 通常在模擬器或數位孿生中訓練（第 19、21 章），或從歷史資料離線學習，再經過嚴格的驗證、限制與人工監督才上線。另外，當問題簡單到一個門檻就能表達最佳策略時，可解釋的規則往往比黑箱策略更容易被現場接受。

26.5.4 閉環決策的治理

表 26-6 閉環決策方法的比較

方法	時間尺度	需要的模型	處理限制	主要風險
PID	毫秒~秒	不需要明確模型	間接（飽和、抗積分飽和）	耦合與限制處理不佳
MPC	秒~分鐘	動態模型	直接	模型誤差、運算量
RTO 與數學規劃	小時~天	穩態或經濟模型	直接	模型與現場不符
貝氏最佳化	每次實驗	代理模型（自動建立）	可加入	高維度時效率下降

方法	時間尺度	需要的模型	處理限制	主要風險
強化學習	視問題而定	不一定需要，但常需模擬器	需特別設計	探索的風險、可解釋性、分布改變

不論哪一種方法，閉環決策都需要治理：明確定義自動執行的範圍與上下限；安全相關的功能由獨立的安全系統把關，而不是由最佳化或 AI 決定；變更要經過驗證與核准；異常時能退回安全的手動模式；決策與依據都要留下紀錄（第 36 章）。

案例

案例 26-1 教學情境：從定期全換到預測維護

這是本章開頭的情境。軸承壽命分散，定期全換既浪費又擋不住提早故障。以振動包絡分析監看軸承缺陷頻率，依 RUL 與前置時間開立工單，並與換模時間合併，非計畫停機與軸承用量都明顯下降。

啟示：維護策略要依故障模式與壽命分布選擇；預測要配合備料與排程才有價值。

案例 26-2 產業：按飛行時數計費的發動機服務

依公開資料，航空發動機製造商提供依飛行時數計費的維護合約，透過發動機的狀態監測資料規劃維修，由製造商承擔維護風險。這種模式被視為服務型製造的經典例子（第 32 章）。

啟示：資料讓設備商可以從賣產品轉向賣可用度，但也承擔了可靠度的風險。

案例 26-3 產業：工具機的主軸健康監測

依公開報導，多工具機與控制器廠商提供主軸負載、振動與溫度的監測功能，可在異常時發出警報並保留資料供遠端診斷。導入時，關鍵在於建立每台機器、每種加工的正常基準。

啟示：控制器內建的資料是預測維護低成本的起點。

案例 26-4 產業：MPC 在流程工業

依公開文獻，模型預測控制在煉油、石化與化工業已有數十年的應用，常用於蒸餾塔、反應器與加熱爐，以在品質與安全限制下提高產量或降低能耗。

啟示：MPC 的價值在於同時處理多變數、延遲與限制；模型的維護是長期的工作。

案例 26-5 台灣情境：設備商的遠程服務轉型

一家台灣的自動化設備商，過去依賴工程師出差維修海外客戶的設備。導入遠程連線與設備資料平台後，先處理參數與軟體問題，需要到場時則預先確認故障與零件。公司也開始推出依可用度計價的服務方案，但同時發現要先建立資安規範與客戶的連線核准流程。

啟示：遠程運維是技術、服務流程與信任的整合。

案例 26-6 教學情境：模型外推的陷阱

某團隊以歷史資料訓練刀具壽命模型，準確率很高。後來工廠為了提高產能把切削速度調高，模型的預測卻完全失準，造成多次斷刀。檢討發現，訓練資料從未涵蓋新的切削條件。

啟示：資料模型不會告訴你它不知道；條件改變時，要以物理知識與新資料重新驗證。

實作活動

活動 26-1 壽命資料分析

以 26.1 節的程式，改變設限的時間點與樣本數，觀察韋伯參數估計的變化；並比較「丟掉設限資料」與「把設限當故障」兩種錯誤做法的結果。

活動 26-2 維護策略模擬

以 26.2 節的程式，改變前置時間、量測間隔與故障成本，找出狀態門檻與 RUL 策略的最佳參數，並說明在什麼條件下預測維護最有價值。

活動 26-3 公開資料集的 RUL 預測

以 NASA C-MAPSS 等公開資料集，比較線性劣化模型、隨機森林與 LSTM 的 RUL 預測，並以預測區間的涵蓋率評估不確定性。

活動 26-4 混合模型

以 26.3 節的程式，把殘差模型換成線性迴歸或高斯過程，比較外推表現；並嘗試加入物理約束（例如磨耗速率隨速度單調增加）。

活動 26-5 遠程運維的效益評估

蒐集（或假設）一家工廠一年的叫修紀錄，分類可遠端排除與需要到場的問題，以 26.4 節的程式估計導入遠程運維的效益，並列出資安要求。

活動 26-6 MPC 與 RL

以 26.5 節的程式，在 MPC 的模型中加入 20% 的增益誤差，比較 MPC 與 PI 的表現；再修改維護 MDP 的成本，觀察最佳門檻與 Q-learning 學習所需的次數如何變化。

延伸閱讀

- F. Stanley Nowlan 與 Howard F. Heap，Reliability-Centered Maintenance（聯合航空執行、美國國防部委託的報告，1978）。
- Andrew K. S. Jardine、Daming Lin 與 Dragan Banjevic，〈A Review on Machinery Diagnostics and Prognostics Implementing Condition-based Maintenance〉，Mechanical Systems and Signal Processing，2006。
- Maziar Raissi、Paris Perdikaris 與 George E. Karniadakis，〈Physics-informed Neural Networks: A Deep Learning Framework for Solving Forward and Inverse Problems Involving Nonlinear Partial Differential Equations〉，Journal of Computational Physics，第 378 卷，2019。

- S. Joe Qin 與 Thomas A. Badgwell · 〈A Survey of Industrial Model Predictive Control Technology〉 · Control Engineering Practice · 2003 。
- Richard S. Sutton 與 Andrew G. Barto · Reinforcement Learning: An Introduction (第 2 版 · MIT Press · 2018) 。

表 26-7 本章思考地圖

本章概念	關鍵提問	後續延伸章節
可靠度與可用度	故障模式是早夭、隨機還是磨耗？	第 8、19 章
維護策略	有可偵測的徵兆嗎？故障後果多嚴重？	第 27 章
狀態監測與 RUL	P-F 間隔與前置時間夠嗎？不確定性多大？	第 22、23 章
混合模型	要外推嗎？物理知識在哪裡？	第 19、20 章
遠程運維	哪些問題能遠端解決？資安與核准如何設計？	第 32、35 章
MPC 與最佳化	模型可信嗎？限制完整嗎？	第 18、27 章
強化學習	在哪裡訓練？如何驗證與監督？	第 29、36 章

章末小結

- 可靠度、可用度與可維護度是維護決策的基礎。韋伯分布的形狀參數區分早夭、隨機與磨耗故障，分析時要正確處理設限資料。範例中 β 約 2.65、平均壽命約 3,535 小時，最佳定期更換週期約 1,219 小時，成本約為壞了才換的三分之一，但大多數零件在還能用很久時就被換掉。
- 維護策略從事後、預防、狀態、預測演進到規範性維護，RCM 依故障模式與後果選擇策略。狀態監測涵蓋振動、油液、熱像、超音波與控制器資料；RUL 預測要配合 P-F 間隔與前置時間。範例中狀態門檻與 RUL 預測把成本降到定期更換的一半左右，RUL 再省約一成，但承擔約 0.5% 的故障率。
- 物理模型可外推，資料模型擅長擷取複雜關係，混合模型結合兩者。範例中隨機森林外推誤差約 45%，物理模型約 7%；殘差混合模型在外推區並未改善。不確定性要同時考慮參數與殘差：只看參數時區間涵蓋率約 36%，加入殘差後約 98%。

- 遠程運維由設備端、安全連線、平台、應用與服務流程組成，需遵循 OT 資安規範。服務模式從按次維修走向訂閱、保證可用度與依產出計價。範例中（依假設）遠程診斷使每台年停機從約 218 小時降到約 55 小時。
- MPC 以模型預測並直接處理限制，範例中在不超溫的前提下誤差小於最佳 PI；強化學習可在不知道模型時學到最佳的維護門檻，但需要大量互動，通常在模擬器中訓練。閉環決策需要權限、限制、安全系統與紀錄的治理。

習題

一、選擇題

1. 韋伯分布的形狀參數 β 大於 1，表示？（A）早夭型故障 （B）隨機故障 （C）磨耗型故障
（D）無法判斷
2. 可用度的常用估計式為？（A） $MTTR \div MTBF$ （B） $MTBF \div (MTBF + MTTR)$ （C） $MTBF \times MTTR$ （D） $1 - MTBF$
3. 觀察結束時尚未故障的資料稱為？（A）離群值 （B）設限資料 （C）缺值 （D）重複資料
4. 依設備實際狀態決定何時維護的策略是？（A）事後維護 （B）定期維護 （C）狀態維護
（D）改良維護
5. 依故障模式與後果選擇維護策略的方法是？（A）RCM （B）SPC （C）DOE （D）MRP
6. 從可偵測潛在故障到功能失效的時間稱為？（A）MTTR （B）P-F 間隔 （C）前置時間 （D）週期時間
7. 以物理方程式殘差作為損失函數的神經網路是？（A）CNN （B）PINN （C）GAN （D）RNN
8. 每個時刻求解最佳化、只執行第一步的控制方法是？（A）PID （B）MPC （C）開關控制
（D）前饋控制
9. 不需要事先知道機率模型、依獎勵學習策略的方法是？（A）動態規劃 （B）強化學習 （C）線

性規劃 (D) 迴歸

10. 工業控制系統資安的國際標準系列是？(A) ISO 9001 (B) IEC 62443 (C) ISO 14001
(D) IEC 61131

二、簡答題

11. 說明浴缸曲線的三個階段與對應的維護策略。
12. 為什麼可用度很高，維護成本仍可能不合理？
13. 比較狀態維護與預測維護的差異。
14. 說明 P-F 間隔與量測頻率、前置時間的關係。
15. 為什麼機器學習模型外推時容易失準？
16. 說明認知不確定性與隨機不確定性的差異。
17. 遠程運維有哪些資安風險？如何降低？
18. 為什麼工業上的強化學習通常在模擬器中訓練？

三、計算題

19. MTBF 為 500 小時，MTTR 為 5 小時，求可用度。
20. 韋伯分布 $\beta = 1$ ， $\eta = 2,000$ 小時，求運轉 1,000 小時的可靠度。
21. 韋伯分布 $\beta = 2.5$ ， $\eta = 4,000$ 小時，求運轉 2,000 小時的可靠度。
22. 某設備 1 年 (8,760 小時) 內故障 4 次，總停機 40 小時，求 MTBF 與 MTTR。
23. 振動指標每 100 小時增加為原來的 1.2 倍，目前為 0.4，失效門檻 1.0，估計剩餘壽命。
24. 預測剩餘壽命 400 小時，標準差 80 小時，前置時間 200 小時，以「平均減 2 個標準差」作為下界，是否需要立即開工單？
25. 遠程排除一次成本 1,500 元，到場一次成本 12,000 元。一年 600 次故障，若 55% 可遠端排除，其餘一次到場，與全部到場相比節省多少？
26. Q-learning 中， $Q(s,a) = 10$ ，獎勵 $r = 100$ ， $\gamma = 0.98$ ， $\max Q(s',\cdot) = 50$ ， $\alpha = 0.05$ ，求更新

後的 $Q(s,a)$ 。

四、申論與設計題

27. 為一座有 30 台 CNC 工具機的工廠設計預測維護計畫：說明設備與故障模式的優先排序、監測技術、資料平台、RUL 與決策規則、與排程的整合。
28. 評論以下說法：「預測維護可以完全取代定期保養。」
29. 為一家設備商設計遠程運維服務：說明系統架構、資安、服務流程、計價模式與成效指標。

五、是非題

30. 分析壽命資料時，可以直接刪除尚未故障的設限資料。
31. 定期更換的最佳週期一定等於平均壽命。
32. 物理模型的參數估計值一定等於真實的物理常數。
33. 只考慮參數不確定性，可能低估預測的不確定性。
34. MPC 可以直接在最佳化中處理輸入與輸出的限制。
35. 強化學習在真實產線上直接試誤學習通常是安全的。

六、進階挑戰題

36. 以 C-MAPSS 資料集訓練 LSTM 或 Transformer 的 RUL 模型，並以保形預測產生預測區間，評估涵蓋率與區間寬度，再設計依區間下界開工單的決策規則。
37. 實作一個簡單的 PINN，學習一維熱傳導方程式的溫度分布，比較只用資料與加入物理損失在少量資料時的表現。
38. 把 26.5 節的維護 MDP 擴充為多台設備共用一組維修人力，以強化學習學習派工策略，並與簡單的優先規則比較。

七、補充題

39. 列出表 26-2 中的六種維護策略。

40. 列出遠程運維架構的五個組成。

提示與參考解答

1. C。

2. B。

3. B。

4. C。

5. A。

6. B。

7. B。

8. B。

9. B。

10. B。

11. 參考：早夭期故障率高且下降，可透過燒機、品質管制與安裝驗收改善；隨機故障期故障率大致固定，定期更換無效，適合狀態監測；磨耗期故障率上升，可定期更換或預測維護。

12. 參考：可用度只看停機時數，但每次故障可能造成報廢、連帶損壞、緊急叫修與交期延誤，代價遠大於停機時數；預防更換也有成本。

13. 參考：狀態維護在量測值超過門檻時維護；預測維護預測故障時間，配合前置時間與排程決定時機，可更充分地使用壽命。

14. 參考：量測間隔要明顯短於 P-F 間隔才能及時偵測；從偵測到失效的時間要大於備料、排程與維修的前置時間，否則來不及處理。

15. 參考：模型只從訓練資料學習，訓練範圍外沒有資訊；例如決策樹只能輸出訓練過的數值範圍，神經網路的外推行為也難以預期。

16. 參考：認知不確定性來自資料與模型的不足，可以透過更多資料降低；隨機不確定性來自系統本

身的變異與雜訊，無法消除。

17. 參考：遠端存取可能成為入侵途徑、帳號被盜、未授權的操作。可採用分區與管道、VPN、多因子驗證、最小權限、連線核准、單向資料傳輸與存取紀錄。

18. 參考：強化學習需要大量試誤，在真實產線上會造成故障、報廢與安全風險；在模擬器或數位孿生中訓練，再經驗證與限制後上線。

19. $500 \div 505 \approx 99.0\%$ 。

20. $R = e^{-(1000/2000)^1} = e^{-0.5} \approx 60.7\%$ 。

21. $R = e^{-(0.5)^{2.5}} = e^{-0.177} \approx 83.8\%$ 。

22. $MTTR = 40 \div 4 = 10$ 小時； $MTBF \approx (8,760 - 40) \div 4 = 2,180$ 小時。

23. 需 $1.2^n = 2.5$ ， $n = \ln 2.5 \div \ln 1.2 \approx 5.03$ ，約 503 小時。

24. 下界 = $400 - 160 = 240$ 小時，大於前置時間 200 小時，尚不需立即開工單，但應在下一次量測重新評估。

25. 全部到場： $600 \times 12,000 = 7,200,000$ 元；遠程： $330 \times 1,500 + 270 \times 12,000 = 495,000 + 3,240,000 = 3,735,000$ 元；節省約 3,465,000 元。

26. $Q \leftarrow 10 + 0.05 \times (100 + 0.98 \times 50 - 10) = 10 + 0.05 \times 139 = 16.95$ 。

27. 參考：以停機代價與故障頻率排序設備與故障模式；主軸以振動與負載、刀具以負載與聲音、液壓以油液監測；以邊緣蒐集、平台儲存；建立基準、RUL 與下界規則；工單與 MES 排程整合，合併在換線或計畫停機時執行。

28. 參考：不同意。預測維護適合有可偵測徵兆、後果嚴重的故障；沒有徵兆的故障、法規要求的檢查、潤滑與清潔等仍需定期保養。應依 RCM 組合策略。

29. 參考：設備端閘道器、加密連線、資料平台與診斷知識庫；IEC 62443、客戶核准與存取紀錄；分級回應、遠端排除、預先備料與到場；訂閱或保證可用度；以停機時間、一次修復率、回應時間與客戶滿意度衡量。

30. 否，應以最大概似等方法正確納入。

- 31. 否，最佳週期取決於壽命分布與成本比，範例中只有平均壽命的三分之一左右。
- 32. 否，參數可能吸收未建模的效應。
- 33. 是。
- 34. 是。
- 35. 否，應在模擬器中訓練並經驗證與監督。
- 36. 提示：以引擎分組切分訓練與測試；比較區間涵蓋率是否接近目標，以及決策規則下的故障率與壽命使用率。
- 37. 提示：損失函數包含資料誤差與偏微分方程式殘差，以自動微分計算導數。
- 38. 提示：狀態包含各設備的劣化等級與維修人力狀態；比較「最劣化優先」等規則與學到的策略。
- 39. 事後維護、預防維護、狀態維護、預測維護、規範性維護、以可靠度為中心的維護。
- 40. 設備端、連線、平台、應用、服務流程。

第 27 章

智慧排程、派工與生產控制

編著 李世淵（機器人叫獸） | 助教 李天宇、李宇晴

本章重點

- 流程型、零工式、彈性零工式與重入式流程的排程問題
- 完工時間、交期、在製品、換線與能源：目標的取舍
- 派工規則、限制規劃求解器與模擬退火：以 FT06 等題目比較
- 學習派工規則、強化學習與數位孿生的動態重排
- 晶圓廠與封測廠的即時派工、Little 定律與 X 因子

叫獸開講：排程表為什麼總是第一個小時就作廢？

有一家做精密機械零件的工廠，生管每天下午花三個小時，用試算表排好隔天三十幾台機器的工作。隔天早上八點開工，九點就有一台機器故障，十點來了一張急單，十一點某個料號的材料晚到——排程表在第一個小時就作廢了，接下來一整天，現場領班憑經驗「看著辦」。

廠長請我幫忙「導入 AI 排程」。我先問了三個問題：排程的目標是什麼？是準時交貨、產能最大、換線最少，還是在製品最少？現場能不能即時知道每台機器、每張工單的狀態？排程算出來之後，誰有權改、改了誰知道？三個問題的答案都很模糊。

我們先做的不是 AI，而是把目標說清楚：以準時交貨為主，換線次數為輔；再從 MES 取得即時的機台狀態與工單進度；然後用一個最佳化求解器每天產生基準排程，遇到故障或急單時，只重排受影響的部分，其餘不動，以免現場無所適從。兩個月後，準時交貨率明顯提高，換線次數也下降了。

生管後來說：「原來排程不是一張表，而是一個不斷修正的過程。」沒錯。排程演算法很重要，但更重要的是：目標要清楚、資料要即時、重排要有規則。

這一章，我們從流程型、零工式、彈性零工式與重入式流程等排程問題談起；說明完工時間、交期、在製品、換線與能源等目標；介紹派工規則、數學規劃與後設啟發式方法；討論機器學習、強化學習與結合數位孿生的排程；最後以晶圓廠與封測廠的智慧派工作結。

（說明：本節情境由作者輔導企業生產管理的經驗綜合改寫，人物與細節已調整。）

學習目標

讀完本章後，你應該能夠：

1. 區分流程型、零工式、彈性零工式與重入式流程的排程問題，並說明它們的計算複雜度。
2. 說明完工時間、交期、在製品、換線與能源等排程目標，以及它們之間的取舍。
3. 應用 Johnson 法則、NEH 與常見派工規則，並以限制規劃求解器求得最佳排程。
4. 以模擬退火等後設啟發式方法求解排程問題，比較其品質與求解時間。

5. 說明機器學習與強化學習在排程中的應用，並以程式學習一條派工規則。
6. 說明晶圓廠與封測廠的重入式流程與派工特性，並以 Little 定律分析週期時間與在製品。

核心概念

表 27-1 本章核心名詞中英對照

中文名詞	英文	一句話說明
排程	Scheduling	決定工作在哪台資源、何時開始與結束
派工	Dispatching	資源空閒時，從等候的工作中即時挑選下一個
流程型工廠	Flow Shop	所有工件依相同的機器順序加工
零工式工廠	Job Shop	每個工件有自己的機器順序（途程）
彈性零工式工廠	Flexible Job Shop, FJSP	每道工序可在多台機器中選一台加工
重入式流程	Re-entrant Flow	工件多次回到同一組機台，如晶圓製造
完工時間	Makespan	所有工作完成的時間
延遲	Tardiness	完成時間超過交期的部分
在製品	Work in Process, WIP	已投入但尚未完成的工件數量
週期時間	Cycle Time	工件從投入到完成的時間
混合整數線性規劃	Mixed-integer Linear Programming, MILP	含整數變數的線性最佳化模型
限制規劃	Constraint Programming, CP	以變數、值域與限制描述並搜尋可行解
後設啟發式方法	Metaheuristic	模擬退火、基因演算法等通用的近似搜尋方法
Little 定律	Little's Law	在製品 = 產出率 × 週期時間

27.1 Flow Shop、Job Shop、FJSP 與重入式流程

27.1.1 排程問題的分類

排程問題常以「機器環境 | 工件特性 | 目標」三欄記號描述。機器環境決定了問題的結構：

表 27-2 常見的排程問題類型

類型	機器環境	例子	難度
單機	一台機器	瓶頸機台、熱處理爐	部分目標有簡單最佳規則
平行機	多台功能相同或相近的機器	射出機群、測試機台	多數目標為 NP-hard
流程型 (Flow Shop)	所有工件依相同順序經過各站	組裝線、PCB 產線	兩機可用 Johnson 法則，三機以上一般為 NP-hard
零工式 (Job Shop)	每個工件有自己的途程	模具、機械加工	NP-hard，經典難題
彈性零工式 (FJSP)	工序可選擇機器	多功能 CNC、智慧工廠	同時決定指派與順序，更難
開放型 (Open Shop)	工序順序不限	檢驗、維修	多數為 NP-hard
重入式	工件多次回到同一組機台	晶圓製造、面板	通常以派工與模擬處理

NP-hard 的意思是：目前沒有已知的方法能在問題規模變大時，以多項式時間保證找到最佳解。

10 個工件在一台機器上的順序就有 $10! \approx 363$ 萬種；20 個工件約有 2.4×10^{18} 種。因此實務上的排程方法，多半是在「解的品質」與「計算時間」之間取捨。

27.1.2 流程型工廠

兩部機器的流程型工廠，若要最小化完工時間，Johnson 在 1954 年提出了最佳法則：把第一部機器加工時間較短的工件排在前面（依第一部機器時間遞增），第二部機器時間較短的排在後面（依第二部機器時間遞減）。三部以上機器就沒有這麼簡單的最佳法則，常用的啟發式方法是 1983 年

Nawaz、Enscore 與 Ham 提出的 NEH：先依總加工時間由大到小排序，再逐一把工件插入目前部分排程中最好的位置。

```
import numpy as np, itertools
# 流程型工廠 (Flow Shop)：兩部機器的 Johnson 法則，以及多部機器的 NEH 啟發式，與窮舉最佳解比較
def makespan(seq, P):                                     # P[j][m]: 工件 j 在機器
m 的加工時間；所有工件依相同順序經過各機器
    C = np.zeros(P.shape[1])
    for j in seq:
        for m in range(P.shape[1]):
            C[m] = max(C[m], C[m-1] if m else 0) + P[j, m]
    return C[-1]
def johnson(P):                                           # 兩部機器：第一部較短的排
前面 (遞增)，第二部較短的排後面 (遞減)
    A = sorted([j for j in range(len(P)) if P[j, 0] <= P[j, 1]], key=lambda j: P[j, 0])
    B = sorted([j for j in range(len(P)) if P[j, 0] > P[j, 1]], key=lambda j: -P[j, 1])
    return A + B
def neh(P):                                               # 依總加工時間由大到小，逐
一插入目前最佳的位置
    order = np.argsort(-P.sum(1)); seq = [order[0]]
    for j in order[1:]:
        seq = min((seq[:i] + [j] + seq[i:] for i in range(len(seq) + 1)), key=lambda s:
makespan(s, P))
    return seq
rng = np.random.default_rng(1)
P2 = rng.integers(1, 20, (7, 2)); P5 = rng.integers(1, 30, (8, 5))
best2 = min(makespan(s, P2) for s in itertools.permutations(range(7)))
print(f"兩部機器、7 個工件：Johnson 法則 {makespan(johnson(P2), P2):.0f}，窮舉最佳
{best2:.0f}，依編號順序 {makespan(range(7), P2):.0f}")
best5 = min(makespan(s, P5) for s in itertools.permutations(range(8)))
rand = np.mean([makespan(rng.permutation(8), P5) for _ in range(1000)])
print(f"五部機器、8 個工件：NEH {makespan(neh(P5), P5):.0f}，窮舉最佳 {best5:.0f} (40,320 種
順序)，隨機順序平均 {rand:.1f}")
```

執行結果：兩部機器、7 個工件時，Johnson 法則的完工時間 85，與窮舉 5,040 種順序的最佳解相同，而依編號順序是 96。五部機器、8 個工件時，NEH 得到 184，窮舉 40,320 種順序的最佳解是 180，隨機順序的平均約 220.8。NEH 只檢查了很少的順序，就得到距最佳約 2% 的解。

27.1.3 零工式、彈性零工式與重入式流程

零工式工廠中，每個工件有自己的途程，機器上的順序決定了工件之間的等待。1963 年 Fisher 與 Thompson 發表的 6×6 、 10×10 題目 (FT06、FT10) 成為經典的測試題；其中 10×10 的 FT10 直到 1989 年才由 Carlier 與 Pinson 以分支界限法證明最佳值為 930，可見零工式問題之難。彈性零工式 (FJSP) 還要決定每道工序在哪一台機器加工，更貼近擁有多功能設備的現代工廠。

重入式流程是半導體製造的特徵：一片晶圓要經過數百道步驟，其中微影、蝕刻、薄膜等機台被同一批晶圓反覆使用數十次。同一機台前的等候線中，混著處於不同製程階段的晶圓批，派工的選擇

會影響整條線的流動，這也是 27.5 節的重點。

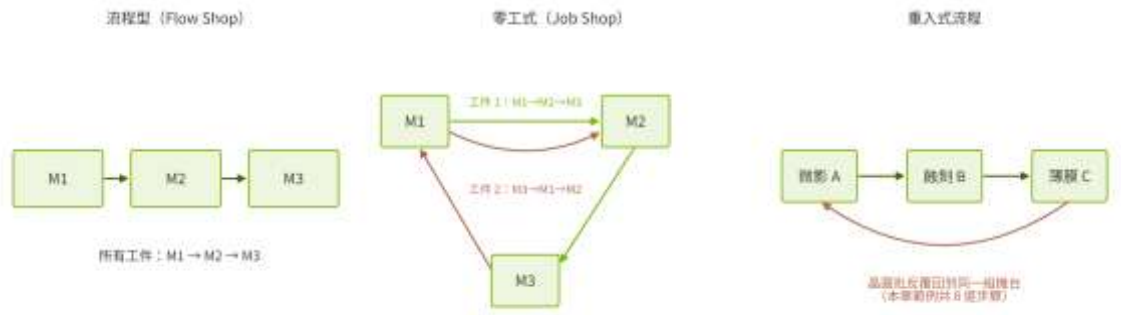


圖 27-1 流程型、零工式與重入式流程的示意

27.2 目標函數：Makespan、交期、WIP、換線與能源

27.2.1 常見的排程目標

表 27-3 常見的排程目標

目標	定義	關心的人	常見做法
完工時間 (Makespan)	最後一個工件完成的時間	產能、設備投資	瓶頸優先、LPT、最佳化
交期 (延遲、延遲件數)	超過交期的時間或件數	業務、客戶	EDD、最小寬裕、ATC
流程時間與在製品	工件在廠內的平均時間與數量	生管、財務	SPT、投料控制
換線次數與時間	產品族之間的切換	現場、品質	依族群排程、換線矩陣
設備利用率	機台忙碌的比例	設備、成本	瓶頸保護、預防保養排程
能源與尖峰負載	用電量、契約容量、時間電價	廠務、ESG (第 34 章)	移峰、能源感知排程
穩定性	重排時對原排程的改變量	現場、供應商	凍結區間、局部重排

這些目標常常互相衝突：依交期排序可能增加換線；依產品族集中可以減少換線，卻可能讓急單延遲；提高利用率會讓在製品與週期時間增加（第 8 章的排隊理論）。排程的第一步，是和業務、生

管、現場一起決定目標的優先順序與權重。

27.2.2 範例：交期與換線的取捨

下面的程式有一台機器、12 張工單，分屬 3 個產品族，換族需要 30 分鐘的換線時間。比較依交期排序 (EDD)、依產品族集中，以及以限制規劃求解器 CP-SAT 最小化「總延遲 + 每次換線計 30」與「只看延遲」兩種目標：

```
import numpy as np
from ortools.sat.python import cp_model
# 多目標：單一機台上 12 張工單，分屬 3 個產品族；換族需換線 30 分鐘。比較 EDD、依族群集中與 CP-
SAT 的加權最佳化
rng = np.random.default_rng(4)
n = 12; fam = rng.integers(0, 3, n); p = rng.integers(20, 60, n); due =
np.sort(rng.integers(80, 650, n)); SU = 30
def evaluate(seq):
    t, tard, su, prev = 0, 0, 0, None
    for j in seq:
        if prev is not None and fam[j] != fam[prev]: t += SU; su += 1
        t += p[j]; tard += max(0, t - due[j]); prev = j
    return t, tard, su
def optimize(w_tard, w_su, limit=20):
    # 以 AddCircuit 建立含換
    # 線時間的順序模型
    md = cp_model.CpModel(); H = int(p.sum() + SU * n); S = [md.NewIntVar(0, H, "") for _
    in range(n)]
    arcs = []; T = [md.NewIntVar(0, H, "") for _ in range(n)]; sus = []
    for i in range(n):
        b = md.NewBoolVar(""); arcs.append((0, i + 1, b)); md.Add(S[i] ==
    0).OnlyEnforceIf(b) # 第一張工單
        arcs.append((i + 1, 0, md.NewBoolVar("")))
    # 最後一張工單
    for j in range(n):
        if i == j: continue
        x = md.NewBoolVar(""); arcs.append((i + 1, j + 1, x)); s = SU if fam[i] !=
    fam[j] else 0
        md.Add(S[j] >= S[i] + int(p[i]) + s).OnlyEnforceIf(x)
        if s: sus.append(x)
        md.Add(T[i] >= S[i] + int(p[i]) - int(due[i]))
    md.AddCircuit(arcs); md.Minimize(w_tard * sum(T) + w_su * sum(sus))
    sv = cp_model.CpSolver(); sv.parameters.max_time_in_seconds = limit;
    sv.parameters.num_workers = 8; sv.Solve(md)
    return list(np.argsort([sv.Value(s) for s in S]))
edd = list(np.argsort(due)); grp = sorted(range(n), key=lambda j: (fam[j], due[j]))
for name, seq in [("EDD (交期優先)", edd), ("依產品族集中", grp), ("CP-SAT (延遲 + 每次換線計
    30)", optimize(1, 30)), ("CP-SAT (只看延遲)", optimize(1, 0))]:
    c, t, s = evaluate(seq); print(f"{name}: 完工 {c} 分, 總延遲 {t} 分, 換線 {s} 次")
```

執行結果：EDD 的總延遲 26 分，但換線 6 次、完工 666 分；依產品族集中只換線 2 次、完工 546 分，總延遲卻高達 339 分。CP-SAT 在兼顧兩者的目標下，找到了總延遲 0、只換線 2 次、完工 546 分的排程——同時優於兩種直覺做法。只看延遲的目標雖然也做到延遲 0，卻換線 5 次。

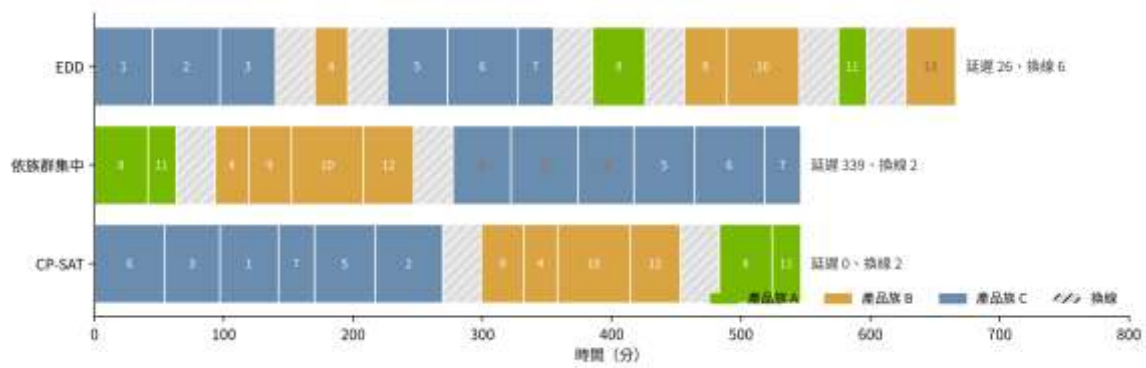


圖 27-2 三種排程的甘特圖：EDD、依族群集中與最佳化

這個例子說明：目標的寫法會直接改變結果。「只看延遲」時，求解器不在乎換線，換線次數只是碰巧的結果；把換線放進目標，才會得到兩者兼顧的排程。另一方面，直覺規則在單一目標上表現不錯，但多目標時往往有明顯的盲點。

27.3 派工規則、MILP 與後設啟發式方法

27.3.1 派工規則

派工規則在資源空閒時，依簡單的優先順序挑選下一個工作。它的優點是快速、容易理解、能即時反應現場的變化，因此是工廠裡最普遍的方法；缺點是只看眼前，不保證整體最佳。

表 27-4 常見的派工規則

規則	優先選擇	擅長的目標	注意事項
FIFO	最早到達的工作	公平、週期時間變異小	不考慮交期與加工時間
SPT	加工時間最短的工序	平均流程時間、在製品	長工件可能一直等
EDD	交期最早的工作	最大延遲	不考慮剩餘工時
最小寬裕 (Slack)	交期減剩餘工時最小者	交期	寬裕為負時需特別處理
CR (臨界比)	剩餘交期 ÷ 剩餘工時最小者	交期	晶圓廠常用
MWKR	剩餘工時最多的工件	完工時間	避免長工件拖到最後

規則	優先選擇	擅長的目標	注意事項
ATC	結合加權、加工時間與寬裕的指數函數	加權延遲	需調整參數
換線最少	與目前機台設定相同的工作	換線次數	可能犧牲交期

以 FT06 為例，比較幾種派工規則與 CP-SAT 求得的最佳解：

```
import numpy as np
from ortools.sat.python import cp_model
# 零工式工廠 (Job Shop)：以經典的 FT06 題目 (6 工件 × 6 機器) 比較派工規則與 CP-SAT 求得的最佳解
FT06 = [[(2,1),(0,3),(1,6),(3,7),(5,3),(4,6)], [(1,8),(2,5),(4,10),(5,10),(0,10),(3,4)],
          [(2,5),(3,4),(5,8),(0,9),(1,1),(4,7)], [(1,5),(0,5),(2,5),(3,3),(4,8),(5,9)],
          [(2,9),(1,3),(4,5),(5,4),(0,3),(3,1)], [(1,3),(3,3),(5,9),(0,10),(4,4),(2,1)]]
# 每個工件的 (機器, 加工時間) 途程
def dispatch(jobs, rule):
    # 非延遲排程：每次在最早可開工的機器上，依規則從可加工的工序中挑一個
    nj = len(jobs); nxt = [0] * nj; jt = [0] * nj; mt = {}; done = 0; total = sum(len(j) for j in jobs)
    rem = [sum(p for _, p in j) for j in jobs]; sched = []
    while done < total:
        cand = [(max(jt[j], mt.get(jobs[j][nxt[j]] [0], 0)), j) for j in range(nj) if
nxt[j] < len(jobs[j])]
        t0 = min(c[0] for c in cand); m0 = min(jobs[j][nxt[j]] [0] for s, j in cand if s == t0)
        ready = [j for s, j in cand if jobs[j][nxt[j]] [0] == m0 and s == t0]
        key = {"FIFO": lambda j: jt[j], "SPT": lambda j: jobs[j][nxt[j]] [1], "LPT": lambda j: -jobs[j][nxt[j]] [1],
"MWKR": lambda j: -rem[j], "LWKR": lambda j: rem[j]}[rule]
        j = min(ready, key=key); m, p = jobs[j][nxt[j]]; s = max(jt[j], mt.get(m, 0))
        sched.append((j, m, s, p)); jt[j] = mt[m] = s + p; rem[j] -= p; nxt[j] += 1; done += 1
    return max(jt), sched
def cpsat(jobs, limit=30):
    md = cp_model.CpModel(); H = sum(p for j in jobs for _, p in j); iv = {}; S = {}
    for j, job in enumerate(jobs):
        for k, (m, p) in enumerate(job):
            S[j, k] = md.NewIntVar(0, H, ""); iv.setdefault(m, []).append(md.NewIntervalVar(S[j, k], p, S[j, k] + p, ""))
            if k: md.Add(S[j, k] >= S[j, k-1] + job[k-1][1])
        for m in iv: md.AddNoOverlap(iv[m])
        C = md.NewIntVar(0, H, ""); [md.Add(C >= S[j, len(job)-1] + job[-1][1]) for j, job in enumerate(jobs)]
    md.Minimize(C); sv = cp_model.CpSolver(); sv.parameters.max_time_in_seconds = limit;
sv.parameters.num_workers = 8
    st = sv.Solve(md)
    sched = [(j, m, sv.Value(S[j, k]), p) for j, job in enumerate(jobs) for k, (m, p) in enumerate(job)]
    return sv.Value(C), sv.StatusName(st), sched
for r in ["FIFO", "SPT", "LPT", "MWKR", "LWKR"]: print(f"{r}: 完工時間 {dispatch(FT06, r)[0]}")
c, st, _ = cpsat(FT06); print(f"CP-SAT: 完工時間 {c} ({st})")
```

執行結果：FIFO 的完工時間 65 · SPT 88 · LPT 77 · MWKR 61 · LWKR 83；CP-SAT 在不到一

秒內求得最佳解 55，並證明它是最佳（OPTIMAL），與文獻中 FT06 的最佳值一致。對完工時間而言，MWKR 最好，距最佳約 11%；SPT 雖然擅長降低平均流程時間，對完工時間卻是最差的規則之一——規則要與目標相配。

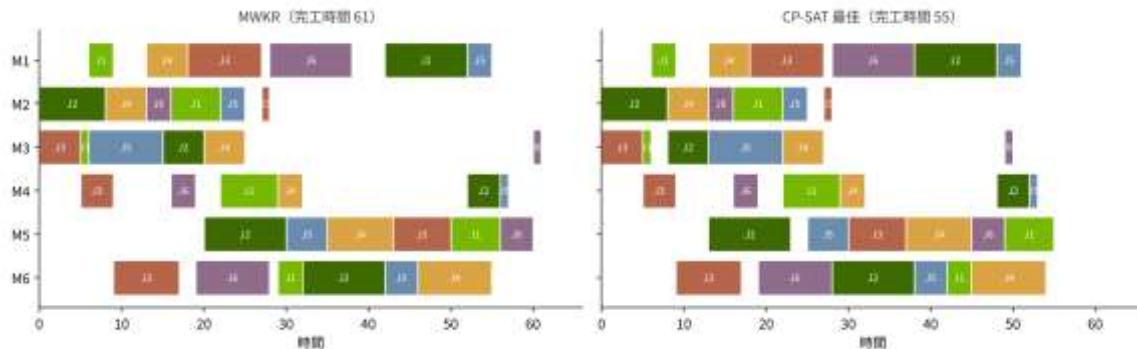


圖 27-3 FT06 的派工規則（MWKR）與最佳排程甘特圖

27.3.2 數學規劃與限制規劃

排程可以寫成混合整數線性規劃（MILP）：以連續變數表示開始時間，以二元變數表示同一台機器上兩個工序的先後，再以「大 M」限制式確保不重疊。MILP 的好處是能精確描述目標與限制，並以通用的求解器求得最佳解或附帶下界的近似解；缺點是規模變大時，求解時間可能急劇增加。

限制規劃（CP）對排程特別有效：它有「區間變數」與「不重疊」等專為排程設計的限制，搜尋時能有效地刪除不可行的組合。上面的程式使用 Google OR-Tools 的 CP-SAT 求解器，只要寫出工序的先後與機器不重疊，就能求得最佳排程。商用與開源的求解器都在持續進步，許多過去只能用啟發式方法處理的中等規模問題，如今可以直接求解。

27.3.3 後設啟發式方法

規模更大、或限制太複雜時，常用後設啟發式方法：模擬退火、禁忌搜尋、基因演算法、蟻群演算法等。它們在解空間中反覆移動，以一定的機率接受較差的解，以跳出局部最佳。下面的程式以「工件編號重複」的方式編碼一個排程（工件編號第 k 次出現代表該工件的第 k 道工序），以交換兩個位置作為鄰域移動，對一個隨機產生的 10×10 題目進行模擬退火：

```
import numpy as np
# 延續上一段程式的 dispatch() 與 cpsat()
# 後設啟發式：以模擬退火搜尋工序的排列順序（以工件編號重複編碼），比較派工規則、模擬退火與 CP-SAT
# （10 工件 × 10 機器）
rng = np.random.default_rng(3)
J10 = [[(int(m), int(rng.integers(1, 100))) for m in rng.permutation(10)] for _ in
```

```

range(10)] # 隨機產生的 10 × 10 題目
def decode(code, jobs): # 編碼：工件編號出現第 k
    次，代表該工件的第 k 道工序；依序以最早可開工時間排入
    nxt = [0] * len(jobs); jt = [0] * len(jobs); mt = {}
    for j in code:
        m, p = jobs[j][nxt[j]]; s = max(jt[j], mt.get(m, 0)); jt[j] = mt[m] = s + p;
    nxt[j] += 1
    return max(jt)
def anneal(jobs, iters=60000, T0=40.0):
    code = np.repeat(np.arange(len(jobs)), len(jobs[0])); rng.shuffle(code)
    cur = best = decode(code, jobs); bc = code.copy()
    for k in range(iters):
        T = T0 * (1 - k / iters) + 0.1; i, j = rng.integers(0, len(code), 2)
        code[i], code[j] = code[j], code[i]; v = decode(code, jobs)
        if v <= cur or rng.random() < np.exp((cur - v) / T): cur = v
        else: code[i], code[j] = code[j], code[i]
        if cur < best: best, bc = cur, code.copy()
    return best
print("派工規則：" + " , ".join(f"{r} {dispatch(J10, r)[0]}" for r in ["FIFO", "SPT",
"MWKR"]))
print(f"模擬退火（6 萬次鄰域移動）：{anneal(J10)}")
c, st, _ = cpsat(J10, 60); print(f"CP-SAT：{c}（{st}）")

```

執行結果：派工規則的完工時間為 FIFO 929、SPT 998、MWKR 906；模擬退火經過 6 萬次鄰域移動得到 848；CP-SAT 求得並證明最佳解 804。模擬退火比最好的派工規則改善約 6%，但距最佳仍有約 5%。

這個結果並不代表後設啟發式方法比較差：對這個規模的問題，CP-SAT 很有效率；但當問題擴大到數百台機器、數千道工序，並加入換線、批量、人員與物料等限制時，精確求解往往不再可行，後設啟發式方法、問題分解（例如先指派機器再排順序）與滾動時域的局部最佳化，就成為實務上的主力。

27.4 機器學習、強化學習與學生排程

27.4.1 學習派工規則

派工規則簡單、快速，但「哪一條規則最好」取決於工廠的情境。一種做法是讓機器從歷史或模擬資料中「學」出一條規則：把每個候選工序的特徵（加工時間、剩餘交期、剩餘工時、剩餘工序數等）組合成優先分數，以演化演算法、基因規劃或強化學習搜尋最好的組合。

下面的程式以 4 個特徵的線性加權作為優先分數，用交叉熵法在 30 個訓練題目上搜尋權重，最小化總延遲；再到 100 個新的測試題目上，與 SPT、EDD、最小寬裕與 CR 比較：

```
import numpy as np
```

```

# 學習派工規則：以多個特徵的線性加權作為優先順序，用交叉熵法（CEM）在訓練題目上搜尋權重，再到新題目上與傳統規則比較總延遲
rng = np.random.default_rng(5)
def instance(nj=15, nm=5):
    jobs = [[(int(m), int(rng.integers(5, 40))) for m in rng.permutation(nm)] for _ in range(nj)]
    rel = rng.integers(0, 120, nj); due = rel + np.array([sum(p for _, p in j) for j in jobs]) * rng.uniform(1.3, 2.5, nj)
    return jobs, rel, due
def run(inst, score):
    # 事件導向的派工模擬：機器空閒時，從等候的工序中挑分數最低者
    jobs, rel, due = inst; nj = len(jobs); nxt = [0] * nj; jt = list(rel); mt = {}; rem = [sum(p for _, p in j) for j in jobs]
    left = sum(len(j) for j in jobs)
    while left:
        cand = [(max(jt[j], mt.get(jobs[j][nxt[j]] [0], 0)), j) for j in range(nj) if
nxt[j] < len(jobs[j])]
        t0 = min(c[0] for c in cand); m0 = min(jobs[j][nxt[j]] [0] for s, j in cand if s
== t0)
        q = [j for s, j in cand if s == t0 and jobs[j][nxt[j]] [0] == m0]
        j = min(q, key=lambda j: score(jobs[j][nxt[j]] [1], due[j] - t0, rem[j],
len(jobs[j]) - nxt[j]))
        m, pt = jobs[j][nxt[j]]; jt[j] = mt[m] = t0 + pt; rem[j] -= pt; nxt[j] += 1; left
-= 1
    return sum(max(0, jt[j] - due[j]) for j in range(nj))
rules = {"SPT": lambda p, s, r, k: p, "EDD": lambda p, s, r, k: s, "最小寬裕 (slack)":
lambda p, s, r, k: s - r,
"CR (臨界比)": lambda p, s, r, k: s / max(r, 1)}
lin = lambda w: (lambda p, s, r, k: w[0] * p + w[1] * s + w[2] * r + w[3] * k)
train = [instance() for _ in range(30)]; test = [instance() for _ in range(100)]
mu, sd = np.zeros(4), np.ones(4)
for it in range(25):
    # 交叉熵法：抽樣權重，保留表現最好的 20%，更新分布
    W = rng.normal(mu, sd, (40, 4)); f = np.array([np.mean([run(x, lin(w)) for x in
train]) for w in W])
    elite = W[np.argsort(f)[:8]]; mu, sd = elite.mean(0), elite.std(0) + 0.02
    for name, r in rules.items(): print(f"{name}: 測試題平均總延遲 {np.mean([run(x, r) for x in
test]):.0f}")
    print(f"學到的規則：測試題平均總延遲 {np.mean([run(x, lin(mu)) for x in test]):.0f}; 權重（加工時間、剩餘交期、剩餘工時、剩餘工序）= " + "、".join(f"{v:.2f}" for v in mu /
np.abs(mu).max()))

```

執行結果：在測試題上，SPT 的平均總延遲約 800，EDD 約 708，最小寬裕約 789，CR 約 760；學到的規則約 638，比最好的傳統規則（EDD）少約一成。學到的權重中，加工時間的權重最大（+1.00），剩餘交期次之（+0.21），剩餘工序數為負（-0.23），也就是「先做短的、交期近的、以及快要完成的工件」。

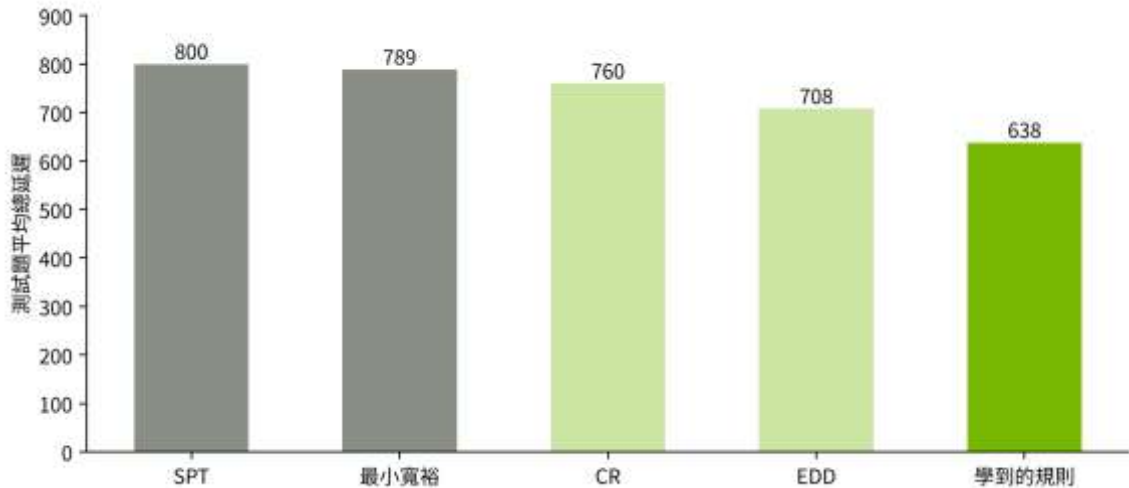


圖 27-4 傳統派工規則與學到的規則：測試題的平均總延遲

學到的規則仍然是一個可以寫下來、解釋的公式，這是它容易被現場接受的原因。不過它是在這類題目上學到的：若產品組合、交期鬆緊或機台結構改變，就需要重新學習與驗證（第 23 章的漂移監測同樣適用）。

27.4.2 強化學習排程

強化學習把排程視為序列決策：每次派工是一個動作，狀態是機台、在製品與工單的情況，獎勵與延遲、產出或完工時間有關。1995 年 Zhang 與 Dietterich 就在 IJCAI 發表以強化學習處理 NASA 太空梭酬載作業排程的研究；近年則以圖神經網路表示零工式工廠的工序關係，再以強化學習訓練派工策略，在大量隨機題目上學到可以泛化到新題目的規則。

強化學習排程的挑戰與第 26 章相同：需要大量的互動，因此通常在模擬器中訓練；狀態與獎勵的設計很關鍵；學到的策略要與最佳化方法和傳統規則公平比較；上線前要確認它在罕見情況（大量急單、多台故障）下不會做出離譜的決策。目前在實務上，強化學習多半用來輔助或調整既有的規則與最佳化，而不是完全取代它們。

27.4.3 學生排程與動態重排

靜態排程很快就會被現實打亂：機台故障、急單、缺料、加工時間的變異、品質重工。動態排程有幾種做法：完全以即時派工因應；定期（例如每班）重新最佳化；或在事件發生時觸發局部重排。重排時常設定「凍結區間」，例如接下來兩小時內已開工或已備料的工作不動，以維持現場的穩定。

數位學生（第 19、21 章）讓排程可以先在虛擬工廠中試算：以最新的機台狀態與在製品初始化模擬模型，比較幾個候選排程或派工規則在未來一段時間的表現，再選出最好的方案下達。這種「模擬最佳化」也能評估排程的穩健性——在加工時間與故障的隨機變化下，哪一個排程的交期表現最穩定。

表 27-5 排程方法的比較

方法	解的品質	計算時間	反應速度	可解釋性	適用情境
派工規則	中	極短	即時	高	現場即時決策、變化頻繁
精確最佳化（MILP、CP）	最佳或附下界	中～長	分鐘～小時	中	中等規模、每日或每班計畫
後設啟發式	良好	中	分鐘	低～中	大規模、限制複雜
學習的規則	良好	極短（訓練需時）	即時	中～高	情境穩定、可大量模擬
強化學習	視訓練而定	極短（訓練需時長）	即時	低	動態環境、有可信的模擬器
學生模擬最佳化	良好、可評估穩健性	中～長	分鐘	中	重排決策、評估方案

27.5 晶圓廠與封測廠智慧派工案例

27.5.1 晶圓廠的派工特性

晶圓廠可能是最複雜的生產排程環境之一：一片晶圓要經過數百道步驟、歷時一到數個月；數百種機台中，有些是單片加工、有些是整批加工（如爐管），有些有多個腔體；微影機台需要搭配特定的光罩；部分步驟之間有「等候時間限制」，例如清洗後必須在一定時間內進入下一站，否則要重洗；再加上機台保養、量測抽樣、工程批與急件。在這樣的環境中，很難一次求出整廠的最佳排程，因此晶圓廠多採用「即時派工系統」：每當機台需要下一批時，由派工系統依規則與即時資料決定，並在瓶頸區域以最佳化或排程器輔助。

表 27-6 晶圓廠派工常見的考量

考量	說明	常見做法
交期與優先等級	急件、客戶承諾、工程批	CR、優先等級分層
瓶頸保護	微影等瓶頸不可空等	投料控制、瓶頸前緩衝、上游配合
重入與線平衡	前段與後段步驟爭用同一機台	依步驟或剩餘步驟排序、線平衡規則
批量加工	爐管等一次處理多批	等湊滿或限時開機的批量規則
設定與光罩	換光罩、換配方的時間	相同設定優先、光罩排程
等候時間限制	兩站之間的時間上限	下游產能確認後才放行
機台限制與保養	機台與產品的配對、預防保養	機台專用化、保養排程整合 (第 26 章)

27.5.2 範例：重入式流程的派工規則

下面的程式模擬一條簡化的重入式產線：晶圓批依 8 道步驟在微影 (A)、蝕刻 (B)、薄膜 (C) 三種機台之間往返，其中微影出現 3 次，是瓶頸。比較三種派工規則：FIFO、前段優先 (FBFS，步驟越前面越優先) 與後段優先 (LBFS，越接近完成越優先)；再以 LBFS 在不同投料率下觀察週期時間與在製品，並以 Little 定律檢查：

```
import numpy as np, heapq
# 重入式流程：晶圓批在微影 (A)、蝕刻 (B)、薄膜 (C) 之間重複進出。比較 FIFO、FBFS (前段優先) 與 LBFS (後段優先) 的週期時間與在製品
route = ["A", "B", "C", "A", "B", "A", "C", "B"] # 8 道步驟，微影 A 出現 3 次 (瓶頸)
ptime = {"A": 1.0, "B": 0.8, "C": 1.1}; nmach = {"A": 1, "B": 1, "C": 1} # 每批加工時間 (小時)、機台數
def sim(rule, rate, N=6000, seed=6):
    rng = np.random.default_rng(seed); ev = []; t_arr = 0.0
    for i in range(N): t_arr += rng.exponential(1 / rate); heapq.heappush(ev, (t_arr, 0, "arr", i))
    queue = {k: [] for k in ptime}; busy = {k: 0 for k in ptime}; step = {}; start = {}; done = {}; seq = 0
    def prio(lot): # 排序鍵：數值小者優先
        return {"FIFO": 0, "FBFS": step[lot], "LBFS": -step[lot]}[rule], seq_in[lot]
    seq_in = {}; area, last, wip = 0.0, 0.0, 0
    def try_start(st, t):
        nonlocal seq
        while busy[st] < nmach[st] and queue[st]:
            lot = min(queue[st], key=prio); queue[st].remove(lot); busy[st] += 1; seq += 1
            heapq.heappush(ev, (t + rng.gamma(4, ptime[st] / 4), seq, "fin", lot))
    while ev:
        t, _, kind, lot = heapq.heappop(ev); area += wip * (t - last); last = t
        if kind == "arr": step[lot] = 0; start[lot] = t; wip += 1
```

```

else:
    busy[route[step[lot]]] -= 1; st0 = route[step[lot]]; step[lot] += 1
    if step[lot] == len(route): done[lot] = t; wip -= 1; try_start(st0, t);
continue
    try_start(st0, t)
    st = route[step[lot]]; seq_in[lot] = t; queue[st].append(lot); try_start(st, t)
    ct = np.array([done[i] - start[i] for i in range(1000, N)]) # 去除暖機期
    return ct.mean(), np.percentile(ct, 95), area / last, len(done) / last
print(f"瓶頸（微影）每批負荷 {3 * ptime['A']:.1f} 小時；投料率 0.30 批/小時時利用率 {0.30 * 3 * ptime['A']:.0%}")
for rule in ["FIFO", "FBFS", "LBFS"]:
    m, p95, w, th = sim(rule, 0.30); print(f"{rule}: 平均週期時間 {m:.1f} 小時，95 百分位 {p95:.1f} 小時，平均在製品 {w:.1f} 批，產出 {th:.3f} 批/小時")
for r in [0.25, 0.30, 0.32]:
    m, _, w, th = sim("LBFS", r); print(f"投料率 {r} (利用率 {r * 3:.0%}) : LBFS 平均週期時間 {m:.1f} 小時，在製品 {w:.1f}，Little 定律 WIP ≈ 產出 × 週期 = {th * m:.1f}")

```

執行結果：投料率 0.30 批/小時時，瓶頸的利用率約 90%。FIFO 的平均週期時間約 33.7 小時、在製品約 10.4 批；FBFS 約 50.1 小時、16.5 批；LBFS 約 27.8 小時、8.9 批，95 百分位也從 FIFO 的約 72.2 小時降到約 55.6 小時。三種規則的產出率相同（約 0.306 批/小時），差別只在於晶圓在廠內等了多久。

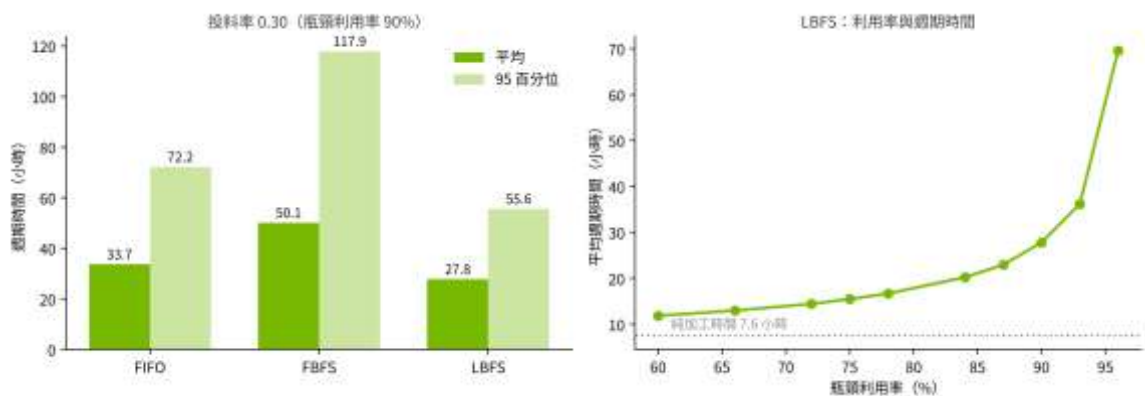


圖 27-5 重入式產線：派工規則與投料率對週期時間的影響

LBFS 讓接近完成的晶圓先走，避免在製品在後段堆積，在這個例子中週期時間最短；FBFS 則讓新投入的晶圓不斷插隊，在製品越積越多。文獻中，Lu 與 Kumar (1991) 等對重入式產線的研究也顯示，派工規則會影響這類系統的穩定性——即使每台機台的負荷都低於產能，不當的規則仍可能使在製品無限累積。不過實務上不會只用單一規則：交期、優先等級、批量與設定都需要同時考慮。

投料率的影響更大：利用率從 75% 提高到 90%，週期時間從約 15.5 小時增加到約 27.8 小時；到 96% 時暴增到約 69.6 小時。這正是第 8 章排隊理論所說的，利用率接近 100% 時等候時間急劇增加。各投料率下，在製品都與「產出率 × 週期時間」相符（例如 8.9 對 8.5、22.2 對 22.7），驗證了 Little 定律。這個例子的純加工時間約 7.6 小時，週期時間約為它的 3.7 倍，這個比值在晶圓廠常被稱

為「X 因子」，是衡量生產線流動效率的重要指標。

27.5.3 封測廠與電子組裝的派工

封裝測試廠的派工有不同的特性：工序較少，但產品種類多、批量小，測試機台需要搭配特定的測試板、測試程式與溫度設定，換線（換測試板、換溫度）往往是最主要的損失；封裝的打線與塑封則受模具與材料的限制。因此封測廠的派工常以「設定最少」與「交期」的平衡為核心，類似 27.2.2 節的範例，並結合測試機台的產能預測與良率回饋（第 25 章）。

不論是晶圓廠或封測廠，智慧派工的基礎都是即時、準確的資料：MES 的工單與在製品狀態、機台的即時狀態（第 17 章）、保養計畫與量測抽樣規則。派工系統的規則也要能被工程師理解與調整，並以模擬（第 21 章）評估規則變更的影響，再逐步上線。

案例

案例 27-1 教學情境：作廢的排程表

這是本章開頭的情境。試算表排程在第一個小時就被故障與急單打亂。先釐清目標（準時交貨為主、換線為輔），再以 MES 的即時資料與求解器產生基準排程，事件發生時只做局部重排並設定凍結區間。

啟示：排程是一個持續修正的過程；目標、資料與重排規則比演算法更先決。

案例 27-2 產業：晶圓廠的即時派工系統

依公開文獻，晶圓廠普遍採用即時派工系統，結合優先等級、CR、線平衡與設定等規則，並在瓶頸區（如微影）以排程最佳化輔助；規則的調整多以模擬驗證後再上線。

啟示：在極複雜的環境中，即時派工結合局部最佳化比一次求整廠最佳解更實際。

案例 27-3 產業：以求解器減少換線

一家食品廠的包裝線依訂單交期排程，換口味、換包材的時間佔去大量產能。導入限制規劃求解

器後，把交期與換線同時放進目標，並加入清洗與過敏原的換線規則，在交期表現不變下減少了換線時間。

啟示：把現場真正在意的成本寫進目標函數，是最佳化有價值的前提。

案例 27-4 產業：封測廠的測試排程

依公開報導，封測廠的測試機台常因換測試板與溫度設定而損失產能。以設定矩陣與交期為目標的排程，搭配測試時間的預測，能把相同設定的批次合併，同時避免急件延遲。

啟示：小批量、多品種時，設定時間往往是排程的核心。

案例 27-5 台灣情境：從白板到 APS

一家台灣的金屬加工廠，過去以白板與磁鐵排程。導入先進規劃與排程系統（APS）時，最大的挑戰不是軟體，而是主資料：途程、標準工時、機台能力與換線時間都不完整。花了三個月整理主資料後，排程系統才開始有用。

啟示：排程的品質受限於主資料的品質（第 17、23 章）。

案例 27-6 教學情境：強化學習的訓練陷阱

一個學生團隊以強化學習訓練派工策略，在模擬中表現遠勝傳統規則。但檢查後發現，模擬中的加工時間沒有變異、機台從不故障；加入真實的變異後，優勢幾乎消失，部分情境甚至不如 EDD。

啟示：學習型排程的成績受模擬器的真實度限制；要與強的基準在相同條件下比較。

實作活動

活動 27-1 Johnson 與 NEH

以 27.1 節的程式，改變工件數與機器數，比較 NEH 與窮舉最佳解的差距，並觀察窮舉所需時間如何隨工件數增加。

活動 27-2 派工規則比賽

以 27.3 節的程式，在 FT06 與自行產生的題目上比較各種派工規則的完工時間與總延遲，討論哪一種規則適合哪一種目標。

活動 27-3 CP-SAT 建模

把 27.3 節的零工式模型擴充為彈性零工式：每道工序有 2 台可選擇的機器、加工時間不同，以 CP-SAT 求解並與固定指派比較。

活動 27-4 換線與能源

在 27.2 節的範例中加入時間電價：尖峰時段每分鐘的成本較高，以 CP-SAT 同時最小化延遲、換線與電費，觀察排程如何避開尖峰。

活動 27-5 學習派工規則

在 27.4 節的程式中加入新的特徵（例如下一台機器的等候量），重新學習權重，並測試在交期較緊與較鬆的題目上是否仍然優於傳統規則。

活動 27-6 重入式產線

以 27.5 節的程式，加入一台批量加工的爐管與等候時間限制，比較不同派工規則與投料控制對週期時間與重工的影響。

延伸閱讀

- Michael L. Pinedo · Scheduling: Theory, Algorithms, and Systems (Springer) : 排程理論的標準教材。
- S. M. Johnson · 〈Optimal Two- and Three-stage Production Schedules with Setup Times Included〉 · Naval Research Logistics Quarterly · 1954 。
- Muhammad Nawaz 、E. Emory Enscore 與 Inyong Ham · 〈A Heuristic Algorithm for the m-Machine, n-Job Flow-shop Sequencing Problem〉 · Omega · 1983 。
- Lars Mönnch 、John W. Fowler 與 Scott J. Mason · Production Planning and Control for Semiconductor Wafer Fabrication Facilities (Springer · 2013) 。
- Wallace J. Hopp 與 Mark L. Spearman · Factory Physics (第 8 章與第 17 章亦曾引用) : 週期時間、在製品與 Little 定律。

表 27-7 本章思考地圖

本章概念	關鍵提問	後續延伸章節
問題分類	是流程型、零工式、彈性還是重入式？	第 8、18 章
排程目標	交期、換線、在製品與能源的優先順序？	第 34 章
派工與最佳化	規則與目標相配嗎？求解器能解多大？	第 26 章
學習型排程	模擬器夠真實嗎？與強基準比較了嗎？	第 29、36 章
動態重排	凍結區間與重排觸發條件是什麼？	第 19、21 章
晶圓與封測派工	瓶頸、重入、批量與設定如何處理？	第 37 章
資料基礎	主資料與即時狀態可信嗎？	第 17、23 章

章末小結

- 排程問題依機器環境分為單機、平行機、流程型、零工式、彈性零工式與重入式；多數為 NP-hard。兩機流程型可用 Johnson 法則求最佳解，多機可用 NEH；範例中 NEH 得到 184，距窮舉最佳 180 約 2%。
- 排程目標包括完工時間、交期、在製品、換線、能源與穩定性，常互相衝突。範例中 EDD 延遲少但換線多，依族群集中換線少但延遲大；把兩者都放進目標的 CP-SAT 排程同時做到延遲 0、換線 2 次。
- 派工規則快速易懂，但要與目標相配：FT06 上 MWKR 得到 61、SPT 得到 88，CP-SAT 求得最佳 55。數學規劃與限制規劃能求最佳解或下界；後設啟發式適合大規模問題，範例中模擬退火得到 848、CP-SAT 證明最佳 804。
- 機器學習可以學出派工規則，範例中學到的線性規則在新題目上的總延遲約 638，比 EDD 的約 708 少約一成；強化學習需要可信的模擬器與強基準比較；數位孿生支援動態重排與穩健性評估。
- 晶圓廠的重入式流程以即時派工為主，需考慮瓶頸、重入、批量、設定與等候時間限制。範例中 LBFS 的週期時間約 27.8 小時，短於 FIFO 的 33.7 小時與 FBFS 的 50.1 小時；利用率從 90% 提高到 96%，週期時間從約 27.8 增加到約 69.6 小時，在製品與 Little 定律相符。封測廠則以設定與交期的平衡為核心。

習題

一、選擇題

1. 所有工件依相同機器順序加工的工廠稱為？（A）Job Shop （B）Flow Shop （C）Open Shop （D）FJSP
2. 每道工序可在多台機器中選一台加工的問題是？（A）單機 （B）FJSP （C）Flow Shop （D）平行機

3. 兩部機器流程型工廠最小化完工時間的最佳法則是？(A) EDD (B) SPT (C) Johnson 法則 (D) NEH
4. 對最大延遲有利、依交期排序的規則是？(A) SPT (B) EDD (C) LPT (D) FIFO
5. 對平均流程時間有利的規則是？(A) SPT (B) LPT (C) MWKR (D) LBFS
6. 以剩餘交期 ÷ 剩餘工時排序的規則是？(A) CR (B) ATC (C) FIFO (D) SPT
7. 在製品 = 產出率 × 週期時間稱為？(A) Johnson 法則 (B) Little 定律 (C) Amdahl 定律 (D) Moore 定律
8. 晶圓多次回到同一組機台的流程稱為？(A) 平行機 (B) 重入式流程 (C) 開放型 (D) 單機
9. 以一定機率接受較差解以跳出局部最佳的方法是？(A) 窮舉法 (B) 模擬退火 (C) EDD (D) 線性迴歸
10. 重排時保持一段時間內的工作不變，稱為？(A) 凍結區間 (B) 投料控制 (C) 瓶頸保護 (D) 線平衡

二、簡答題

11. 說明 NP-hard 對排程實務的意義。
12. 為什麼交期、換線與在製品等目標常常互相衝突？舉例說明。
13. 比較派工規則與最佳化排程的優缺點。
14. 說明 MILP 與限制規劃在排程建模上的差異。
15. 學習型派工規則有什麼優點與風險？
16. 說明動態排程中凍結區間的作用。
17. 晶圓廠為什麼多採用即時派工，而不是一次求整廠最佳排程？
18. 說明 X 因子的意義，以及利用率對它的影響。

三、計算題

19. 兩部機器，工件加工時間 ($M1, M2$) : $A(3, 6)$ 、 $B(7, 2)$ 、 $C(4, 4)$ 、 $D(5, 3)$ 、 $E(2, 5)$ ，以 Johnson 法則排序並計算完工時間。
20. 單機上 4 個工件，加工時間 4、2、6、3，依 SPT 排序並計算平均流程時間。
21. 接上題，若交期分別為 6、4、12、5 (依原編號)，依 EDD 排序並計算總延遲。
22. 10 個工件的單機排序有幾種可能？若電腦每秒評估 100 萬種，窮舉需多久？
23. 某產線產出率每天 200 片，平均週期時間 30 天，平均在製品約多少？
24. 純加工時間 10 天，週期時間 35 天，X 因子為多少？
25. 兩個工件在同一台機器，加工 3 與 5 小時，交期 4 與 6 小時，比較兩種順序的總延遲。
26. 某派工規則的平均總延遲 708，新規則 638，改善百分比為多少？

四、申論與設計題

27. 為一座有 40 台 CNC 的零工式工廠設計排程系統：說明目標、資料來源、基準排程方法、即時派工與重排規則。
28. 評論以下說法：「導入 AI 排程之後，就不需要生管人員了。」
29. 為一座封測廠的測試區設計派工規則，說明如何兼顧交期、換線與測試機台利用率，並說明如何以模擬驗證。

五、是非題

30. SPT 規則對所有排程目標都表現最好。
31. 三部以上機器的流程型工廠，一般沒有像 Johnson 法則那樣簡單的最佳法則。
32. 利用率越接近 100%，週期時間會急劇增加。
33. CP-SAT 等求解器可以證明解是否為最佳。
34. 在模擬中表現好的強化學習策略，上線後一定表現好。
35. 排程系統的效果受主資料品質影響。

六、進階挑戰題

36. 以 OR-Tools 求解 FT10 (10×10) 與更大的 Taillard 題目，記錄求解時間與差距，並與模擬退火、派工規則比較。
37. 以圖神經網路表示零工式工廠的工序關係，用強化學習訓練派工策略，並在不同規模的題目上測試泛化能力。
38. 建立一個結合故障與加工時間變異的離散事件模擬，比較靜態排程、每班重排與事件觸發局部重排的交期表現與排程穩定性。

七、補充題

39. 列表 27-3 中的七種排程目標。
40. 列表 27-6 中晶圓廠派工的七項考量。

提示與參考解答

1. B。
2. B。
3. C。
4. B。
5. A。
6. A。
7. B。
8. B。
9. B。
10. A。
11. 參考：問題規模增大時，沒有已知方法能在多項式時間保證求得最佳解；實務上需在解的品質與計算時間之間取捨，使用規則、近似方法或限時求解。

12. 參考：依交期排序可能讓不同產品族交錯而增加換線；依族群集中減少換線卻使急單等待；提高利用率則增加在製品與週期時間。
13. 參考：派工規則快速、易懂、能即時反應，但不保證整體最佳；最佳化能兼顧多目標與限制，但需要較多計算與完整資料，對變動的反應較慢。
14. 參考：MILP 以二元變數與大 M 限制描述先後關係，適合含成本與線性限制的模型；限制規劃有區間變數與不重疊等專用限制，搜尋時能有效刪除不可行組合，對排程常更有效率。
15. 參考：優點是快速、可針對情境調整、可解釋；風險是只適用於訓練情境，條件改變時可能失準，需要重新學習與驗證。
16. 參考：近期已開工或已備料的工作不變，避免頻繁重排造成現場混亂，維持排程的穩定性與可執行性。
17. 參考：步驟多、機台多、重入、批量、設定與等候時間限制使問題極為龐大，且變動頻繁；即時派工能依最新狀態快速決策，再以局部最佳化輔助瓶頸。
18. 參考： $X \text{ 因子} = \text{週期時間} \div \text{純加工時間}$ ，衡量等候的多寡；利用率越高，等候越多， X 因子越大，接近 100% 時急劇增加。
19. $M1 \leq M2$ 的工件： $E(2)$ 、 $A(3)$ 、 $C(4)$ ，依 $M1$ 遞增；其餘 $D(M2 = 3)$ 、 $B(M2 = 2)$ ，依 $M2$ 遞減。順序 E 、 A 、 C 、 D 、 B 。 $M1$ 完成時間 2、5、9、14、21； $M2$ 完成時間 7、13、17、20、23，完工時間 23。
20. SPT 順序：2、3、4、6；完成時間 2、5、9、15，平均流程時間 $(2 + 5 + 9 + 15) \div 4 = 7.75$ 。
21. EDD 順序：工件 2 (交期 4)、工件 4 (5)、工件 1 (6)、工件 3 (12)；完成時間 2、5、9、15；延遲 0、0、3、3，總延遲 6。
22. $10! = 3,628,800$ 種，約 3.6 秒。
23. $200 \times 30 = 6,000$ 片。
24. $35 \div 10 = 3.5$ 。

25. 先 3 後 5：完成 3、8，延遲 $0 + 2 = 2$ ；先 5 後 3：完成 5、8，延遲 $(5 - 6 \rightarrow 0) + (8 - 4 = 4) = 4$ 。前者較佳。
26. $(708 - 638) \div 708 \approx 9.9\%$ 。
27. 參考：以準時交貨為主、換線與利用率為輔；從 MES、ERP 取得工單、途程、機台狀態；每日以求解器產生基準排程；現場以規則即時派工；故障或急單時觸發局部重排，設凍結區間，並記錄變更。
28. 參考：不同意。排程系統依賴目標、主資料與規則的設定，例外狀況、客戶協調與跨部門取捨仍需人員判斷；生管的角色會轉向管理目標、資料與例外。
29. 參考：以交期分級為主，同等級中優先選擇與目前設定相同的批次；限制連續相同設定的時間以免急件等待；以測試時間預測與產能模擬評估規則，並以歷史資料重播比較。
30. 否，SPT 對完工時間與交期不一定好。
31. 是。
32. 是。
33. 是，在求解完成時可證明最佳，或給出下界與差距。
34. 否，受模擬器真實度限制。
35. 是。
36. 提示：FT10 的最佳完工時間在文獻中為 930；記錄求解器的下界與上界隨時間的變化。
37. 提示：以工序為節點、先後關係與機器共用為邊；比較在 6×6 、 10×10 、 20×15 題目上的表現與傳統規則。
38. 提示：以相同的隨機事件序列公平比較；穩定性可以「重排後改變開始時間的工序數」衡量。
39. 完工時間、交期、流程時間與在製品、換線、設備利用率、能源與尖峰負載、穩定性。
40. 交期與優先等級、瓶頸保護、重入與線平衡、批量加工、設定與光罩、等候時間限制、機台限制與保養。